



APEX YAML Syntax Reference Guide

Version: 2.4 (Dual Format Support - APEX 2.2+) **Date:** 2026-01-17 **Author:** Mark Andrew Ray-Smith Cityline Ltd **Change Summary:** Added dual format support for queries, operations, and endpoints (map format and array format); maintained 100% backward compatibility; enhanced metadata capabilities

SYNTAX VERIFIED: This document has been updated and verified to use the correct APEX SpEL syntax. APEX processes HashMap data where fields are accessed using `#fieldName` syntax, NOT `#fieldName`. All examples in this document use the correct `#fieldName` syntax for field access in APEX YAML configurations.

Table of Contents

This section provides a definitive reference for APEX YAML keywords based on actual APEX core engine implementation. Approximately **155 keywords** are defined in apex-core, with **~140 functionally implemented** in execution logic. See Appendix C for planned/future keywords.

NEW

DUAL FORMAT SUPPORT (APEX 2.2+): The **queries**, **operations**, and **endpoints** fields now support both **map format** (legacy, concise) and **array format** (new, metadata-rich). Both formats are fully supported and can be mixed in the same file. See Section 2.3 for complete details.

1.1 Complete Keyword Reference Table

Keyword	Category	Required	Type	Description
actions-false	TransformationRule	No	List	Actions to execute when transformation rule condition evaluates to false (alias for else-actions)
actions-true	TransformationRule	No	List	Actions to execute when transformation rule condition evaluates to true (takes precedence over actions)
authentication	DataSource	No	Map	Authentication configuration for external data sources
author	Metadata	Conditional	String	Author of the configuration (required for rule-config, enrichment)
base-path	DataSource	No	String	Base file system path for file-based data sources
bootstrap-servers	DataSource	No	String	Kafka bootstrap servers (connection-level property)
business-domain	Rule	No	String	Business domain classification
business-owner	Rule	No	String	Business owner responsible for the rule
cache	DataSource	No	Map	Caching configuration for data sources
calculation-config	Enrichment	No	List	Configuration for calculation-enrichment type

Keyword	Category	Required	Type	Description
categories	Document	No	List	Category definitions for the configuration
category	Rule	No	String	Single category for rule classification
circuit-breaker	DataSource	No	Map	Circuit breaker configuration for resilience
component-refs	Component	No	List	References to other component files
config-files	Component	No	List	Configuration files with execution order and failure policy
condition	Rule/Enrichment/Stage	No	String	SpEL expression defining when rule/enrichment/stage applies
conditional-mappings	Enrichment	No	List	Conditional field mapping configurations
config-file	Stage	Yes	String	Path to rule configuration file for stage
connection	DataSource	Yes	Map	Database/external system connection configuration
connection-pool	DataSource	No	Map	Connection pool settings for database sources
created	Metadata	No	String	Creation timestamp (ISO 8601 format)
created-by	Rule	No	String	Creator identifier
custom-properties	Rule	No	Map	Custom extensible properties for rules
custom-validators	ValidationConfig	No	List	Custom validation logic references
debug-mode	RuleGroup	No	Boolean	Enable debug mode for rule group execution
data-sinks	Document	No	List	Output destinations for processed data

Keyword	Category	Required	Type	Description
data-source-refs	Document	No	List	References to external data source configurations
data-sources	Document	No	List	Inline data source definitions
data-types	Scenario	No	List	Data types this scenario applies to
default-value	FieldMapping	No	Any	Fallback value when source field is missing/null
depends-on	Stage	No	List	List of stage dependencies
description	Metadata	No	String	Human-readable description of the configuration
effective-date	Rule	No	String	Date when rule becomes effective (ISO 8601)
enabled	Rule/Enrichment	No	Boolean	Whether the rule/enrichment is active
encoding	DataSource	No	String	Character encoding for file-based sources
endpoints	DataSource	No	Map	REST API endpoint definitions
enrichment-group	EnrichmentGroup	No	String	Singular reference to another enrichment group for hierarchical composition
enrichment-refs	Component	No	List	References to enrichment configuration files
enrichments	Document	No	List	Data enrichment configurations
error-handling	RuleGroup/EnrichmentGroup	No	String	Exception handling strategy: "fail-fast" (default), "continue-on-error", "skip-on-error"

Keyword	Category	Required	Type	Description
error-recovery	Document	No	Map	Error recovery configuration for resilience and fault tolerance
execution-order	Stage/FileRef	No	Integer	Numeric execution order for stage or file reference
execution-settings	Enrichment	No	Map	Execution behavior configuration for enrichments
expiration-date	Rule	No	String	Date when rule expires (ISO 8601)
failure-policy	Stage/FileRef	No	String	Stage or file reference failure handling policy (terminate, continue-with-warnings, flag-for-review)
field-mappings	Enrichment	No	List	Field mapping configurations for enrichments
field-types	ValidationConfig	No	Map	Expected data types for rule validation
file	FileRef	Yes	String	Path to configuration file in component file reference
file-format	DataSource	No	Map	File format configuration (CSV, JSON, XML)
file-pattern	DataSource	No	String	File name pattern for file-based sources
health-check	DataSource	No	Map	Health check configuration for data sources
id	Metadata	Yes	String	Unique identifier for the configuration
implementation	DataSource	No	String	Implementation class for custom data sources
key-patterns	DataSource	No	Map	Key pattern definitions for key-value stores

Keyword	Category	Required	Type	Description
last-modified	Metadata	No	String	Last modification timestamp (ISO 8601)
lookup-config	Enrichment	No	Map	Configuration for lookup-enrichment type
mapping-rules	Enrichment	No	List	Complex mapping rule definitions
message	Rule	No	String	Message displayed when rule is triggered (condition=true). Supports <code>{{#expr}}</code> and <code>#{expr}</code> placeholders
metadata	Document	Yes	Map	Document metadata section
name	Metadata	No	String	Human-readable name for the configuration
no-match-message	Rule	No	String	Message displayed when rule does not match (condition=false). If omitted, uses <code>message</code> . Supports same placeholders as <code>message</code>
operations	DataSource	No	Map	Operation definitions for REST APIs
operator	RuleGroup	No	String	Logical operator for rule group (AND/OR)
override-priority	RuleReference	No	Integer	Override priority for rule within group
parallel-execution	RuleGroup	No	Boolean	Enable parallel execution of rules in group
parameter-names	DataSource	No	Array	Parameter names for parameterized queries
rule-configurations	Component	No	List	References to rule configuration files
pipeline	Document	No	Map	Pipeline configuration for processing

Keyword	Category	Required	Type	Description
polling-interval	DataSource	No	Integer	Polling interval for file-based sources
priority	Rule/Enrichment	No	Integer	Execution priority (lower numbers = higher priority)
processing-stages	Scenario	No	List	Stage-based processing configuration
queries	DataSource	No	Map	Named query definitions for database sources
required	FieldMapping	No	Boolean	Whether field mapping is mandatory
required-fields	ValidationConfig	No	List	List of required fields for rule validation
response-mapping	DataSource	No	Map	Response transformation configuration
rule-chains	Document	No	List	Rule chain definitions
rule-group-references	RuleGroup	No	List	References to other rule groups
rule-groups	Document	No	List	Rule group definitions
enrichment-groups	Document	No	List	Enrichment group definitions
#ruleResults	ContextVariable	Auto	Map	Access to individual rule evaluation results (key: rule-id, value: boolean)
#ruleGroupResults	ContextVariable	Auto	Map	Access to rule group evaluation results with passed/failed status
rule-id	RuleReference	Yes	String	ID of rule being referenced
rule-ids	RuleGroup	No	List	List of rule IDs in the group
rule-references	RuleGroup	No	List	Detailed rule references with metadata
rule-refs	Document	No	List	References to external rule configurations
rules	Document	No	List	Rule definitions

Keyword	Category	Required	Type	Description
sasl-mechanism	DataSource	No	String	SASL mechanism for Kafka authentication (connection-level)
scenario-id	Scenario	Yes	String	Unique identifier for the scenario
security-protocol	DataSource	No	String	Security protocol for Kafka connections (connection-level)
sequence	RuleReference	No	Integer	Execution sequence for rule within group
severity	Rule	No	String	Severity level (ERROR, WARNING, INFO)
source-field	FieldMapping	Yes	String	Source field name in field mappings
stage-metadata	Stage	No	Map	Additional metadata for stage
stage-name	Stage	Yes	String	Unique identifier for processing stage
stop-on-first-failure	RuleGroup	No	Boolean	Stop group execution on first rule failure
source-type	DataSource	Yes	String	Type of data source (database, rest-api, file, etc.)
tags	Metadata	No	List	Classification tags for the configuration
target-field	FieldMapping	Yes	String	Target field name in field mappings
target-type	Enrichment	No	String	Target object type for enrichment
topics	DataSource	No	Map	Kafka topic definitions
expression	FieldMapping	No	String	SpEL expression for field transformation
transformation-rules	Transformation	No	List	Transformation rule definitions
transformations	Document	No	List	Data transformation configurations

Keyword	Category	Required	Type	Description
type	Metadata	Yes	String	Document type (rule-config, enrichment, dataset, etc.)
validation	Rule	No	Map	Validation configuration for rules
version	Metadata	No	String	Version identifier for the configuration

1.2 Document Types

Valid values for the `type` field in metadata:

- `rule-config` - Rule configuration documents
- `enrichment` - Enrichment configuration documents
- `dataset` - Dataset configuration documents
- `scenario` - Scenario configuration documents
- `scenario-registry` - Scenario registry documents
- `component` - Component configuration documents (groups multiple config files)
- `external-data-config` - External data configuration documents
- `pipeline-config` - Pipeline configuration documents
- `bootstrap` - Bootstrap configuration documents
- `rule-chain` - Rule chain configuration documents

1.3 Required Fields by Document Type

Document Type	Required Fields
<code>rule-config</code>	<code>id</code> , <code>type</code> , <code>author</code>
<code>enrichment</code>	<code>id</code> , <code>type</code> , <code>author</code>
<code>dataset</code>	<code>id</code> , <code>type</code>
<code>scenario</code>	<code>id</code> , <code>type</code> , <code>business-domain</code>
<code>external-data-config</code>	<code>id</code> , <code>type</code> , <code>author</code>

2. Introduction & Overview

What is APEX YAML

APEX YAML is a declarative configuration language for the APEX Rules Engine that enables business users and developers to define data validation rules, enrichment logic, and business processes without writing code. It combines the simplicity of YAML with the power of Spring Expression Language (SpEL) to create maintainable, testable business logic.

Key Principles

- **Declarative:** Describe what you want, not how to achieve it
- **Readable:** Business-friendly syntax that non-developers can understand
- **Powerful:** Full access to SpEL expressions and Java functionality
- **Maintainable:** Clear structure with separation of concerns
- **Testable:** Configuration can be validated and tested independently
- **Modular:** External data-source references enable clean architecture and reusable components

Design Philosophy

APEX YAML follows these core principles:

1. **Data-Driven:** All logic operates on HashMap data context using direct field references (`#fieldName`)
2. **Expression-Based:** Conditions and calculations use SpEL expressions
3. **Type-Safe:** Strong typing with automatic type conversion
4. **Null-Safe:** Built-in null safety with optional navigation operators
5. **Performance-Oriented:** Optimized for high-throughput processing

Critical Syntax Note

 **Field Access Syntax:** APEX processes HashMap data where fields are accessed using `#fieldName` syntax, **NOT** `#fieldName`. This is the correct syntax for all APEX YAML configurations:

- **Correct:** `#currencyCode != null`
- **X Incorrect:** `#currencyCode != null`
- **Correct:** `lookup-key: "#customerId"`
- **X Incorrect:** `lookup-key: "#customerId"`

Relationship to Spring Expression Language (SpEL)

APEX YAML leverages SpEL for all expressions, providing:

- Mathematical operations and functions
- String manipulation and regex support
- Date/time operations
- Java class and method access
- Collection operations
- Conditional logic (ternary operators)

Document Structure Overview

Every APEX YAML document follows this structure:

```
metadata:
  # Document identification and configuration

data-source-refs: # Optional: External data-source references
  # References to external infrastructure configurations

pipeline: # Optional: Pipeline orchestration
  # Complete ETL/data processing workflows
```

```
rules:
  # Validation and business rules

enrichments:
  # Data enrichment logic

data-sources: # Optional: Inline data-source configurations
  # Direct data-source configurations (legacy approach)
```

Clean Architecture with External References

APEX 2.0 introduces **external data-source references** that enable clean separation of concerns:

- **Infrastructure Configuration:** External, reusable data-source configurations
- **Business Logic Configuration:** Lean, focused enrichment and validation rules
- **Configuration Caching:** External configurations cached for performance
- **Enterprise Scalability:** Shared infrastructure across multiple rule configurations

2.3 Dual Format Support (APEX 2.2+)

Overview

APEX 2.2 introduces **dual format support** for **queries**, **operations**, and **endpoints** fields in data source configurations. This enhancement provides flexibility to choose between concise map-based syntax or metadata-rich array-based syntax, depending on your project needs.

Key Benefits:

- **100% Backward Compatible:** All existing map format configurations work unchanged
- **Rich Metadata:** Array format enables descriptions, tags, and custom properties
- **Better Documentation:** Self-documenting configurations improve team collaboration
- **Mixed Format:** Both formats can coexist in the same file
- **Zero Performance Impact:** Transparent deserialization with minimal overhead
- **Enterprise Ready:** Supports governance, versioning, and compliance

Quick Decision Matrix

Your Scenario	Recommended Format
Simple queries, small team	Map format
Enterprise projects with governance	Array format
Need versioning/deprecation tracking	Array format
Rapid prototyping	Map format
Multi-team collaboration	Array format
Compliance/audit requirements	Array format

Your Scenario	Recommended Format
Personal/internal projects	Map format

Format Comparison

Map Format (Concise):

```
queries:
  customerProfile: "SELECT * FROM customers WHERE id = :id"
  getAllActive: "SELECT * FROM customers WHERE status = 'ACTIVE'"
```

Characteristics:

- Concise and readable
- Fast Map.get() lookups
- Simple YAML structure
- Low memory footprint
- No metadata
- Limited documentation

Array Format (Metadata-Rich):

```
queries:
  - name: "customerProfile"
    value: "SELECT * FROM customers WHERE id = :id"
    description: "Retrieve customer profile by unique ID"
    tags: ["customer", "read", "pii-data"]
    owner: "customer-team"
    version: "1.0"
```

Characteristics:

- Rich metadata
- Self-documenting
- Versioning support
- Tagging and categorization
- Compliance tracking
- ⚠ More verbose
- ⚠ Slightly higher memory during deserialization

When to Use Each Format

Use Map Format When:

1. Building Small Projects or Prototypes

- Team size: 1-5 developers
- Configuration files are simple and self-explanatory
- Rapid iteration is priority

2. Queries Are Self-Explanatory

```
queries:
  getById: "SELECT * FROM users WHERE id = :id"
  deleteUser: "DELETE FROM users WHERE id = :id"
```

3. Speed of Configuration Matters

- Quick setup without ceremony
- Minimal typing required

Use Array Format When:

1. Working on Enterprise Projects

- Multiple teams collaborate on same codebase
- Queries require documentation for maintainability
- Configuration files shared across departments

2. Compliance Requirements

```
queries:
  - name: "getCustomerData"
    value: "SELECT * FROM customers WHERE id = :id"
    tags: ["pii-data", "gdpr-compliant"]
    compliance: ["GDPR", "CCPA"]
    retention: "7-years"
```

3. API Versioning and SLA Tracking

```
endpoints:
  - name: "currencyLookup"
    value: "/api/v2/currency/{code}"
    version: "2.0"
    deprecated: false
    sla: "99.9%"
```

4. Complex Queries Requiring Context

```
queries:
  - name: "complexRiskAssessment"
```

```

value: |
  SELECT
    c.customer_id,
    c.credit_score,
    r.risk_rating,
    COUNT(d.default_id) as default_count
  FROM customers c
  LEFT JOIN risk_profiles r ON c.customer_id = r.customer_id
  LEFT JOIN defaults d ON c.customer_id = d.customer_id
  WHERE c.customer_id = :customerId
  GROUP BY c.customer_id, c.credit_score, r.risk_rating
description: "Comprehensive risk assessment for credit decisions"
parameters: ["customerId"]
tags: ["risk-management", "credit-analysis", "financial"]
owner: "risk-team"
performance: "indexed"

```

Map Format (Legacy - Fully Supported)

The traditional key-value format remains the recommended choice for simple, concise configurations:

```

data-sources:
- name: "customer-db"
  type: "database"
  source-type: "postgresql"

queries:
  getCustomer: "SELECT * FROM customers WHERE id = :id"
  listActive: "SELECT * FROM customers WHERE status = 'ACTIVE'"
  countAll: "SELECT COUNT(*) FROM customers"

operations:
  createCustomer: "INSERT INTO customers (name, email) VALUES (:name, :email)"
  updateCustomer: "UPDATE customers SET email = :email WHERE id = :id"
  deleteCustomer: "DELETE FROM customers WHERE id = :id"

```

Features:

- **Simple key-value pairs**
- **Direct lookup:** `Map.get("queryName")`
- **Minimal memory overhead**
- **Works with all APEX versions**

Array Format (New - Recommended for Enterprise)

The new array format provides rich metadata for better documentation and discoverability:

```
queries:
  - name: "getCustomer"
    value: "SELECT * FROM customers WHERE id = :id"
    description: "Retrieve customer profile by unique ID"
    tags: ["customer-management", "primary-lookup"]
    performance: "indexed"
    owner: "customer-team"

  - name: "listOrders"
    value: "SELECT * FROM orders WHERE customer_id = :customerId"
    description: "Fetch all orders for a specific customer"
    tags: ["order-management", "batch-query"]
    cacheable: true
    cache-ttl: "300s"

operations:
  - name: "insertOrder"
    value: "INSERT INTO orders (id, amount) VALUES (:id, :amount)"
    description: "Create new order record in database"
    tags: ["order-lifecycle", "write-operation"]
    transaction: "required"
    audit: true

endpoints:
  - name: "paymentGateway"
    value: "https://api.payments.com/v1/process"
    description: "Third-party payment processing endpoint"
    tags: ["payments", "external-api"]
    method: "POST"
    timeout: "5000ms"
    retry-policy: "exponential-backoff"
```

Array Format Field Reference

Field	Type	Required	Description
name	String	Yes	Unique identifier (becomes Map key)
value	String	Yes	The actual query/operation/endpoint
description	String	No	Human-readable description
tags	List	No	Categorization tags
version	String	No	Version number (e.g., "1.0", "2.1")
owner	String	No	Team or person responsible
deprecated	Boolean	No	Deprecation flag
deprecationMessage	String	No	Why deprecated, what to use instead
parameters	List	No	List of parameter names

Field	Type	Required	Description
compliance	List	No	Compliance frameworks (GDPR, PCI-DSS)
performance	String	No	Performance notes (e.g., "indexed")
sla	String	No	Service level agreement (for endpoints)

Additional Custom Fields:

- `cacheable`: Whether results can be cached
- `cache-ttl`: Cache time-to-live
- `transaction`: Transaction requirements
- `audit`: Audit logging enabled
- `method`: HTTP method for endpoints
- `timeout`: Timeout configuration
- `retry-policy`: Retry behavior

Mixed Format (Best of Both Worlds)

Both formats can coexist in the same configuration file:

```
data-sources:
- name: "customer-service"
  type: "database"
  source-type: "postgresql"

# Simple queries - use map format
queries:
  getById: "SELECT * FROM customers WHERE id = :id"
  getAll: "SELECT * FROM customers"
  countAll: "SELECT COUNT(*) FROM customers"

# Complex operations with metadata - use array format
operations:
- name: "createCustomerWithAudit"
  value: |
    WITH inserted AS (
      INSERT INTO customers (name, email, status)
      VALUES (:name, :email, 'PENDING')
      RETURNING customer_id
    )
    INSERT INTO audit_log (entity_type, entity_id, action, user_id)
    SELECT 'customer', customer_id, 'CREATE', :userId
    FROM inserted
  description: "Create customer and log audit trail"
  parameters: ["name", "email", "userId"]
  tags: ["customer", "write", "audit"]
  version: "1.0"
  compliance: ["SOX", "GDPR"]
```

Best Practice for Mixed Format:

Rule of Thumb: Use map format for simple queries, array format for queries that benefit from documentation.

```
queries:
  # Simple, self-explanatory → map format
  simpleCount: "SELECT COUNT(*) FROM users"

  # Complex, needs context → array format
  - name: "complexReport"
    value: "SELECT dept, COUNT(*) FROM employees GROUP BY dept"
    description: "Department staffing report for HR analytics"
    tags: ["hr", "analytics"]
```

Implementation Details

Technical Architecture

Deserialization Flow:

1. **Jackson encounters field** (`queries`, `operations`, or `endpoints`)
2. **Custom deserializer inspects JSON node type:**
 - **Object node** → Map format → Extract as `Map<String, String>`
 - **Array node** → Array format → Parse objects, build `Map<String, String>`
3. **Result:** Runtime code sees identical `Map<String, String>` interface
4. **Impact:** Zero changes to existing lookup logic

Core Components:

Custom Jackson Deserializers:

- `FlexibleQueriesDeserializer.java` - Handles queries field
- `FlexibleOperationsDeserializer.java` - Handles operations field
- `FlexibleEndpointsDeserializer.java` - Handles endpoints field

Model Classes:

- `NamedQuery.java` - Array format query model
- `NamedOperation.java` - Array format operation model
- `NamedEndpoint.java` - Array format endpoint model

Integration Points:

- `YamlDataSource.java` - Uses `@JsonDeserialize` annotations
- `YamlDataSink.java` - Uses `@JsonDeserialize` annotations

Runtime Behavior

All runtime code is unchanged:

```
// Query lookup - works identically for both formats
String query = dataSourceConfig.getQueries().get("customerProfile");

// Endpoint lookup - works identically for both formats
String endpoint = endpoints.get("currencyLookup");

// Operation lookup - works identically for both formats
String operation = operations.get("createCustomer");
```

Key Insight: Format conversion happens during deserialization only. Runtime uses fast Map lookups.

Migration Guide

Step 1: Identify Candidates for Migration

Good Candidates for Array Format:

- Complex queries with many parameters
- Queries used across multiple teams
- Queries requiring compliance documentation
- Deprecated queries needing replacement guidance
- Versioned APIs or queries

Keep as Map Format:

- Simple one-liner queries
- Internal/private queries
- Prototype configurations

Step 2: Convert Query to Array Format

Before (Map Format):

```
queries:
  getCustomerRisk: "SELECT customer_id, risk_score FROM risk_profiles WHERE
customer_id = :id"
```

After (Array Format):

```
queries:
  - name: "getCustomerRisk"
    value: "SELECT customer_id, risk_score FROM risk_profiles WHERE customer_id =
:id"
    description: "Retrieve risk score for credit decision workflows"
    parameters: ["id"]
    tags: ["risk-management", "credit"]
```

```
owner: "risk-team"
version: "1.0"
```

Step 3: Add Metadata Incrementally

Start simple:

```
queries:
  - name: "getCustomerRisk"
    value: "SELECT customer_id, risk_score FROM risk_profiles WHERE customer_id = :id"
    description: "Retrieve risk score for credit decision workflows"
```

Add metadata as needed:

```
queries:
  - name: "getCustomerRisk"
    value: "SELECT customer_id, risk_score FROM risk_profiles WHERE customer_id = :id"
    description: "Retrieve risk score for credit decision workflows"
    parameters: ["id"]
    tags: ["risk-management", "credit"]
    owner: "risk-team"
    version: "1.0"
    compliance: ["BASEL-III"]
```

Step 4: Test Both Formats

```
# Test file: migration-test.yaml
data-sources:
  - name: "test-source"
    type: "database"

# Old queries (map format) - should still work
queries:
  oldQuery: "SELECT * FROM table1"

# New queries (array format) - should work too
operations:
  - name: "newOperation"
    value: "INSERT INTO table2 VALUES (:value)"
    description: "Test array format"
```

Migration Timeline (Optional)

Phase 1 (Week 1-2): Identify high-value queries

Phase 2 (Week 3-4): Convert complex queries with metadata

Phase 3 (Month 2+): Gradual adoption based on team preference

No Pressure: Migration is entirely optional. Map format remains fully supported.

Best Practices

1. Choose Format Based on Complexity

```
# GOOD: Simple query → map format
queries:
  getUser: "SELECT * FROM users WHERE id = :id"

# GOOD: Complex query → array format
queries:
  - name: "getUserWithProfile"
    value: |
      SELECT u.*, p.bio, p.avatar
      FROM users u
      LEFT JOIN profiles p ON u.id = p.user_id
      WHERE u.id = :id
    description: "Get user with profile data for dashboard"
    tags: ["user-management"]
```

2. Use Consistent Naming Conventions

```
# GOOD: Consistent naming
queries:
  getCustomer: "..."/>
  getCustomerById: "..."/>
  listActiveCustomers: "..."/>

# BAD: Inconsistent naming
queries:
  customer: "..."/>
  fetchCustomerById: "..."/>
  active_cust_list: "..."/>
```

3. Document Parameters

```
# GOOD: Parameters documented
queries:
  - name: "searchCustomers"
    value: "SELECT * FROM customers WHERE name LIKE :searchTerm AND status = :status"
```

```

    parameters: ["searchTerm", "status"]
    description: "Search customers by name and status"

# MISSING: No parameter documentation
queries:
  - name: "searchCustomers"
    value: "SELECT * FROM customers WHERE name LIKE :searchTerm AND status =
:status"

```

4. Use Tags for Organization

```

queries:
  - name: "getCustomer"
    value: "SELECT * FROM customers WHERE id = :id"
    tags: ["customer-management", "read-operation", "indexed"]

  - name: "createCustomer"
    value: "INSERT INTO customers (...) VALUES (...)"
    tags: ["customer-management", "write-operation", "audited"]

```

5. Mark Deprecated Queries

```

queries:
  - name: "getCustomerLegacy"
    value: "SELECT * FROM customers_old WHERE id = :id"
    deprecated: true
    deprecationMessage: "Use 'getCustomer' instead. This table will be removed in
v3.0"
    version: "1.0"

```

6. Version Your Queries

```

queries:
  - name: "getCustomerV1"
    value: "SELECT id, name FROM customers WHERE id = :id"
    version: "1.0"
    deprecated: true
    deprecationMessage: "Use getCustomerV2 for enhanced profile data"

  - name: "getCustomerV2"
    value: "SELECT id, name, email, created_at FROM customers WHERE id = :id"
    version: "2.0"

```

Error Handling

The dual format deserializer provides clear error messages for common issues:

Duplicate Key Detection

Error Scenario:

```
queries:
  - name: "getCustomer"
    value: "SELECT * FROM customers WHERE id = :id"
  - name: "getCustomer" # Duplicate key
    value: "SELECT * FROM customers WHERE email = :email"
```

Error Message:

Duplicate query name 'getCustomer' found in array format queries.
Each query must have a unique name within the same data source.

Missing Required Fields

Error Scenario:

```
queries:
  - name: "getCustomer"
    # Missing 'value' field
    description: "Get customer by ID"
```

Error Message:

Query object is missing required field 'value' at index 0.
Each query must have both 'name' and 'value' fields.

Error Scenario:

```
queries:
  - value: "SELECT * FROM customers WHERE id = :id"
    # Missing 'name' field
```

Error Message:

Query object is missing required field 'name' at index 0.
Each query must have both 'name' and 'value' fields.

Invalid Format Type

Error Scenario:

```
queries: "SELECT * FROM customers" # String instead of map or array
```

Error Message:

```
Invalid format for 'queries' field.  
Expected either:  
- Map format: { "queryName": "query string" }  
- Array format: [ { "name": "queryName", "value": "query string" } ]  
Got: String value
```

Complete Examples

Example 1: Financial Services - Risk Management

```
metadata:  
  name: "Risk Assessment Queries"  
  version: "2.1"  
  type: "data-source-config"  
  
data-sources:  
  - name: "risk-database"  
    type: "database"  
    source-type: "postgresql"  
  
queries:  
  # Simple lookup queries - map format  
  getCustomerId: "SELECT customer_id FROM customers WHERE account_number =  
:accountNumber"  
  
  # Complex risk queries - array format with compliance metadata  
  - name: "comprehensiveRiskAssessment"  
    value: |  
      SELECT  
        c.customer_id,  
        c.credit_score,  
        c.credit_limit,  
        c.outstanding_balance,  
        r.risk_rating,  
        r.risk_factors,  
        COUNT(d.default_id) as historical_defaults  
      FROM customers c
```

```

LEFT JOIN risk_profiles r ON c.customer_id = r.customer_id
LEFT JOIN defaults d ON c.customer_id = d.customer_id
  AND d.default_date > CURRENT_DATE - INTERVAL '5 years'
WHERE c.customer_id = :customerId
  AND c.status = 'ACTIVE'
GROUP BY c.customer_id, c.credit_score, c.credit_limit,
         c.outstanding_balance, r.risk_rating, r.risk_factors
description: "Comprehensive risk assessment for credit decisions"
parameters: ["customerId"]
tags: ["risk-management", "credit-analysis", "basel-compliant"]
version: "2.1"
compliance: ["BASEL-III", "IFRS-9"]
owner: "risk-analytics-team"
performance: "indexed on customer_id, includes 5-year lookback"

```

Example 2: E-Commerce - Customer Service

```

data-sources:
- name: "customer-service-db"
  type: "database"
  source-type: "mysql"

queries:
  # Basic queries - map format
  getOrderCount: "SELECT COUNT(*) FROM orders WHERE customer_id = :customerId"

  # Customer service queries - array format
  - name: "customerServiceProfile"
    value: |
      SELECT
        c.customer_id,
        c.name,
        c.email,
        c.loyalty_tier,
        COUNT(DISTINCT o.order_id) as total_orders,
        SUM(o.total_amount) as lifetime_value,
        MAX(o.order_date) as last_order_date,
        COUNT(DISTINCT t.ticket_id) as support_tickets,
        AVG(t.satisfaction_score) as avg_satisfaction
      FROM customers c
      LEFT JOIN orders o ON c.customer_id = o.customer_id
      LEFT JOIN support_tickets t ON c.customer_id = t.customer_id
      WHERE c.customer_id = :customerId
      GROUP BY c.customer_id, c.name, c.email, c.loyalty_tier
    description: "Complete customer profile for service representatives"
    parameters: ["customerId"]
    tags: ["customer-service", "dashboard", "analytics"]
    owner: "customer-experience-team"
    version: "1.5"

```

Example 3: REST API Integration

```

data-sources:
  - name: "currency-api"
    type: "rest-api"
    base-url: "https://api.exchangerate.com"

endpoints:
  # Simple endpoints - map format
  healthCheck: "/health"

  # Versioned API endpoints - array format
  - name: "currencyExchangeRate"
    value: "/v2/rates/{baseCurrency}/{targetCurrency}"
    description: "Get exchange rate between two currencies"
    version: "2.0"
    tags: ["currency", "exchange-rate", "public-api"]
    sla: "99.9%"
    deprecated: false

  - name: "currencyExchangeRateLegacy"
    value: "/v1/rates/{baseCurrency}/{targetCurrency}"
    description: "Legacy exchange rate endpoint"
    version: "1.0"
    deprecated: true
    deprecationMessage: "Use 'currencyExchangeRate' (v2) for enhanced
precision"
    tags: ["currency", "exchange-rate", "deprecated"]

```

Example 4: Data Pipeline Operations

```

data-sinks:
  - name: "audit-database"
    type: "database"
    source-type: "postgresql"

operations:
  # Simple operations - map format
  logEvent: "INSERT INTO events (type, timestamp) VALUES (:type,
CURRENT_TIMESTAMP)"

  # Complex audit operations - array format
  - name: "recordComplianceAudit"
    value: |
      INSERT INTO compliance_audit_log (
        entity_type,
        entity_id,
        action,
        user_id,

```

```

        compliance_framework,
        data_snapshot,
        timestamp
    ) VALUES (
        :entityType,
        :entityId,
        :action,
        :userId,
        :complianceFramework,
        :dataSnapshot,
        CURRENT_TIMESTAMP
    )
description: "Record compliance audit trail for regulatory reporting"
parameters: ["entityType", "entityId", "action", "userId",
"complianceFramework", "dataSnapshot"]
tags: ["audit", "compliance", "regulatory"]
compliance: ["SOX", "GDPR", "HIPAA"]
owner: "compliance-team"
version: "1.0"
performance: "partitioned by month"

```

Technical Reference

Test Coverage

Unit Tests: 15/15 passing ☒

- Map format deserialization
- Array format deserialization
- Multiline query handling
- Mixed format support
- Empty collection handling
- Duplicate key detection
- Missing field validation
- Invalid format error handling

Integration Tests: 8/8 passing ☒

- REST API integration
- CSV file system queries
- JSON file system queries
- Data source caching
- Circuit breaker patterns
- Concurrent access

Implementation Files

Core Classes:

- [apex-core/src/main/java/dev/mars/apex/core/config/yaml/YamlDataSource.java](#)

- `apex-core/src/main/java/dev/mars/apex/core/config/yaml/YamlDataSink.java`

Model Classes:

- `apex-core/src/main/java/dev/mars/apex/core/config/yaml/NamedQuery.java`
- `apex-core/src/main/java/dev/mars/apex/core/config/yaml/NamedOperation.java`
- `apex-core/src/main/java/dev/mars/apex/core/config/yaml/NamedEndpoint.java`

Jackson Deserializers:

- `apex-core/src/main/java/dev/mars/apex/core/config/yaml/FlexibleQueriesDeserializer.java`
- `apex-core/src/main/java/dev/mars/apex/core/config/yaml/FlexibleOperationsDeserializer.java`
- `apex-core/src/main/java/dev/mars/apex/core/config/yaml/FlexibleEndpointsDeserializer.java`

Test Files:

- `apex-core/src/test/java/dev/mars/apex/core/config/yaml/DualFormatDeserializationTest.java`
- `apex-demo/src/test/java/dev/mars/apex/demo/external/ExternalDataSourceIntegrationTest.java`

Performance Characteristics

- **Deserialization Overhead:** < 1ms per configuration file
- **Memory Impact:** Negligible (metadata discarded after Map conversion)
- **Runtime Performance:** Identical to map format (uses same Map structure)
- **Backward Compatibility:** 100% - all existing YAML files work unchanged

Frequently Asked Questions

Q: Do I need to migrate existing configurations?

A: No. Migration is entirely optional. All existing map format configurations continue to work perfectly. Only migrate if you want the additional metadata benefits.

Q: Can I mix both formats in the same file?

A: Yes! You can use map format for some fields and array format for others in the same configuration file.

Q: Is there a performance penalty for using array format?

A: Minimal. The overhead is only during deserialization (< 1ms). At runtime, both formats use identical Map structures for fast lookups.

Q: What happens if I have duplicate names in array format?

A: The deserializer will throw a clear error message identifying the duplicate key name. Each query/operation/endpoint must have a unique name within its section.

Q: Can I use multiline queries in array format?

A: Yes! Use YAML's pipe (|) or greater-than (>) operators for multiline strings in the **value** field.

Q: Are all metadata fields required in array format?

A: No. Only **name** and **value** are required. All other fields (description, tags, version, etc.) are optional.

3. Document Structure & Metadata

Required Metadata Section

Every APEX YAML document must begin with a metadata section:

```
metadata:
  id: "unique-document-identifier"
  name: "Document Name"
  version: "1.0.0"
  description: "Document description"
  type: "rule-config"
  author: "author@company.com"
  created-by: "author@company.com"
  created-date: "2024-12-24"
  business-domain: "Business Domain"
  source: "Data Source System"
  tags: ["tag1", "tag2", "tag3"]
```

Metadata Properties

Property	Required	Description	Example
id	Yes	Unique document identifier	"financial-settlement-rules-v1"
name	Yes	Human-readable document name	"Financial Settlement Rules"
version	Yes	Semantic version number	"1.2.3"
description	Yes	Brief description of purpose	"Post-trade settlement enrichment"
type	Yes	Document type identifier	"rule-config"
author	No	Document author	"john.doe@bank.com"
created-by	No	Creator identifier	"settlement-team@bank.com"
created-date	No	Creation date (ISO format)	"2024-12-24"

Property	Required	Description	Example
business-domain	No	Business domain for scenarios	"Financial Services"
source	No	Data source system (for dataset types)	"Reference Data Service"
tags	No	Categorization tags	["finance", "settlement"]

Metadata Validation and Best Practices

Required vs Optional Fields

- **Always Required:** `id`, `name`, `version`, `description`, `type`
- **Type-Specific Requirements:** See document types table below
- **Recommended:** `author`, `created-date`, `tags` for better maintainability

Consistency Guidelines

- **Version Format:** Use semantic versioning (e.g., "2.1.0") for all documents
- **Date Format:** Use ISO format "YYYY-MM-DD" for `created-date`
- **Email Format:** Use consistent email format for `author` and `created-by`
- **Tag Format:** Use lowercase, hyphenated tags: `["apex-demo", "trade-validation"]`

Validation Notes

APEX validates metadata at configuration load time and will reject documents with:

- Missing required fields for the document type
- Invalid `type` values not in the supported list
- Malformed version numbers or dates
- Duplicate `id` values within the same deployment

Document-Level Configuration

Additional configuration options:

```
metadata:
  id: "example-configuration"
  name: "Example Configuration"
  version: "1.0.0"
  type: "rule-config"

# Processing configuration
processing:
  parallel: true
  timeout: 30000 # milliseconds
  retry-count: 3
```

```
# Logging configuration
logging:
  level: "INFO"
  include-context: true

# Performance configuration
performance:
  cache-enabled: true
  cache-ttl: 3600 # seconds
```

Important Notes on Metadata

The **id** Attribute

The **id** attribute is **critical** for APEX document identification and is used throughout the system for:

- **Configuration Loading:** APEX services use the **id** to identify and cache configurations
- **Error Reporting:** Error messages reference the document **id** for troubleshooting
- **Dependency Resolution:** Other configurations reference documents by their **id**
- **Audit Trails:** All processing logs include the document **id** for traceability

Best Practices for **id** Values:

- Use descriptive, kebab-case identifiers: "**customer-profile-enrichment-v2**"
- Include version information when appropriate: "**trade-validation-rules-2024**"
- Keep them unique across your APEX deployment
- Avoid spaces and special characters except hyphens and underscores

Date Field Standardization

Use **created-date** with ISO format (**YYYY-MM-DD**) for consistency across all APEX configurations.

Document Types

APEX supports several document types, each with specific purposes and validation requirements:

Type	Purpose	Required Fields	Top-level Sections
rule-config	Business rules and validation logic	id, author	rules, enrichments
enrichment	Data enrichment configurations	id, author	enrichments
dataset	Reference data and lookup tables	id, source	data
scenario	End-to-end processing scenarios	id, business-domain, owner	scenario, data-types, (rule-configurations OR processing-stages)

Type	Purpose	Required Fields	Top-level Sections
scenario-registry	Scenario collection management	id, created-by	scenarios
component	Reusable configuration component	id, name, version	rule-configurations, enrichment-refs, component-refs, config-files
rule-chain	Sequential rule execution definitions	id, author	rule-chains
external-data-config	External data source configurations	id, author	dataSources, configuration
pipeline	ETL and data processing pipeline orchestration	id, author	pipeline, data-sources, data-sinks

External Data Configuration

External data configuration files define how APEX connects to and interacts with external data sources such as databases, REST APIs, file systems, and message queues.

Example: Database Configuration

```
metadata:
  name: "Production Database Sources"
  version: "1.0.0"
  description: "Database connections for production environment"
  type: "external-data-config"
  author: "data.team@company.com"
  tags: ["database", "production", "postgresql"]

dataSources:
- name: "user-database"
  type: "database"
  sourceType: "postgresql"
  enabled: true
  description: "Primary user database"

connection:
  host: "prod-db.company.com"
  port: 5432
  database: "userdb"
  username: "app_user"
  password: "${DB_PASSWORD}"

# DUAL FORMAT SUPPORT (APEX 2.2+)
# Map format (legacy - still fully supported)
queries:
  getUserById: "SELECT * FROM users WHERE id = :id"
```

```

    getActiveUsers: "SELECT * FROM users WHERE status = 'ACTIVE'"

# Array format (new - recommended for documentation-rich projects)
# queries:
#   - name: "getUserById"
#     value: "SELECT * FROM users WHERE id = :id"
#     description: "Fetch user profile by unique ID"
#     tags: ["user-management", "primary-lookup"]
#
#   - name: "getActiveUsers"
#     value: "SELECT * FROM users WHERE status = 'ACTIVE'"
#     description: "List all users with active status"
#     tags: ["user-management", "batch-query"]
#     cacheable: true

cache:
  enabled: true
  ttlSeconds: 300
  maxSize: 1000

configuration:
  defaultConnectionTimeout: 30000
monitoring:
  enabled: true
  healthCheckLogging: true

```

Example: REST API Configuration

```

metadata:
  name: "External API Sources"
  version: "1.0.0"
  description: "REST API connections for data enrichment"
  type: "external-data-config"
  author: "integration.team@company.com"

dataSources:
  - name: "currency-rates-api"
    type: "rest-api"
    enabled: true
    description: "Real-time currency exchange rates"

    connection:
      base-url: "https://api.exchangerates.com/v1"

# DUAL FORMAT SUPPORT (APEX 2.2+)
# Map format (legacy - concise for simple cases)
endpoints:
  getCurrentRate: "/rates/{currency}"
  getHistoricalRate: "/rates/{currency}/{date}"

# Array format (new - recommended for enterprise APIs)
# endpoints:

```

```
# - name: "getCurrentRate"
#   value: "/rates/{currency}"
#   description: "Get latest exchange rate for currency"
#   method: "GET"
#   timeout: "2000ms"
#
# - name: "getHistoricalRate"
#   value: "/rates/{currency}/{date}"
#   description: "Get historical rate for specific date"
#   method: "GET"
#   cacheable: true
#   cache-ttl: "3600s"

authentication:
  type: "api-key"
  keyHeader: "X-API-Key"
  keyValue: "${EXCHANGE_API_KEY}"
```

Pipeline Configuration

Pipeline configuration files define complete ETL (Extract, Transform, Load) workflows that orchestrate data processing from multiple sources to multiple destinations.

Example: CSV to Database Pipeline

```
metadata:
  id: "csv-to-h2-pipeline-demo"
  name: "CSV to H2 ETL Pipeline Demo"
  version: "1.0.0"
  description: "Demonstration of CSV data processing with H2 database output"
  type: "pipeline"
  author: "APEX Demo Team"
  tags: ["demo", "etl", "csv", "h2", "pipeline"]

# Pipeline orchestration - defines the complete ETL workflow
pipeline:
  name: "customer-etl-pipeline"
  description: "Extract customer data from CSV, transform, and load into H2
database"

# Pipeline steps executed in sequence
steps:
  - name: "extract-customers"
    type: "extract"
    source: "customer-csv-input"
    operation: "getAllCustomers"
    description: "Read all customer records from CSV file"

  - name: "load-to-database"
    type: "load"
    sink: "customer-h2-database"
```

```

    operation: "insertCustomer"
    description: "Insert customer records into H2 database"
    depends-on: ["extract-customers"]

# Pipeline execution configuration
execution:
  mode: "sequential" # or "parallel" for independent steps
  error-handling: "stop-on-error" # or "continue-on-error"
  max-retries: 3
  retry-delay-ms: 1000

# Input data source configuration
data-sources:
- name: "customer-csv-input"
  type: "file-system"
  source-type: "csv"
  enabled: true
  description: "Customer CSV file input for ETL processing"

  connection:
    base-path: "./target/demo/etl/data"
    file-pattern: "customers.csv"
    encoding: "UTF-8"

  file-format:
    type: "csv"
    has-header-row: true
    delimiter: ","
    column-mappings:
      "customer_id": "id"
      "customer_name": "customerName"
      "email_address": "email"

# Output data sink configuration
data-sinks:
- name: "customer-h2-database"
  type: "database"
  source-type: "h2"
  enabled: true
  description: "H2 database for storing processed customer data"

  connection:
    database: "./target/demo/etl/output/customer_database"
    username: "sa"
    password: ""
    mode: "PostgreSQL"

  operations:
    insertCustomer: |
      INSERT INTO customers (
        customer_id, customer_name, email, processed_at
      ) VALUES (
        :id, :customerName, :email, CURRENT_TIMESTAMP
      )

```

```
schema:
  auto-create: true
  table-name: "customers"
  init-script: |
    CREATE TABLE IF NOT EXISTS customers (
      customer_id INTEGER PRIMARY KEY,
      customer_name VARCHAR(255) NOT NULL,
      email VARCHAR(255),
      processed_at TIMESTAMP
    );
```

4. Core Syntax Elements

3.1 Data Access Patterns

Direct Field Access

All data access in APEX YAML uses direct field references to access HashMap data:

```
# Accessing top-level fields
condition: "#fieldName != null"

# Accessing nested fields (when data contains nested objects)
condition: "#trade.security.instrumentId != null"

# Using in calculations
expression: "#quantity * #price"
```

⚠ CRITICAL SYNTAX NOTE: APEX processes HashMap data structures where field names are accessed directly using `#fieldName` syntax. Do **NOT** use `#fieldName` syntax as this will cause SpEL evaluation errors. The correct pattern is always `#fieldName` for HashMap keys.

Nested Field Access with Dot Notation

Access nested objects using dot notation:

```
# Simple nesting
condition: "#customer.address.country == 'US'"

# Deep nesting
condition: "#trade.tradeHeader.partyTradeIdentifier.tradeId != null"

# Array/list access
condition: "#positions[0].instrumentId != null"
```

Null-Safe Navigation

Use the `?.` operator for null-safe navigation:

```
# Safe navigation - won't throw NullPointerException
condition: "#trade?.security?.instrumentId != null"

# Equivalent to checking each level for null
condition: "#trade != null && #trade.security != null &&
#trade.security.instrumentId != null"
```

Array and Collection Access

Access arrays and collections:

```
# Array index access
condition: "#positions[0].quantity > 0"

# Collection size
condition: "#positions.size() > 0"

# Collection operations
condition: "#positions.?[quantity > 1000].size() > 0" # Filter collection
```

Context Variables

APEX provides special context variables that are automatically available during rule and enrichment processing:

Rule Result References

Access individual rule evaluation results using the `#ruleResults` context variable:

```
rules:
  - id: "high-value-rule"
    name: "High Value Transaction Rule"
    condition: "#amount > 10000"
    message: "Transaction amount exceeds $10,000"

  - id: "premium-customer-rule"
    name: "Premium Customer Rule"
    condition: "#customerType == 'PREMIUM'"
    message: "Customer has premium status"

enrichments:
  # Conditional enrichment based on individual rule result
  - id: "high-value-processing"
```

```

    type: "field-enrichment"
    condition: "#ruleResults['high-value-rule'] == true"
    field-mappings:
      - target-field: "processingPriority"
        expression: "'HIGH'"

# Multiple rule results in complex conditions
- id: "priority-calculation"
  type: "field-enrichment"
  condition: "#ruleResults != null"
  field-mappings:
    - target-field: "processingPriority"
      expression: |
        #ruleResults['premium-customer-rule'] == true && #ruleResults['high-
value-rule'] == true ? 'IMMEDIATE' :
        #ruleResults['high-value-rule'] == true ? 'HIGH' :
        #ruleResults['premium-customer-rule'] == true ? 'ELEVATED' :
        'STANDARD'

```

Available Properties:

- `#ruleResults['rule-id']` - Boolean value (true/false) indicating if the rule passed
- `#ruleResults.containsKey('rule-id')` - Check if a rule was evaluated
- `#ruleResults.get('rule-id')` - Get rule result with null safety

Rule Group Result References

Access rule group evaluation results using the `#ruleGroupResults` context variable:

```

rule-groups:
  - id: "validation-group"
    name: "Transaction Validation Group"
    operator: "OR"
    rule-ids:
      - "high-value-rule"
      - "premium-customer-rule"

enrichments:
  # Conditional enrichment based on rule group result
  - id: "validation-status"
    type: "field-enrichment"
    condition: "#ruleGroupResults['validation-group']['passed'] == true"
    field-mappings:
      - target-field: "validationStatus"
        expression: "'VALIDATED'"

# Access failed rules from group
- id: "failure-handling"
  type: "field-enrichment"
  condition: "#ruleGroupResults['validation-group']['passed'] == false"
  field-mappings:

```

```
- target-field: "failedValidations"
  expression: "#ruleGroupResults['validation-group']['failedRules']"
```

Available Properties:

- `#ruleGroupResults['group-id']['passed']` - Boolean indicating if the group passed
- `#ruleGroupResults['group-id']['failedRules']` - List of failed rule IDs
- `#ruleGroupResults['group-id']['passedRules']` - List of passed rule IDs
- `#ruleGroupResults.containsKey('group-id')` - Check if a group was evaluated

Use Cases:

- **Conditional Enrichments:** Apply enrichments only when specific rules pass
- **Multi-Stage Processing:** Route data based on validation results
- **Complex Decision Trees:** Build sophisticated logic using rule outcomes
- **Fallback Logic:** Provide defaults when validation fails

3.2 Condition Syntax

Boolean Expressions

Basic boolean logic:

```
# Simple boolean check
condition: "#isActive"

# Negation
condition: "!#isDeleted"

# Complex boolean logic
condition: "#isActive && !#isDeleted"
```

Comparison Operators

All standard comparison operators are supported:

```
# Equality
condition: "#status == 'ACTIVE'"

# Inequality
condition: "#quantity != 0"

# Numeric comparisons
condition: "#price > 100.0"
condition: "#quantity >= 1000"
condition: "#discount < 0.1"
condition: "#rating <= 5"
```

Logical Operators

Combine conditions with logical operators:

```
# AND operator
condition: "#isActive && #quantity > 0"

# OR operator
condition: "#status == 'PENDING' || #status == 'PROCESSING'"

# NOT operator
condition: "!#isDeleted && #isVisible"

# Complex combinations with parentheses
condition: "(#type == 'EQUITY' || #type == 'BOND') && #quantity > 0"
```

String Operations

String manipulation and comparison:

```
# String equality (case-sensitive)
condition: "#currency == 'USD'"

# String contains
condition: "#description.contains('SWAP')"

# String starts with / ends with
condition: "#instrumentId.startsWith('US')"
condition: "#instrumentId.endsWith('005')"

# String length
condition: "#instrumentId.length() == 12"

# Case-insensitive comparison
condition: "#currency.toUpperCase() == 'USD'"
```

Regular Expression Support

Use regex for pattern matching:

```
# ISIN format validation
condition: "#instrumentId.matches('^([A-Z]{2}[A-Z0-9]{9}[0-9]$)')"

# Email validation
condition: "#email.matches('^([A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$)'"
```

```
# Phone number validation
condition: "#phone.matches('^\\+?[1-9]\\d{1,14}$')"
```

Null Checks and Validation

Proper null handling:

```
# Null check
condition: "#fieldName != null"

# Not null and not empty for strings
condition: "#fieldName != null && #fieldName.trim().length() > 0"

# Null-safe string operations
condition: "#fieldName?.trim()?.length() > 0"

# Default values for null fields
expression: "#fieldName != null ? #fieldName : 'DEFAULT_VALUE'"
```

3.3 Expression Language

SpEL Integration

APEX YAML provides full access to Spring Expression Language features:

```
# Variable assignment and reuse
expression: "#root.setVariable('tradeValue', #quantity * #price); #tradeValue"

# Method chaining
expression: "#instrumentId.substring(0, 2).toUpperCase()"

# Collection operations
expression: "#positions.![quantity * price].sum()"
```

Mathematical Operations

Standard mathematical operations:

```
# Basic arithmetic
expression: "#quantity * #price"
expression: "#total - #discount"
expression: "#principal + #interest"
expression: "#amount / #exchangeRate"
expression: "#base % #divisor"

# Mathematical functions via Java Math class
```

```

expression: "T(java.lang.Math).max(#value1, #value2)"
expression: "T(java.lang.Math).min(#value1, #value2)"
expression: "T(java.lang.Math).abs(#value)"
expression: "T(java.lang.Math).sqrt(#value)"
expression: "T(java.lang.Math).pow(#base, #exponent)"
expression: "T(java.lang.Math).round(#value * 100) / 100.0" # Round to 2 decimals

```

String Manipulation

String operations and formatting:

```

# String concatenation
expression: "#firstName + ' ' + #lastName"

# String formatting
expression: "T(java.lang.String).format('Trade %s: %,2f %s', #tradeId, #amount,
#currency)"

# String manipulation
expression: "#text.toUpperCase()"
expression: "#text.toLowerCase()"
expression: "#text.trim()"
expression: "#text.substring(0, 10)"
expression: "#text.replace('OLD', 'NEW')"

```

Date and Time Functions

Date/time operations using Java time classes:

```

# Current date/time
expression: "T(java.time.LocalDate).now()"
expression: "T(java.time.Instant).now().toString()"

# Date formatting
expression:
"T(java.time.LocalDate).now().format(T(java.time.format.DateTimeFormatter).ofPatte
rn('yyyyMMdd'))"

# Date arithmetic
expression: "#tradeDate.plusDays(2)" # Add 2 days
expression: "#startDate.plusMonths(1)" # Add 1 month
expression: "#endDate.minusYears(1)" # Subtract 1 year

# Date comparisons
condition: "#settlementDate.isAfter(T(java.time.LocalDate).now())"
condition: "#maturityDate.isBefore(#tradeDate.plusYears(10))"

```

Java Class Access

Access Java classes and static methods using `T()` syntax:

```
# UUID generation
expression: "T(java.util.UUID).randomUUID().toString()"

# BigDecimal operations
expression:
"T(java.math.BigDecimal).valueOf(#amount).multiply(T(java.math.BigDecimal).valueOf
(#rate))"

# Collections utilities
expression: "T(java.util.Collections).max(#values)"
expression: "T(java.util.Collections).min(#values)"

# Custom utility classes
expression: "T(com.company.utils.FinancialUtils).calculateInterest(#principal,
#rate, #days)"
```

5. Rules Section

4.1 Validation Rules

Validation rules check data integrity and business constraints:

```
rules:
  - id: "trade-id-required"
    name: "Trade ID Required"
    condition: "#trade != null && #trade.tradeId != null &&
#trade.tradeId.trim().length() > 0"
    message: "Trade ID is required and cannot be empty"
    severity: "ERROR"
    priority: 1

  - id: "isin-format-validation"
    name: "ISIN Format Validation"
    condition: "#security != null && #security.isin != null &&
#security.isin.matches('[A-Z]{2}[A-Z0-9]{9}[0-9]$')"
    message: "ISIN must follow format: 2 country letters + 9 alphanumeric + 1
check digit"
    severity: "ERROR"
    priority: 1

  - id: "trade-value-positive"
    name: "Trade Value Must Be Positive"
    condition: "#quantity != null && #price != null && (#quantity * #price) > 0"
    message: "Trade value must be positive"
```

```
severity: "ERROR"
priority: 1
```

Rule Properties

Property	Required	Description	Example
id	Yes	Unique rule identifier	"trade-id-required"
name	Yes	Human-readable rule name	"Trade ID Required"
condition	Yes	SpEL expression that must be true	"#field != null"
message	Yes	Message for match (condition=true)	"Field is valid"
no-match-message	No	Message for no-match (condition=false). Falls back to message if omitted	"Field is required"
severity	Yes	ERROR, WARNING, INFO	"ERROR"
priority	No	Execution priority (1 = highest)	1
result-field	No	Field name to store rule result (boolean)	"isHighValue"

Severity Levels

- **ERROR:** Critical validation failure, stops processing
- **WARNING:** Non-critical issue, processing continues
- **INFO:** Informational message, no impact on processing

Result Field Storage

The `result-field` property allows you to store a rule's boolean evaluation result (true/false) in the facts map and enriched data for use by subsequent rules or enrichments. This enables rule chaining and conditional logic based on previous rule evaluations.

Key Features:

- Stores the rule's boolean result (true = rule matched, false = rule did not match)
- Result is available to subsequent rules via SpEL expressions (e.g., `#isHighValue`)
- Supports nested field notation (e.g., `"validation.isHighValue"` creates nested structure)
- Minimal overhead - only rules with `result-field` configured store results
- Results are stored in both the facts map (for SpEL access) and enrichedData (for API consumers)
- Performance impact is negligible for typical business rule scenarios (sub-millisecond overhead)

Example - Basic Rule Chaining:

```
rules:
  # First rule: Check if trade is high value
```

```

- id: "check-high-value"
  name: "Check High Value Trade"
  condition: "#notionalAmount != null && #notionalAmount > 10000000"
  message: "Trade is high value"
  severity: "INFO"
  result-field: "isHighValue" # Store result for subsequent rules

# Second rule: Use the stored result
- id: "check-approval-required"
  name: "Check Approval Required"
  condition: "#isHighValue == true" # Access previous rule's result
  message: "Trade requires approval"
  severity: "WARNING"
  result-field: "requiresApproval"

```

Example - Nested Field Storage:

```

rules:
  # Store result in nested structure
  - id: "check-high-value-nested"
    name: "Check High Value (Nested)"
    condition: "#notionalAmount > 10000000"
    message: "High value trade detected"
    severity: "INFO"
    result-field: "validation.isHighValue" # Creates {"validation": {"isHighValue": true}}

  # Access nested result
  - id: "check-credit-rating"
    name: "Check Credit Rating"
    condition: "#validation['isHighValue'] == true && #creditRating != null"
    message: "Credit check required for high value trades"
    severity: "WARNING"
    result-field: "requiresCreditCheck"

```

Example - Complex Conditional Logic:

```

rules:
  - id: "check-region"
    name: "Check APAC Region"
    condition: "#region == 'APAC'"
    message: "Trade is in APAC region"
    severity: "INFO"
    result-field: "isApacTrade"

  - id: "check-high-value"
    name: "Check High Value"
    condition: "#notionalAmount > 10000000"
    message: "High value trade"

```

```

    severity: "INFO"
    result-field: "isHighValue"

  - id: "check-complex-approval"
    name: "Check Complex Approval Logic"
    condition: "(#isApacTrade == true && #isHighValue == true) ||
#requiresCreditCheck == true"
    message: "Complex approval required"
    severity: "WARNING"
    result-field: "requiresComplexApproval"

```

Best Practices:

- Use descriptive field names that clearly indicate what the result represents
- Use nested notation (e.g., "validation.isHighValue") to organize related results
- Only configure `result-field` when you need to use the result in subsequent rules
- Access stored results using SpEL syntax: `#fieldName` or `#nested['fieldName']`
- Results are boolean values: `true` (rule matched) or `false` (rule did not match)

No-Match Message

The `no-match-message` property allows configuring a separate message for when a rule's condition evaluates to `false` (NO_MATCH). This is useful when you want to communicate different information for pass versus fail outcomes.

Behavior:

- When condition is `true` (MATCH): the `message` property is used
- When condition is `false` (NO_MATCH): the `no-match-message` property is used if present, otherwise falls back to `message`
- Supports the same placeholder formats as `message: {{#expression}}` (Handlebars) and `# {expression}` (SpEL template)
- All placeholders are resolved against the evaluation context (facts map)

Example:

```

rules:
  - id: "age-check"
    name: "Age Validation"
    condition: "#age != null && #age >= 18"
    message: "Age {{#age}} is valid (18 or older)"
    no-match-message: "Age {{#age}} does not meet minimum requirement of 18"
    severity: "INFO"

  - id: "amount-threshold"
    name: "Amount Threshold Check"
    condition: "#amount > 10000"
    message: "Amount {{#amount}} exceeds reporting threshold"
    no-match-message: "Amount {{#amount}} is within normal range"
    severity: "WARNING"

```

Complex Validation Examples

```
rules:
  # Multi-field validation
  - id: "settlement-date-validation"
    name: "Settlement Date Must Be After Trade Date"
    condition: "#tradeDate != null && #settlementDate != null &&
#settlementDate.isAfter(#tradeDate)"
    message: "Settlement date must be after trade date"
    severity: "ERROR"
    priority: 1

  # Conditional validation
  - id: "margin-required-for-derivatives"
    name: "Margin Required for Derivative Trades"
    condition: "#instrumentType != 'DERIVATIVE' || (#instrumentType ==
'DERIVATIVE' && #marginAmount != null && #marginAmount > 0)"
    message: "Margin amount is required for derivative trades"
    severity: "ERROR"
    priority: 2

  # Range validation
  - id: "credit-rating-range"
    name: "Credit Rating Must Be Valid"
    condition: "#creditRating == null || (#creditRating >= 1 && #creditRating <=
10)"
    message: "Credit rating must be between 1 and 10"
    severity: "WARNING"
    priority: 3
```

4.2 Business Rules

Business rules implement domain-specific logic:

```
rules:
  # Business logic rule
  - id: "high-value-trade-approval"
    name: "High Value Trade Requires Approval"
    condition: "#tradeValue > 10000000" # $10M threshold
    message: "Trade exceeds $10M threshold and requires additional approval"
    severity: "WARNING"
    priority: 1

  # Regulatory compliance rule
  - id: "emir-reporting-required"
    name: "EMIR Reporting Required"
    condition: "#counterparty.jurisdiction == 'EU' && #notionalAmount > 1000000"
    message: "Trade requires EMIR reporting"
```

```

    severity: "INFO"
    priority: 2

# Risk management rule
- id: "concentration-limit-check"
  name: "Concentration Limit Check"
  condition: "#portfolioConcentration <= 0.25" # 25% limit
  message: "Position exceeds 25% concentration limit"
  severity: "ERROR"
  priority: 1

```

4.3 Rule Categories

Rule categories provide organizational structure and enterprise governance for rules. Categories enable metadata inheritance, business domain tracking, and lifecycle management of rules across your APEX deployment.

4.3.1 Category Overview

Categories serve multiple purposes:

1. **Organizational Structure** - Group related rules by business function (validation, enrichment, compliance)
2. **Metadata Inheritance** - Rules inherit enterprise metadata from their category
3. **Governance & Audit** - Track business ownership, domain, and creation information
4. **Lifecycle Management** - Control rule effectiveness with dates and status
5. **Execution Control** - Configure category-level execution behavior (priority, parallel execution)

4.3.2 Defining Categories

Categories are defined at the document level in a `categories` section:

```

metadata:
  id: "trade-processing-rules"
  name: "Trade Processing Rules"
  version: "1.0.0"
  type: "rule-config"
  author: "trading-team@bank.com"

# Define categories at document level
categories:
  - name: "validation"
    description: "Data validation and integrity checks"
    priority: 10
    business-domain: "Trade Processing"
    business-owner: "validation-team@bank.com"
    created-by: "john.doe@bank.com"
    enabled: true

  - name: "enrichment"

```

```
description: "Trade data enrichment and classification"
priority: 20
business-domain: "Trade Processing"
business-owner: "enrichment-team@bank.com"
created-by: "jane.smith@bank.com"
enabled: true

- name: "compliance"
  description: "Regulatory compliance checks"
  priority: 30
  business-domain: "Compliance"
  business-owner: "compliance-team@bank.com"
  created-by: "compliance-admin@bank.com"
  enabled: true

# Rules reference categories
rules:
  - id: "trade-id-required"
    name: "Trade ID Required"
    category: "validation" # Reference the category
    condition: "#tradeId != null && #tradeId.trim().length() > 0"
    message: "Trade ID is required"
    severity: "ERROR"

  - id: "market-classification"
    name: "Market Classification"
    category: "enrichment" # Different category
    condition: "#instrumentType != null"
    message: "Market classification determined"
    severity: "INFO"

  - id: "emir-reporting-check"
    name: "EMIR Reporting Required"
    category: "compliance" # Compliance category
    condition: "#counterparty.jurisdiction == 'EU'"
    message: "EMIR reporting required"
    severity: "WARNING"
```

4.3.3 Category Properties

Property	Required	Type	Description	Example
name	Yes	String	Unique category identifier	"validation"
description	No	String	Human-readable description	"Data validation checks"
priority	No	Integer	Execution priority (lower = higher)	10
enabled	No	Boolean	Whether category is active	true
business-domain	No	String	Business domain classification	"Trade Processing"

Property	Required	Type	Description	Example
business-owner	No	String	Owner responsible for category	"team@bank.com"
created-by	No	String	Creator identifier	"user@bank.com"
effective-date	No	String	Date when category becomes effective (ISO 8601)	"2025-01-01"
expiration-date	No	String	Date when category expires (ISO 8601)	"2025-12-31"
stop-on-first-failure	No	Boolean	Stop processing on first failure	false
parallel-execution	No	Boolean	Execute rules in parallel	false

4.3.4 Metadata Inheritance

Rules inherit metadata from their category when not explicitly specified on the rule:

```
categories:
- name: "validation"
  business-domain: "Trade Processing"
  business-owner: "validation-team@bank.com"
  created-by: "admin@bank.com"

rules:
# Rule 1: Inherits all category metadata
- id: "rule-1"
  name: "Rule 1"
  category: "validation"
  condition: "true"
  message: "Rule 1"
  severity: "ERROR"
# Inherits: business-domain, business-owner, created-by from category

# Rule 2: Overrides category metadata
- id: "rule-2"
  name: "Rule 2"
  category: "validation"
  condition: "true"
  message: "Rule 2"
  severity: "ERROR"
  business-owner: "special-team@bank.com" # Override category owner
  created-by: "special-user@bank.com" # Override category creator
# Inherits: business-domain from category
```

Inheritance Priority: Rule-level metadata > Category-level metadata > System defaults

4.3.5 Practical Examples

Example 1: Multi-Domain Organization

```
categories:
- name: "front-office-validation"
  description: "Front office trade validation"
  business-domain: "Front Office"
  business-owner: "front-office-head@bank.com"
  priority: 10

- name: "middle-office-enrichment"
  description: "Middle office enrichment"
  business-domain: "Middle Office"
  business-owner: "middle-office-head@bank.com"
  priority: 20

- name: "back-office-compliance"
  description: "Back office compliance"
  business-domain: "Back Office"
  business-owner: "back-office-head@bank.com"
  priority: 30

rules:
- id: "trade-format-check"
  name: "Trade Format Validation"
  category: "front-office-validation"
  condition: "#tradeId != null"
  message: "Trade format valid"
  severity: "ERROR"

- id: "counterparty-enrichment"
  name: "Counterparty Data Enrichment"
  category: "middle-office-enrichment"
  condition: "#counterpartyId != null"
  message: "Counterparty enriched"
  severity: "INFO"

- id: "regulatory-check"
  name: "Regulatory Compliance Check"
  category: "back-office-compliance"
  condition: "#jurisdiction != null"
  message: "Regulatory check passed"
  severity: "WARNING"
```

Example 2: Lifecycle Management

```
categories:
- name: "active-rules"
  description: "Currently active validation rules"
```

```

    enabled: true
    effective-date: "2025-01-01"
    expiration-date: "2025-12-31"
    business-owner: "rules-team@bank.com"

  - name: "deprecated-rules"
    description: "Legacy rules being phased out"
    enabled: false
    effective-date: "2024-01-01"
    expiration-date: "2024-12-31"
    business-owner: "legacy-team@bank.com"

rules:
  - id: "new-validation"
    name: "New Validation Rule"
    category: "active-rules"
    condition: "true"
    message: "New validation"
    severity: "ERROR"

  - id: "old-validation"
    name: "Old Validation Rule"
    category: "deprecated-rules"
    condition: "true"
    message: "Old validation"
    severity: "ERROR"

```

Example 3: Execution Control

```

categories:
  - name: "critical-validation"
    description: "Critical validations - stop on first failure"
    stop-on-first-failure: true
    parallel-execution: false
    priority: 1
    business-owner: "critical-team@bank.com"

  - name: "optional-enrichment"
    description: "Optional enrichments - continue on failure"
    stop-on-first-failure: false
    parallel-execution: true
    priority: 100
    business-owner: "enrichment-team@bank.com"

rules:
  - id: "mandatory-check"
    name: "Mandatory Field Check"
    category: "critical-validation"
    condition: "#requiredField != null"
    message: "Mandatory field present"
    severity: "ERROR"

```

```
- id: "optional-check"
  name: "Optional Enhancement"
  category: "optional-enrichment"
  condition: "true"
  message: "Optional enhancement applied"
  severity: "INFO"
```

4.3.6 Best Practices

1. Organize by Business Function

```
categories:
- name: "input-validation"
  description: "Validate incoming data"
  business-domain: "Data Quality"

- name: "business-rules"
  description: "Apply business logic"
  business-domain: "Business Logic"

- name: "compliance-checks"
  description: "Regulatory compliance"
  business-domain: "Compliance"
```

2. Use Consistent Naming

```
categories:
- name: "trade-validation"      # Descriptive, kebab-case
- name: "TradeValidation"      # Avoid PascalCase
- name: "trade_validation"    # Avoid snake_case
- name: "validation"          # Simple, clear
```

3. Assign Clear Ownership

```
categories:
- name: "validation"
  business-owner: "validation-team@bank.com"
  created-by: "john.doe@bank.com"
  # Clear accountability for rule maintenance
```

4. Use Lifecycle Dates

```
categories:
- name: "new-rules"
  effective-date: "2025-01-01"
```

```
expiration-date: "2025-12-31"
# Track when rules are active
```

5. Document Purpose

```
categories:
- name: "validation"
  description: "Validates trade data integrity and required fields"
  # Clear documentation of category purpose
```

4.3.7 Category vs Rule-Level Properties

When both category and rule specify the same property, the rule-level value takes precedence:

```
categories:
- name: "validation"
  priority: 50
  business-owner: "category-owner@bank.com"

rules:
# Uses category priority (50)
- id: "rule-1"
  name: "Rule 1"
  category: "validation"
  condition: "true"
  message: "Rule 1"
  severity: "ERROR"

# Overrides category priority with rule priority (10)
- id: "rule-2"
  name: "Rule 2"
  category: "validation"
  priority: 10 # Override category priority
  condition: "true"
  message: "Rule 2"
  severity: "ERROR"

# Overrides category owner
- id: "rule-3"
  name: "Rule 3"
  category: "validation"
  business-owner: "rule-owner@bank.com" # Override category owner
  condition: "true"
  message: "Rule 3"
  severity: "ERROR"
```

6. Rule Groups Section

5.1 Overview

Rule Groups allow you to organize related rules and apply logical operators (AND/OR) to combine their results. Rule Groups support advanced execution features including parallel processing, configurable short-circuiting, and debug mode for comprehensive testing and troubleshooting.

5.2 Rule Group Configuration Approaches

APEX supports two approaches for referencing rules in rule groups: **rule-ids** (simple) and **rule-references** (advanced).

⚠ **CRITICAL: Inline Rule Definitions Not Supported**

Rule groups do **NOT** support inline rule definitions. Rules must be defined in the **rules** section and referenced by ID in rule groups.

INCORRECT (Inline Definitions):

```
rule-groups:
  - id: "my-rule-group"
    rules: # NOT SUPPORTED!
      - id: "my-rule"
        condition: "#value > 100"
        message: "Value too high"
```

CORRECT (Reference by ID):

```
# Define rules in the rules section
rules:
  - id: "my-rule"
    condition: "#value > 100"
    message: "Value too high"
    severity: "ERROR"

# Reference rules in rule groups
rule-groups:
  - id: "my-rule-group"
    operator: "AND"
    rule-ids:
      - "my-rule"
```

Why This Design? Rules are designed to be reusable across multiple rule groups. Defining rules once in the **rules** section and referencing them by ID promotes reusability and maintainability.

5.2.1 Simple Approach: **rule-ids**

Use **rule-ids** for straightforward rule grouping with automatic sequencing:

```
rule-groups:
  - id: "simple-validation-group"
    name: "Simple Input Validation"
    description: "Basic validation using rule-ids"
    operator: "AND"
    stop-on-first-failure: true
    rule-ids:
      - "trade-id-required"      # Executes first (sequence 1)
      - "isin-format-validation" # Executes second (sequence 2)
      - "trade-value-positive"   # Executes third (sequence 3)
```

5.2.2 Advanced Approach: rule-references

Use rule-references for fine-grained control over rule execution:

```
rule-groups:
  - id: "advanced-validation-group"
    name: "Advanced Input Validation"
    description: "Advanced validation using rule-references"
    operator: "AND"
    stop-on-first-failure: true
    rule-references:
      - rule-id: "trade-value-positive" # Custom sequence: executes first
        sequence: 1
        enabled: true
      - rule-id: "trade-id-required"    # Custom sequence: executes second
        sequence: 2
        enabled: true
      - rule-id: "isin-format-validation" # Disabled: skipped entirely
        sequence: 3
        enabled: false
        override-priority: 5           # Future feature: priority override
```

5.2.3 Comparison: rule-ids vs rule-references

Feature	rule-ids	rule-references	Use Case
Syntax	Simple string array	Complex object array	Quick setup vs detailed control
Execution Sequence	Auto (1, 2, 3...)	Custom sequence property	Default order vs custom order
Enable/Disable	All rules enabled	Individual enabled: true/false	All rules vs selective execution
Priority Override	Uses rule's priority	override-priority property*	Fixed priority vs custom priority

Feature	rule-ids	rule-references	Use Case
Configuration Size	Minimal	Larger	Simple configs vs complex workflows
Performance	Slightly faster	Slightly slower	High-performance vs flexibility

*Note: `override-priority` is documented but not yet implemented in the engine.

Rule Group Properties

Property	Required	Default	Description	Example
<code>id</code>	Yes	-	Unique rule group identifier	"validation-group"
<code>name</code>	Yes	-	Human-readable group name	"Input Validation"
<code>description</code>	No	""	Group description	"Validates all input parameters"
<code>category</code>	No	"default"	Group category	"validation"
<code>priority</code>	No	100	Execution priority (lower = higher priority)	10
<code>enabled</code>	No	true	Whether group is active	true
<code>operator</code>	No	"AND"	Logic operator: "AND" or "OR"	"AND"
<code>stop-on-first-failure</code>	No	false	Enable short-circuit evaluation	true
<code>parallel-execution</code>	No	false	Execute rules in parallel	false
<code>debug-mode</code>	No	false	Enable debug logging	false
<code>error-handling</code>	No	"fail-fast"	Exception handling strategy: "fail-fast", "continue-on-error", "skip-on-error"	"continue-on-error"
<code>rule-ids</code>	Conditional*	-	Simple: List of rule IDs	["rule1", "rule2"]
<code>rule-references</code>	Conditional*	-	Advanced: Rule reference objects	See examples below

*Either `rule-ids` OR `rule-references` is required, but not both.

5.2.4 Rule Reference Properties

When using `rule-references`, each rule reference supports the following properties:

Property	Required	Default	Description	Example
rule-id	Yes	-	ID of the rule to reference	"trade-id-required"
sequence	No	Auto-assigned	Execution order (1 = first)	1
enabled	No	true	Whether to execute this rule	true
override-priority	No	Rule's priority	Override rule's default priority*	5

*Note: `override-priority` is documented but not yet implemented in the engine.

5.2.5 Error Handling Strategy

The `error-handling` property controls how the rule group handles **exceptions** during rule evaluation (e.g., SpEL evaluation errors, null pointer exceptions, data access errors).

 **IMPORTANT DISTINCTION:**

- `error-handling`: Controls **exception handling** (technical errors during evaluation)
- `stop-on-first-failure`: Controls **business logic short-circuiting** (AND/OR evaluation behavior)

Valid Values:

Strategy	Behavior	Use Case
fail-fast	Stop immediately and return error (default)	Critical validation where any error must halt processing
continue-on-error	Log error and continue with remaining rules	Best-effort validation where partial results are acceptable
skip-on-error	Skip the failed rule and continue	Resilient processing where individual rule failures shouldn't block the group

Example: Error Handling Strategies

```
rule-groups:
  # Critical validation - any error stops processing
  - id: "critical-validation"
    name: "Critical Validation Rules"
    operator: "AND"
    error-handling: "fail-fast" # Default - stop on any exception
    rule-ids:
      - "mandatory-field-check"
      - "data-integrity-check"

  # Best-effort validation - continue despite errors
```

```

- id: "optional-validation"
  name: "Optional Validation Rules"
  operator: "OR"
  error-handling: "continue-on-error" # Log errors but continue
  rule-ids:
    - "optional-field-check"
    - "supplementary-check"

# Resilient processing - skip problematic rules
- id: "resilient-validation"
  name: "Resilient Validation Rules"
  operator: "AND"
  error-handling: "skip-on-error" # Skip failed rules, continue with others
  rule-ids:
    - "external-api-check"
    - "optional-enrichment-check"

```

Error Handling vs Stop-on-First-Failure:

```

rule-groups:
  # Demonstrates the difference between error-handling and stop-on-first-failure
  - id: "combined-behavior"
    name: "Combined Error Handling and Short-Circuiting"
    operator: "AND"
    stop-on-first-failure: true # Business logic: stop on first FAILED rule
    error-handling: "continue-on-error" # Exception handling: continue on
EXCEPTIONS
  rule-ids:
    - "rule-1" # If this FAILS (returns false), stop-on-first-failure kicks in
    - "rule-2" # If this throws EXCEPTION, error-handling kicks in
    - "rule-3"

```

5.2.6 Practical Examples

Example 1: Custom Execution Order

```

rule-groups:
  - id: "custom-order-group"
    operator: "AND"
    rule-references:
      - rule-id: "expensive-rule" # Execute last
        sequence: 3
      - rule-id: "quick-check" # Execute first
        sequence: 1
      - rule-id: "medium-rule" # Execute second
        sequence: 2

```

Example 2: Conditional Rule Execution

```
rule-groups:
  - id: "conditional-group"
    operator: "AND"
    rule-references:
      - rule-id: "always-check"
        enabled: true
      - rule-id: "optional-check"      # Disabled for this group
        enabled: false
      - rule-id: "debug-only-rule"    # Can be toggled
        enabled: false
```

Example 3: Mixed Configuration

```
rule-groups:
  - id: "mixed-group"
    operator: "OR"
    rule-references:
      - rule-id: "primary-validation"
        sequence: 1
        enabled: true
        override-priority: 1      # Highest priority (future feature)
      - rule-id: "fallback-validation"
        sequence: 2
        enabled: true
        override-priority: 10     # Lower priority (future feature)
      - rule-id: "legacy-validation"
        sequence: 3
        enabled: false           # Disabled legacy rule
```

5.3 Execution Behavior

AND Groups (All Rules Must Pass)

Using **rule-ids** (Simple Approach):

```
rule-groups:
  - id: "strict-validation-simple"
    name: "Strict Validation Group (Simple)"
    description: "All validation rules must pass"
    operator: "AND"
    stop-on-first-failure: true  # Stop on first failure for efficiency
    rule-ids:
      - "trade-id-required"      # Must pass (sequence 1)
      - "isin-format-validation" # Must pass (sequence 2)
      - "trade-value-positive"   # Must pass (sequence 3)
```

Using **rule-references** (Advanced Approach):

```

rule-groups:
- id: "strict-validation-advanced"
  name: "Strict Validation Group (Advanced)"
  description: "All validation rules must pass with custom control"
  operator: "AND"
  stop-on-first-failure: true
  rule-references:
    - rule-id: "trade-value-positive" # Execute first (fastest check)
      sequence: 1
      enabled: true
    - rule-id: "trade-id-required" # Execute second
      sequence: 2
      enabled: true
    - rule-id: "isin-format-validation" # Execute third (slowest check)
      sequence: 3
      enabled: true

```

Execution Flow (Short-Circuit Enabled):

```

Rule 1: PASS → Continue to Rule 2
Rule 2: PASS → Continue to Rule 3
Rule 3: FAIL → STOP (return false) - Remaining rules NOT evaluated

```

OR Groups (Any Rule Can Pass)

Using **rule-ids** (Simple Approach):

```

rule-groups:
- id: "eligibility-check-simple"
  name: "Customer Eligibility Check (Simple)"
  description: "Customer meets at least one eligibility criteria"
  operator: "OR"
  stop-on-first-failure: true # Stop on first success for OR groups
  rule-ids:
    - "high-value-customer" # Any can pass (sequence 1)
    - "premium-member" # Any can pass (sequence 2)
    - "long-term-client" # Any can pass (sequence 3)

```

Using **rule-references** (Advanced Approach):

```

rule-groups:
- id: "eligibility-check-advanced"
  name: "Customer Eligibility Check (Advanced)"
  description: "Customer meets eligibility criteria with selective rules"
  operator: "OR"
  stop-on-first-failure: true

```

```
rule-references:
  - rule-id: "premium-member"      # Check most likely first
    sequence: 1
    enabled: true
  - rule-id: "high-value-customer" # Check second most likely
    sequence: 2
    enabled: true
  - rule-id: "long-term-client"    # Check least likely
    sequence: 3
    enabled: true
  - rule-id: "legacy-vip-status"   # Disabled legacy rule
    sequence: 4
    enabled: false
```

Execution Flow (Short-Circuit Enabled):

```
Rule 1: FAIL → Continue to Rule 2
Rule 2: FAIL → Continue to Rule 3
Rule 3: PASS → STOP (return true) - Remaining rules NOT evaluated
```

5.4 Advanced Execution Features

Short-Circuit Control

```
rule-groups:
  # Production-optimized (short-circuit enabled)
  - id: "production-validation"
    operator: "AND"
    stop-on-first-failure: true  # Stop on first failure for performance
    debug-mode: false           # Disable debug for performance
    rule-ids: ["rule1", "rule2", "rule3"]

  # Complete evaluation (short-circuit disabled)
  - id: "comprehensive-validation"
    operator: "AND"
    stop-on-first-failure: false # Evaluate all rules regardless of failures
    debug-mode: false           # No debug logging
    rule-ids: ["rule1", "rule2", "rule3"]
```

Parallel Execution

```
rule-groups:
  - id: "parallel-validation"
    name: "Parallel Rule Execution"
    description: "Execute CPU-intensive rules in parallel"
    operator: "AND"
```

```
parallel-execution: true      # Enable parallel processing
stop-on-first-failure: false # Parallel execution disables short-circuiting
rule-ids:
  - "complex-calculation-rule"
  - "external-api-validation"
  - "database-lookup-rule"
```

Parallel Execution Characteristics:

- **Thread Pool:** `min(rule_count, available_processors)`
- **Short-Circuiting:** Automatically disabled to ensure all rules complete
- **Error Handling:** Individual rule failures don't crash the group
- **Use Cases:** CPU-intensive rules, independent validations

Debug Mode

```
rule-groups:
  - id: "debug-validation"
    name: "Debug Mode Validation"
    description: "Complete evaluation with debug logging"
    operator: "AND"
    debug-mode: true      # Enable debug logging
    stop-on-first-failure: false # Debug mode disables short-circuiting
    rule-ids:
      - "rule1"
      - "rule2"
      - "rule3"
```

Debug Output Example:

```
DEBUG: Rule 'trade-id-required' in group 'debug-validation' evaluated to: true
DEBUG: Rule 'isin-format-validation' in group 'debug-validation' evaluated to:
false
DEBUG: Rule 'trade-value-positive' in group 'debug-validation' evaluated to: true
DEBUG: Group 'debug-validation' evaluation complete. Evaluated: 3, Passed: 2,
Failed: 1, Final result: false
```

Debug Mode Configuration Options:

```
# Option 1: YAML configuration
debug-mode: true

# Option 2: System property (overrides YAML if not specified)
# -Dapex.rulegroup.debug=true
```

5.5 Configuration Scenarios

Production-Optimized Configuration

```
rule-groups:
  - id: "production-group"
    name: "Production Validation"
    operator: "AND"
    stop-on-first-failure: true # Enable short-circuiting for performance
    parallel-execution: false # Disable parallel for simplicity
    debug-mode: false # Disable debug for performance
    rule-ids: ["critical-rule1", "critical-rule2"]
```

Debug-Optimized Configuration

```
rule-groups:
  - id: "debug-group"
    name: "Debug Validation"
    operator: "AND"
    stop-on-first-failure: false # Disable short-circuiting for complete
evaluation
    parallel-execution: false # Disable parallel for deterministic debugging
    debug-mode: true # Enable debug logging
    rule-ids: ["test-rule1", "test-rule2", "test-rule3"]
```

Performance-Optimized Configuration

```
rule-groups:
  - id: "performance-group"
    name: "High-Performance Validation"
    operator: "OR"
    stop-on-first-failure: true # Stop on first success
    parallel-execution: true # Use parallel processing
    debug-mode: false # Disable debug for performance
    rule-ids: ["fast-rule1", "fast-rule2", "fast-rule3"]
```

5.6 Performance Comparison

Configuration	Speed	Memory	CPU	Use Case
Short-Circuit + Sequential	Fastest	Lowest	Lowest	Production systems
Complete + Sequential	Slower	Medium	Medium	Debugging, reporting
Complete + Parallel	Variable*	Higher	Higher	CPU-intensive rules

Configuration	Speed	Memory	CPU	Use Case
Debug Mode	Slowest	Highest	Medium	Development, troubleshooting

*Parallel execution speed depends on rule complexity and available CPU cores.

5.7 Best Practices

Choosing Between rule-ids and rule-references

Use rule-ids when:

- You need simple, straightforward rule grouping
- Rules should execute in definition order
- All rules should always be enabled
- You want minimal configuration overhead
- Performance is critical (slightly faster)

Use rule-references when:

- You need custom execution sequence
- You want to enable/disable individual rules
- You're building complex rule workflows
- You need fine-grained control over rule behavior
- You want future-proof configuration

Performance Best Practices

With rule-ids (Simple Approach):

```
rule-groups:
  # Order rules by likelihood of failure (most likely to fail first)
  - id: "optimized-validation-simple"
    operator: "AND"
    stop-on-first-failure: true
    rule-ids:
      - "quick-null-check"           # Fast, likely to fail
      - "format-validation"         # Medium speed
      - "complex-business-rule"     # Slow, unlikely to fail
```

With rule-references (Advanced Approach):

```
rule-groups:
  # Custom sequence for optimal performance
  - id: "optimized-validation-advanced"
    operator: "AND"
    stop-on-first-failure: true
    rule-references:
      - rule-id: "quick-null-check" # Execute first (fastest, most likely to
```

```

fail)
  sequence: 1
  enabled: true
  - rule-id: "format-validation"      # Execute second (medium speed)
    sequence: 2
    enabled: true
  - rule-id: "complex-business-rule" # Execute last (slowest, least likely to
fail)
  sequence: 3
  enabled: true
  - rule-id: "debug-only-rule"      # Disabled in production
    sequence: 4
    enabled: false

```

Error Handling Best Practices

```

rule-groups:
  # Separate critical and non-critical validations
  - id: "critical-validation"
    name: "Critical Business Rules"
    operator: "AND"
    stop-on-first-failure: true
    rule-ids: ["mandatory-field-check", "regulatory-compliance"]

  - id: "warning-validation"
    name: "Warning-Level Checks"
    operator: "OR"
    stop-on-first-failure: false # Check all warnings
    rule-ids: ["data-quality-warning", "business-recommendation"]

```

Testing Best Practices

Simple Testing with **rule-ids**:

```

rule-groups:
  # Use debug mode for comprehensive testing
  - id: "test-validation-simple"
    name: "Test Environment Validation (Simple)"
    operator: "AND"
    debug-mode: true          # Enable for testing
    stop-on-first-failure: false # See all test results
    rule-ids: ["test-rule1", "test-rule2", "test-rule3"]

```

Advanced Testing with **rule-references**:

```
rule-groups:
  # Selective testing with rule control
  - id: "test-validation-advanced"
    name: "Test Environment Validation (Advanced)"
    operator: "AND"
    debug-mode: true
    stop-on-first-failure: false
    rule-references:
      - rule-id: "test-rule1"
        sequence: 1
        enabled: true
      - rule-id: "test-rule2"
        sequence: 2
        enabled: true
      - rule-id: "experimental-rule" # Can be toggled for testing
        sequence: 3
        enabled: false
```

Migration and Compatibility

Converting from rule-ids to rule-references:

```
# Before: Simple rule-ids approach
rule-groups:
  - id: "validation-group"
    operator: "AND"
    rule-ids: ["rule1", "rule2", "rule3"]

# After: Equivalent rule-references approach
rule-groups:
  - id: "validation-group"
    operator: "AND"
    rule-references:
      - rule-id: "rule1"
        sequence: 1
        enabled: true
      - rule-id: "rule2"
        sequence: 2
        enabled: true
      - rule-id: "rule3"
        sequence: 3
        enabled: true
```

Both approaches are fully supported and can coexist in the same APEX configuration.

Implementation Status Summary

Feature	rule-ids	rule-references	Status
---------	----------	-----------------	--------

Feature	rule-ids	rule-references	Status
Basic Processing	Fully implemented	Fully implemented	COMPLETE
Sequence Control	Auto-sequence (1,2,3...)	Custom sequence property	COMPLETE
Enable/Disable	All rules enabled	Individual enabled property	COMPLETE
Priority Override	Uses rule's priority	X override-priority documented only	PLANNED FEATURE

Key Points:

- Both rule-ids and rule-references are production-ready
- sequence and enabled properties work correctly in rule-references
- override-priority is documented but not yet implemented in the engine
- All examples in this documentation have been validated with working tests

override-priority Implementation Plan

Use Cases for Priority Override: The override-priority feature enables context-sensitive rule behavior where the same rule needs different priorities in different scenarios:

Simplest Example:

```
metadata:
  name: "Priority Override Demo"
  version: "1.0.0"

rules:
  - id: "data-validation"
    name: "Data Validation Rule"
    condition: "#value != null && #value > 0"
    message: "Data validation passed"
    severity: "ERROR"
    priority: 50 # Default medium priority

rule-groups:
  # Critical processing - validation is highest priority
  - id: "critical-processing"
    name: "Critical Data Processing"
    operator: "AND"
    rule-references:
      - rule-id: "data-validation"
        sequence: 1
        enabled: true
        override-priority: 1 # Override to highest priority

  # Batch processing - validation is lower priority
  - id: "batch-processing"
    name: "Batch Data Processing"
```

```

operator: "AND"
rule-references:
  - rule-id: "data-validation"
    sequence: 1
    enabled: true
    override-priority: 100 # Override to lowest priority

```

Result: Same rule, different priorities based on processing context.

- **Regulatory Compliance:** KYC rules have higher priority for high-value transactions
- **Customer Tiers:** VIP customers get different rule priorities than standard customers
- **Environment-Specific:** Production vs development environments need different rule priorities
- **Seasonal Adjustments:** Holiday trading rules with adjusted priorities
- **Context Sensitivity:** Same validation rule, different importance based on business context

Example Use Case:

```

rules:
  - id: "credit-limit-check"
    name: "Credit Limit Validation"
    condition: "#transactionAmount <= #creditLimit"
    message: "Transaction exceeds credit limit"
    priority: 20 # Standard priority

rule-groups:
  # VIP customers - more lenient credit checks
  - id: "vip-processing"
    name: "VIP Customer Processing"
    operator: "AND"
    rule-references:
      - rule-id: "credit-limit-check"
        sequence: 3
        enabled: true
        override-priority: 80 # Lower priority - allow flexibility
      - rule-id: "fraud-detection"
        sequence: 1
        enabled: true
        override-priority: 1 # Fraud still top priority

  # Standard customers - strict credit enforcement
  - id: "standard-processing"
    name: "Standard Customer Processing"
    operator: "AND"
    rule-references:
      - rule-id: "credit-limit-check"
        sequence: 1
        enabled: true
        override-priority: 1 # HIGHEST priority - strict enforcement

```

Implementation Status: This feature is planned for a future release. The YAML syntax is documented and ready, but the engine implementation is pending.

Implementation Plan: See [Override Priority Implementation Plan](#) for detailed technical specifications, implementation steps, and testing strategy.

 **Test Specifications:** Ready-to-use test cases are available in `OverridePriorityTest.java` (currently disabled) that demonstrate expected behavior and serve as acceptance criteria for the implementation.

7. Enrichments Section

6.1 Lookup Enrichments

Lookup enrichments add data by matching keys against datasets:

```
enrichments:
- id: "lei-enrichment"
  type: "lookup-enrichment"
  condition: "#counterparty != null && #counterparty.name != null"
  lookup-config:
    lookup-key: "counterparty.name" # Field path (no # prefix in lookup-key)
    lookup-dataset:
      type: "inline"
      key-field: "name"
      data:
        - name: "Deutsche Bank AG"
          lei: "7LTFZYICNSX8D621K86"
          jurisdiction: "DE"
          entityType: "BANK"
        - name: "JPMorgan Chase"
          lei: "8EE8DF3643E15DBFDA05"
          jurisdiction: "US"
          entityType: "BANK"
    field-mappings:
      - source-field: "lei"
        target-field: "counterparty.lei"
      - source-field: "jurisdiction"
        target-field: "counterparty.jurisdiction"
      - source-field: "entityType"
        target-field: "counterparty.entityType"
```

SpEL in Field Mappings (New in v2.3)

Field mappings now support SpEL expressions for accessing nested fields and complex data structures. Use the `#` prefix to indicate a SpEL expression:

```
enrichments:
- id: "instrument-lookup"
```

```

type: "lookup-enrichment"
condition: "#symbol != null"
lookup-config:
  lookup-key: "#symbol"
  lookup-dataset:
    type: "inline"
    key-field: "symbol"
    data:
      - symbol: "AAPL"
        data:
          instrument:
            name: "Apple Inc."
            type: "EQUITY"
          pricing:
            bid: 150.25
            ask: 150.30
  field-mappings:
    # NEW: Access nested fields in lookup result with SpEL
    - source-field: "#instrument.name"
      target-field: "instrument_name"
    - source-field: "#instrument.type"
      target-field: "instrument_type"
    - source-field: "#pricing.bid"
      target-field: "bid_price"

```

SpEL Features in Field Mappings:

```

field-mappings:
  # Nested field access
  - source-field: "#trade.counterparty"
    target-field: "counterparty_name"

  # Safe navigation (prevents null pointer exceptions)
  - source-field: "#pricing?.bid"
    target-field: "bid_price"

  # Array indexing
  - source-field: "#legs[0].currency"
    target-field: "first_leg_currency"

  # Method calls
  - source-field: "#currency.toUpperCase()"
    target-field: "currency_code"

  # Complex expressions
  - source-field: "#status == 'ACTIVE' ? #activePrice : #inactivePrice"
    target-field: "current_price"

  # Combination with transformations
  - source-field: "#amount"
    target-field: "adjusted_amount"
    expression: "#value * 1.1"

```

Backward Compatibility:

```
field-mappings:
  # Old style (no # prefix) - still works
  - source-field: "lei"
    target-field: "counterparty_lei"

  # New style (with # prefix) - SpEL expression
  - source-field: "#lei"
    target-field: "counterparty_lei"
```

See Also: [APEX SpEL Guide](#) for comprehensive SpEL documentation and examples.

Lookup Enrichment Properties

Property	Required	Description
id	Yes	Unique enrichment identifier
type	Yes	Must be "lookup-enrichment"
condition	Yes	When to apply this enrichment
lookup-config	Yes	Lookup configuration
field-mappings	Yes	How to map lookup results
result-field	No	Field name to store lookup success (boolean: true if lookup found data, false otherwise)

Lookup Configuration

Property	Required	Description
lookup-key	Yes	Field path or expression for lookup key
lookup-dataset	Yes	Dataset definition

Dynamic Lookup Keys

Use expressions for complex lookup keys:

```
lookup-config:
  lookup-key: "#counterparty.lei + '_' + #venue.country" # Composite key
  # or
  lookup-key: "#instrumentId.substring(0, 2)" # Derived key
```

Enrichment Result Field Storage

The **result-field** property is available for all enrichment types (lookup, field, conditional-mapping) and allows you to store the enrichment's boolean evaluation result in the facts map for use by subsequent enrichments or rules.

What Gets Stored:

- **lookup-enrichment:** **true** if lookup found data, **false** if lookup failed
- **field-enrichment:** **true** if condition matched, **false** if condition didn't match
- **conditional-mapping-enrichment:** **true** if any mapping rule matched, **false** if no rules matched

Example - Lookup Result Field:

```
enrichments:
  - id: "lookup-counterparty"
    type: "lookup-enrichment"
    condition: "#counterparty != null"
    result-field: "counterpartyFound" # Stores lookup success
    lookup-config:
      lookup-key: "#counterparty"
      lookup-dataset:
        type: "inline"
        key-field: "counterpartyId"
        data:
          - counterpartyId: "BANK_A"
            rating: "AAA"
      field-mappings:
        - source-field: "rating"
          target-field: "counterpartyRating"

# Use the result in a subsequent enrichment
- id: "set-default-rating"
  type: "field-enrichment"
  condition: "#counterpartyFound == false" # Only if lookup failed
  field-mappings:
    - source-field: "counterpartyRating"
      target-field: "counterpartyRating"
      expression: "'UNRATED'"
```

Example - Field Enrichment Result Field:

```
enrichments:
  - id: "check-high-value"
    type: "field-enrichment"
    condition: "#notionalAmount > 10000000"
    result-field: "isHighValue" # Stores condition result
    field-mappings:
```

```

    - source-field: "notionalAmount"
      target-field: "tradeCategory"
      expression: "'HIGH_VALUE'"

# Use the result in a subsequent enrichment
- id: "set-approval-required"
  type: "field-enrichment"
  condition: "#isHighValue == true" # Only if high value
  field-mappings:
    - source-field: "requiresApproval"
      target-field: "requiresApproval"
      expression: "true"

```

Example - Conditional Mapping Result Field:

```

enrichments:
  - id: "classify-risk"
    type: "conditional-mapping-enrichment"
    target-field: "riskClass"
    result-field: "riskClassified" # Stores whether any rule matched
    mapping-rules:
      - id: "high-risk"
        priority: 1
        conditions:
          operator: "AND"
          rules:
            - condition: "#notionalAmount > 10000000"
        mapping:
          type: "direct"
          expression: "'HIGH'"

# Use the result in a subsequent enrichment
- id: "set-default-risk"
  type: "field-enrichment"
  condition: "#riskClassified == false" # Only if no rule matched
  field-mappings:
    - source-field: "riskClass"
      target-field: "riskClass"
      expression: "'NORMAL'"

```

Best Practices:

- Use descriptive field names that clearly indicate what the result represents
- Store results when you need to chain enrichments or implement fallback logic
- Access stored results using SpEL syntax: `#fieldName`
- Results are boolean values: `true` (success/match) or `false` (failure/no-match)
- Performance impact is minimal (sub-millisecond overhead per enrichment)

6.2 Calculation Enrichments

Calculation enrichments derive new fields using expressions:

```
enrichments:
- id: "trade-value-calculation"
  type: "calculation-enrichment"
  condition: "#quantity != null && #price != null"
  calculation-config:
    expression: "#quantity * #price"
    result-field: "tradeValue"
  field-mappings:
    - source-field: "tradeValue"
      target-field: "tradeValue"

- id: "risk-calculation"
  type: "calculation-enrichment"
  condition: "#tradeValue != null"
  calculation-config:
    expression: "#tradeValue * 0.025" # 2.5% VaR
    result-field: "var1Day"
  field-mappings:
    - source-field: "var1Day"
      target-field: "var1Day"
```

Calculation Enrichment Properties

Property	Required	Description
id	Yes	Unique enrichment identifier
type	Yes	Must be "calculation-enrichment"
condition	No	When to apply this enrichment (SpEL expression)
calculation-config	Yes	Calculation configuration
field-mappings	Yes	How to map the calculated result

Calculation Config Properties

Property	Required	Description
expression	Yes	SpEL expression to calculate the value
result-field	Yes	Field name where result will be stored

Complex Calculations

```
enrichments:
  # Conditional calculation with ternary operators
  - id: "settlement-priority"
```

```

    type: "calculation-enrichment"
    calculation-config:
      expression: "#tradeValue > 100000000 ? 'HIGH' : (#tradeValue > 10000000 ?
'MEDIUM' : 'NORMAL')"
      result-field: "settlementPriority"
    field-mappings:
      - source-field: "settlementPriority"
        target-field: "settlementPriority"

# Date calculation
- id: "settlement-date"
  type: "calculation-enrichment"
  calculation-config:
    expression: "#tradeDate.plusDays(#settlementCycle)"
    result-field: "settlementDate"
  field-mappings:
    - source-field: "settlementDate"
      target-field: "settlementDate"

# String manipulation
- id: "trade-reference"
  type: "calculation-enrichment"
  calculation-config:
    expression: "#counterpartyCode + '-' + #tradeId + '-' +
T(java.time.LocalDate).now().format(T(java.time.format.DateTimeFormatter).ofPatter
n('yyyyMMdd'))"
    result-field: "tradeReference"
  field-mappings:
    - source-field: "tradeReference"
      target-field: "tradeReference"

```

Note: For multiple related calculations, you can use multiple calculation enrichments or use **field-enrichment** with transformations for simpler cases.

6.3 Field Enrichments

Field enrichments transform, copy, or map fields using direct transformations or conditional logic. This is the most flexible enrichment type and is commonly used for field-level transformations, conditional enrichments based on rule results, and simple field mappings.

Basic Field Enrichment

```

enrichments:
- id: "status-mapping"
  type: "field-enrichment"
  condition: "#statusCode != null"
  field-mappings:
    - source-field: "statusCode"
      target-field: "status"
  expression: |
    #statusCode == 'A' ? 'ACTIVE' :

```

```

    #statusCode == 'I' ? 'INACTIVE' :
    #statusCode == 'P' ? 'PENDING' : 'UNKNOWN'

- target-field: "processedAt"
  expression: "T(java.time.LocalDateTime).now()"

- source-field: "amount"
  target-field: "formattedAmount"
  expression: "T(java.lang.String).format('$%,.2f', #amount)"

```

SpEL in Field Mappings (New in v2.3)

Field mappings now support SpEL expressions in **source-field** for accessing nested fields and complex data structures:

```

enrichments:
- id: "nested-field-mapping"
  type: "field-enrichment"
  condition: "#field != null"
  field-mappings:
    # NEW: Access nested fields with SpEL (use # prefix)
    - source-field: "#currency"
      target-field: "buy_currency"

    - source-field: "#trade.counterparty"
      target-field: "counterparty_name"

    # Safe navigation prevents null pointer exceptions
    - source-field: "#trade?.amount"
      target-field: "trade_amount"

    # Array indexing
    - source-field: "#legs[0].currency"
      target-field: "first_leg_currency"

    # Method calls
    - source-field: "#currency.toUpperCase()"
      target-field: "currency_code"

    # Complex expressions
    - source-field: "#status == 'ACTIVE' ? #activePrice : #inactivePrice"
      target-field: "current_price"

    # Combine SpEL source-field with transformation
    - source-field: "#amount"
      target-field: "adjusted_amount"
      expression: "#value * 1.1" # Apply 10% markup

```

Backward Compatibility:

```
field-mappings:
  # Old style (no # prefix) - still works
  - source-field: "currency"
    target-field: "currency_code"

  # New style (with # prefix) - SpEL expression
  - source-field: "#currency"
    target-field: "currency_code"

  # Both can be used in the same enrichment
  - source-field: "status"           # Simple field
    target-field: "trade_status"
  - source-field: "#nested.field"   # SpEL expression
    target-field: "nested_value"
```

See Also: [APEX SpEL Guide](#) for comprehensive SpEL documentation and examples.

Field Enrichment Properties

Property	Required	Description
id	Yes	Unique enrichment identifier
type	Yes	Must be "field-enrichment"
condition	No	When to apply this enrichment (SpEL expression)
field-mappings	Yes*	List of field mapping configurations
conditional-mappings	Yes*	List of conditional mapping configurations
result-field	No	Field name to store condition evaluation result (boolean: true if condition matched, false otherwise)
success-code	No	Code to set in the result object upon successful execution
error-code	No	Code to set in the result object upon failure
map-to-field	No	Target field(s) for mapping results (String or List)

*At least one of `field-mappings` or `conditional-mappings` is required.

Field Mapping Properties

Property	Required	Description
source-field	No	Source field name (optional if using expression only)
target-field	Yes	Target field name where value will be stored
expression	No	SpEL expression to transform the value

Property	Required	Description
<code>required</code>	No	Whether this mapping is mandatory (default: false)

Field Enrichment with Rule Results

Field enrichments can use rule evaluation results for conditional logic:

```
rules:
  - id: "high-value-rule"
    condition: "#amount > 10000"
  - id: "premium-customer-rule"
    condition: "#customerType == 'PREMIUM'"

enrichments:
  - id: "conditional-processing"
    type: "field-enrichment"
    condition: "#ruleResults != null"
    field-mappings:
      # Apply different processing based on rule results
      - target-field: "processingPriority"
        expression: |
          #ruleResults['premium-customer-rule'] == true && #ruleResults['high-
value-rule'] == true ? 'IMMEDIATE' :
          #ruleResults['high-value-rule'] == true ? 'HIGH' :
          #ruleResults['premium-customer-rule'] == true ? 'ELEVATED' : 'STANDARD'

      # Conditional fee calculation
      - target-field: "processingFee"
        expression: "#ruleResults['high-value-rule'] == true ? #amount * 0.05 :
#amount * 0.02"

      # Set flags based on rule results
      - target-field: "requiresApproval"
        expression: "#ruleResults['high-value-rule'] == true"
```

Conditional Mappings

Field enrichments support conditional mappings for complex branching logic:

```
enrichments:
  - id: "conditional-field-mapping"
    type: "field-enrichment"
    conditional-mappings:
      - condition: "#status == 'A'"
        field-mappings:
          - target-field: "displayStatus"
            expression: "'Active'"
          - target-field: "canTransact"
```

```

        expression: "true"

    - condition: "#status == 'I'"
      field-mappings:
        - target-field: "displayStatus"
          expression: "'Inactive'"
        - target-field: "canTransact"
          expression: "false"

    - default: true
      field-mappings:
        - target-field: "displayStatus"
          expression: "'Unknown'"
        - target-field: "canTransact"
          expression: "false"

```

When to Use Field Enrichment

Use **field-enrichment** when you need to:

- Transform or copy fields without external lookups
- Apply conditional logic based on rule results
- Map fields with complex SpEL transformations
- Set calculated fields that don't require external data
- Apply different mappings based on conditions

vs. lookup-enrichment: Use lookup when you need to fetch data from external sources **vs. calculation-enrichment:** Use calculation when you have multiple related calculations **vs. conditional-mapping-enrichment:** Use conditional-mapping for priority-based rule matching

6.4 Conditional Mapping Enrichments

Conditional mapping enrichments provide priority-based conditional field mapping with first-match-wins logic. This enrichment type is useful for complex routing scenarios where multiple conditions need to be evaluated in priority order.

Basic Conditional Mapping Enrichment

```

enrichments:
  - id: "priority-based-routing"
    type: "conditional-mapping-enrichment"
    target-field: "routingDestination"

    mapping-rules:
      # Rule 1: Highest priority
      - id: "urgent-high-value"
        priority: 1
        conditions:
          operator: "AND"
          rules:

```

```
      - condition: "#priority == 'URGENT'"
      - condition: "#amount > 100000"
    mapping:
      type: "direct"
      expression: "'IMMEDIATE_PROCESSING_QUEUE'"

# Rule 2: Medium priority
- id: "high-value"
  priority: 2
  conditions:
    operator: "AND"
    rules:
      - condition: "#amount > 100000"
  mapping:
    type: "direct"
    expression: "'HIGH_VALUE_QUEUE'"

# Rule 3: Default (lowest priority)
- id: "standard"
  priority: 999
  mapping:
    type: "direct"
    expression: "'STANDARD_QUEUE'"

execution-settings:
  stop-on-first-match: true
  log-matched-rule: true
  validate-result: false
```

Conditional Mapping Properties

Property	Required	Description
id	Yes	Unique enrichment identifier
type	Yes	Must be "conditional-mapping-enrichment"
target-field	Yes	Field where the mapped value will be stored
mapping-rules	Yes	List of priority-based mapping rules
execution-settings	No	Execution configuration
result-field	No	Field name to store mapping success (boolean: true if any rule matched, false otherwise)

Mapping Rule Properties

Property	Required	Description
id	Yes	Unique rule identifier

Property	Required	Description
priority	Yes	Priority order (lower numbers = higher priority)
conditions	No	Conditions that must be met for this rule to match
mapping	Yes	Mapping configuration

Mapping Configuration

Property	Required	Description
type	Yes	Mapping type: "direct" or "lookup"
expression	Yes*	SpEL expression for direct mappings
source-field	No	Source field for the mapping

Execution Settings

Property	Default	Description
stop-on-first-match	true	Stop processing after first matching rule
log-matched-rule	false	Log which rule was matched
validate-result	false	Validate the mapping result

When to Use Conditional Mapping Enrichment

Use `conditional-mapping-enrichment` when you need to:

- Evaluate multiple conditions in priority order
- Apply first-match-wins logic
- Route data based on complex priority rules
- Have clear separation between condition evaluation and value mapping

vs. field-enrichment: Use field-enrichment for simpler conditional logic with ternary operators **vs. lookup-enrichment:** Use lookup when mapping values come from external datasets **vs. calculation-enrichment:** Use calculation for mathematical derivations

8. Scenario Configurations

8.1 Overview

Scenario configurations define end-to-end processing pipelines for specific data types. They enable systematic routing of different data types through appropriate validation, enrichment, and compliance processing stages.

7.2 Basic Scenario Structure

```

metadata:
  id: "trade-processing-scenario"
  name: "Trade Processing Scenario"
  type: "scenario"
  business-domain: "Trading"
  owner: "trading-team@bank.com"

scenario:
  scenario-id: "trade-processing"
  name: "Trade Processing Pipeline"
  description: "Complete trade processing with validation and enrichment"

data-types:
  - "Trade"
  - "java.util.Map"

```

Enrichment Groups Section

Enrichment groups compose individual enrichments into reusable, ordered collections with AND/OR semantics, optional short-circuiting, and optional parallel execution. This mirrors Rule Groups, but operates on `enrichments` instead of `rules`.

```

> **⚠ CRITICAL: Inline Enrichment Definitions Not Supported**
>
> Enrichment groups do **NOT** support inline enrichment definitions. Enrichments
must be defined in the `enrichments` section and referenced by ID in enrichment
groups.
>
> **INCORRECT (Inline Definitions):**
> ```yaml
> enrichment-groups:
>   - id: "my-enrichment-group"
>     enrichments: # NOT SUPPORTED!
>       - id: "my-enrichment"
>         type: "field-enrichment"
>         field-mappings:
>           - source-field: "sourceField"
>             target-field: "targetField"
> ```
>
> **CORRECT (Reference by ID):**
> ```yaml
> # Define enrichments in the enrichments section
> enrichments:
>   - id: "my-enrichment"
>     type: "field-enrichment"
>     field-mappings:
>       - source-field: "sourceField"
>         target-field: "targetField"
>
> # Reference enrichments in enrichment groups

```

```
> enrichment-groups:
>   - id: "my-enrichment-group"
>     operator: "AND"
>     enrichment-ids:
>       - "my-enrichment"
> ```
>
> **Why This Design?** Enrichments are designed to be reusable across multiple
enrichment groups. Defining enrichments once in the `enrichments` section and
referencing them by ID promotes reusability and maintainability.
```

Example:

```
```yaml
file: enrichments.yaml
enrichments:
 - id: e1
 type: field-enrichment
 field-mappings: [{ source-field: a, target-field: a_copy, required: true }]
 - id: e2
 type: field-enrichment
 field-mappings: [{ source-field: b, target-field: b_copy, required: true }]
 - id: e3
 type: field-enrichment
 field-mappings: [{ source-field: c, target-field: c_copy, required: true }]

file: enrichment-groups.yaml
enrichment-groups:
 - id: base_and
 operator: AND
 stop-on-first-failure: true
 enrichment-ids: [e1, e2]

 - id: composite
 operator: AND
 enrichment-ids: [e3]
 enrichment-group: base_and # Singular reference (alias for enrichment-group-
references)

 - id: composite_plural
 operator: AND
 enrichment-ids: [e3]
 enrichment-group-references: [base_and]

 - id: composite_par_and
 operator: AND
 parallel-execution: true
 enrichment-ids: [e3]
 enrichment-group-references: [base_and]
```

Properties (per group):

Property	Type	Required	Default	Description
id	string	Yes	—	Group identifier
operator	string (AND or OR)	Yes	—	Logical operator applied to member results
stop-on-first-failure	boolean	No	true	Enable short-circuiting (stops on first failure for AND, first success for OR)
parallel-execution	boolean	No	false	Evaluate all member enrichments concurrently; disables short-circuiting
error-handling	string	No	"fail-fast"	Exception handling strategy: "fail-fast", "continue-on-error", "skip-on-error"
enrichment-ids	list	Optional	—	Direct enrichments included in listed order
enrichment-group	string	Optional	—	Singular alias for enrichment-group-references (convenience)
enrichment-group-references	list	Optional	—	Include enrichments from referenced groups appended in order

Execution semantics:

- AND: all member enrichments must succeed for overall success; with stop-on-first-failure=true, short-circuiting on first failure.
- OR: overall success if any member succeeds; with stop-on-first-failure=true, short-circuiting on first success.
- parallel-execution=true:
  - All member enrichments are evaluated concurrently.
  - Short-circuiting is disabled; results are aggregated after completion.

Reference processing and validation:

- Group references are flattened after all groups are created (two-phase processing) to establish final execution order.
- Validation ensures: referenced enrichment ids exist; referenced groups exist; no self-reference; cyclic enrichment-group-references are rejected.

Error handling strategy:

- error-handling controls exception handling during enrichment evaluation (e.g., SpEL errors, null pointer exceptions, data access errors)
- stop-on-first-failure controls business logic short-circuiting (AND/OR evaluation behavior)
- Valid values: "fail-fast" (default - stop on exception), "continue-on-error" (log and continue), "skip-on-error" (skip failed enrichment)

Example: Error Handling in Enrichment Groups

```

enrichment-groups:
 # Critical enrichment - any error stops processing
 - id: "critical-enrichment"
 operator: "AND"
 error-handling: "fail-fast" # Default - stop on any exception
 enrichment-ids:
 - "mandatory-customer-lookup"
 - "required-account-enrichment"

 # Best-effort enrichment - continue despite errors
 - id: "optional-enrichment"
 operator: "OR"
 error-handling: "continue-on-error" # Log errors but continue
 enrichment-ids:
 - "optional-address-lookup"
 - "supplementary-data-enrichment"

 # Resilient enrichment - skip problematic enrichments
 - id: "resilient-enrichment"
 operator: "AND"
 error-handling: "skip-on-error" # Skip failed enrichments
 enrichment-ids:
 - "external-api-enrichment"
 - "optional-calculation"

```

See also:

- Rule Groups Section (conceptual parity, advanced execution semantics)
- Enrichments Section (available enrichment types and their properties)

Notes:

- Sequence/ordering follows the Rule Group pattern: enrichment-ids are applied first in their listed order; referenced groups are appended in order of reference.
- Multi-file configurations are supported: load yaml files without validation, merge sections, then call `processReferencesAndValidate`.

## Legacy approach (deprecated but supported)

---

rule-configurations:

- "config/trade-validation-rules.yaml"
- "config/trade-enrichment-rules.yaml"

### ### 7.3 Stage-Based Processing (Modern Approach)

Stage-based processing provides explicit control over execution order,

dependencies, and failure handling:

```
```yaml
scenario:
  scenario-id: "otc-options-processing"
  name: "OTC Options Processing"
  description: "Multi-stage OTC options processing pipeline"

data-types:
  - "OTCOption"
  - "java.util.Map"

processing-stages:
  - stage-name: "validation"
    config-file: "config/otc-validation-rules.yaml"
    execution-order: 1
    failure-policy: "terminate"
    required: true
    stage-metadata:
      description: "Critical data validation"
      sla-ms: 500
      critical: true

  - stage-name: "market-data-enrichment"
    config-file: "config/market-data-enrichment.yaml"
    execution-order: 2
    failure-policy: "continue-with-warnings"
    depends-on: ["validation"]
    stage-metadata:
      description: "Market data enrichment"
      sla-ms: 2000
      data-sources: ["bloomberg", "reuters"]

  - stage-name: "compliance-check"
    config-file: "config/compliance-rules.yaml"
    execution-order: 3
    failure-policy: "flag-for-review"
    depends-on: ["validation"]
    stage-metadata:
      description: "Regulatory compliance validation"
      sla-ms: 1000
      critical: true

  - stage-name: "us-regulatory-check"
    config-file: "config/us-regulatory.yaml"
    execution-order: 4
    condition: "['region'] == 'US'" # Conditional execution
    failure-policy: "terminate"
    depends-on: ["compliance-check"]
    stage-metadata:
      description: "US-specific regulatory checks"
      sla-ms: 800
      critical: true
```

7.4 Stage Configuration Properties

Property	Required	Type	Description
stage-name	Yes	String	Unique identifier for the stage
config-file	Yes	String	Path to rule configuration file
execution-order	Yes	Integer	Numeric execution order
condition	No	String	SpEL expression for conditional execution
failure-policy	No	String	How to handle stage failures
required	No	Boolean	Whether stage is mandatory
depends-on	No	List	Stage dependencies
stage-metadata	No	Map	Additional stage metadata

7.5 Failure Policies

Policy	Behavior	Use Case
terminate	Stop processing immediately	Critical validations
continue-with-warnings	Log warnings, continue processing	Optional enrichments
flag-for-review	Mark for manual review, continue	Risk assessments

7.6 Stage Dependencies

Stages can depend on other stages for sequential or conditional processing:

```
processing-stages:
- stage-name: "validation"
  execution-order: 1
  failure-policy: "terminate"

- stage-name: "enrichment"
  execution-order: 2
  depends-on: ["validation"] # Only runs if validation passes

- stage-name: "compliance"
  execution-order: 3
  depends-on: ["validation"] # Parallel with enrichment
```

7.7 Conditional Stage Execution

Stages can include optional SpEL conditions that control whether the stage executes. This enables dynamic, data-driven workflow logic:

```
processing-stages:
- stage-name: "us-compliance"
  config-file: "config/us-compliance.yaml"
  execution-order: 2
  condition: "'region' == 'US'" # Only execute for US region
  failure-policy: "terminate"

- stage-name: "emea-compliance"
  config-file: "config/emea-compliance.yaml"
  execution-order: 3
  condition: "'region' == 'EMEA'" # Only execute for EMEA region
  failure-policy: "terminate"

- stage-name: "high-value-check"
  config-file: "config/high-value-validation.yaml"
  execution-order: 4
  condition: "'amount' > 10000" # Only for high-value trades
  failure-policy: "flag-for-review"
```

Condition Evaluation Behavior:

- **Condition True:** Stage executes normally
- **Condition False:** Stage is skipped (logged as "skipped")
- **No Condition:** Stage always executes (backward compatible)
- **Evaluation Error:** Stage is skipped with error logged (safe default)

Complex Conditions:

Conditions support full SpEL syntax with access to `#context`, and other variables:

```
processing-stages:
- stage-name: "us-high-value-compliance"
  config-file: "config/us-high-value.yaml"
  condition: "'region' == 'US' && #'amount' > 10000"
  failure-policy: "terminate"

- stage-name: "exotic-options-pricing"
  config-file: "config/exotic-pricing.yaml"
  condition: "'productType' == 'EXOTIC' && #'underlyingAsset' != null"
  failure-policy: "continue-with-warnings"
```

Conditions with Dependencies:

Conditions are evaluated **before** dependencies. A stage will only check dependencies if its condition is met:

```
processing-stages:
- stage-name: "validation"
  config-file: "config/validation.yaml"
```

```

    execution-order: 1
    # No condition - always runs

  - stage-name: "us-compliance"
    config-file: "config/us-compliance.yaml"
    execution-order: 2
    condition: "'region' == 'US'" # Checked first
    depends-on: ["validation"]    # Checked only if condition is true
    failure-policy: "terminate"

```

Best Practices:

1. **Use conditions for data-driven routing** - Route trades to region-specific compliance checks
2. **Combine with dependencies** - Ensure prerequisite stages complete before conditional stages
3. **Keep conditions simple** - Complex logic should be in rules, not stage conditions
4. **Handle evaluation errors gracefully** - Conditions that fail to evaluate will skip the stage
5. **Log skipped stages** - Monitor which stages are being skipped to verify routing logic

7.8 Multi-Environment Configuration

Different environments can have different processing requirements:

```

# Development Environment
scenario:
  scenario-id: "trade-processing-dev"
  processing-stages:
    - stage-name: "basic-validation"
      config-file: "config/dev/lenient-validation.yaml"
      failure-policy: "continue-with-warnings" # Lenient for development

# Production Environment
scenario:
  scenario-id: "trade-processing-prod"
  processing-stages:
    - stage-name: "strict-validation"
      config-file: "config/prod/strict-validation.yaml"
      failure-policy: "terminate" # Strict for production

```

7.9 Classification-Based Routing

Scenarios can include classification rules to automatically route data to the appropriate scenario based on data characteristics:

```

metadata:
  id: "otc-option-us-scenario"
  name: "OTC Option US Processing"
  type: "scenario"
  business-domain: "Trading"

```

```

scenario:
  scenario-id: "otc-option-us"
  name: "OTC Option US Processing"
  description: "Processing for US OTC options"

  # Classification rules determine if this scenario applies
  classification-rules:
    - condition: "#tradeType == 'OTC_OPTION' && #region == 'US'"
      description: "US OTC Options"
    - condition: "#assetClass == 'OPTION' && #jurisdiction == 'US'"
      description: "US jurisdiction options"

  data-types:
    - "java.util.Map"

  processing-stages:
    - stage-name: "us-validation"
      config-file: "config/us-otc-validation.yaml"
      execution-order: 1
      failure-policy: "terminate"

    - stage-name: "us-compliance"
      config-file: "config/us-compliance-rules.yaml"
      execution-order: 2
      failure-policy: "flag-for-review"
      depends-on: ["us-validation"]

```

7.9 Scenario Registries

Scenario registries organize multiple scenarios for classification-based routing:

```

metadata:
  id: "trade-processing-registry"
  name: "Trade Processing Scenario Registry"
  type: "scenario-registry"
  version: "1.0.0"
  description: "Registry of all trade processing scenarios"
  created-by: "trading-team@bank.com"

scenarios:
  # High-priority specific scenarios
  - config-file: "scenarios/otc-option-us.yaml"
    priority: 1
    enabled: true

  - config-file: "scenarios/otc-option-emea.yaml"
    priority: 2
    enabled: true

  - config-file: "scenarios/bond-us.yaml"

```

```

    priority: 3
    enabled: true

# Generic fallback scenarios
- config-file: "scenarios/generic-trade.yaml"
  priority: 100
  enabled: true

```

Classification Evaluation Order:

1. Scenarios are evaluated in priority order (lowest number = highest priority)
2. First scenario whose classification rules match the data is selected
3. If no scenario matches, processing fails with "No matching scenario" error

7.10 Using Scenarios with RulesEngine API

Loading a Scenario Registry:

```

// Load scenario registry and all referenced scenario configurations
RulesEngine engine = RulesEngine.fromScenarioRegistry("config/trade-scenarios-registry.yaml");

```

Classification-Based Routing:

```

// Automatically select and execute the matching scenario
Map<String, Object> tradeData = new HashMap<>();
tradeData.put("tradeType", "OTC_OPTION");
tradeData.put("region", "US");
tradeData.put("notional", 1000000.0);

ScenarioExecutionResult result = engine.evaluateWithClassification(tradeData);

// Check results
if (result.isSuccess()) {
    System.out.println("Scenario: " + result.getScenarioId());
    System.out.println("All stages passed");
} else {
    System.out.println("Failures: " + result.getFailures());
}

```

Direct Scenario Execution:

```

// Execute a specific scenario by ID
ScenarioExecutionResult result = engine.evaluateScenario("otc-option-us", tradeData);

```

Accessing Stage Results:

```
// Get results for individual stages
Map<String, Object> stageResults = result.getStageResults();
for (Map.Entry<String, Object> entry : stageResults.entrySet()) {
    System.out.println("Stage: " + entry.getKey());
    System.out.println("Result: " + entry.getValue());
}

// Check if specific stage passed
boolean validationPassed = result.isStageSuccessful("validation");
```

7.11 Migration from DataTypeScenarioService

Deprecated API (DataTypeScenarioService):

```
// OLD - Deprecated approach
DataTypeScenarioService scenarioService = new DataTypeScenarioService();
scenarioService.loadScenarios("config/scenarios-registry.yaml");
ScenarioExecutionResult result = scenarioService.processMapData(tradeData);
```

New API (RulesEngine):

```
// NEW - Recommended approach
RulesEngine engine = RulesEngine.fromScenarioRegistry("config/scenarios-registry.yaml");
ScenarioExecutionResult result = engine.evaluateWithClassification(tradeData);
```

Benefits of RulesEngine API:

- Unified entry point for all APEX processing (rules, enrichments, pipelines, scenarios)
- Consistent error handling and result types
- Better integration with other APEX features
- Improved performance and resource management
- Modern fluent API design

9. Component Configurations

9.1 Overview

Component Configurations are reusable YAML files that group multiple configuration files together with controlled execution order and failure policies. Components enable:

- **Modular Configuration:** Group related rules, enrichments, and other configs into logical units
- **Reusability:** Reference the same component from multiple scenarios

- **Execution Control:** Define explicit execution order and failure handling
- **Nesting Support:** Components can reference other components (with depth limits)
- **Backward Compatibility:** Existing YAML files work unchanged

8.2 Component Structure

```
metadata:
  id: "component-id"
  name: "Component Name"
  version: "1.0.0"
  description: "Component description"
  type: "component"
  business-domain: "Domain"
  owner: "team@company.com"
  criticality: "high"
  sla-ms: 200
  tags:
    - tag1
    - tag2

# Optional: Rule configuration file references
rule-configurations:
  - file: "path/to/validation-rules.yaml"
    execution-order: 1
    failure-policy: "terminate"
  - file: "path/to/business-rules.yaml"
    execution-order: 2
    failure-policy: "continue-with-warnings"

# Optional: Enrichment file references
enrichment-refs:
  - file: "path/to/enrichments.yaml"
    execution-order: 3
    failure-policy: "continue-with-warnings"

# Optional: Other component references (nesting)
component-refs:
  - file: "path/to/nested-component.yaml"
    execution-order: 4
    failure-policy: "terminate"

# Optional: Generic config file references
config-files:
  - file: "path/to/any-config.yaml"
    execution-order: 5
    failure-policy: "flag-for-review"
```

8.3 File Reference Structure

Each file reference in a component supports:

Field	Required	Type	Description
file	Yes	String	Path to configuration file (relative or absolute)
execution-order	No	Integer	Numeric execution order (lower numbers execute first)
failure-policy	No	String	Failure handling: <code>terminate</code> , <code>continue-with-warnings</code> , <code>flag-for-review</code>

Execution Order Rules:

- Files with explicit `execution-order` are sorted numerically (ascending)
- Files without `execution-order` execute in document order
- Mixed mode: Ordered files execute first, then unordered files in document order

Failure Policy Options:

- `terminate` - Stop processing immediately on failure (default for critical operations)
- `continue-with-warnings` - Log warning and continue processing (default for enrichments)
- `flag-for-review` - Mark for manual review but continue processing

8.4 Basic Component Example

Simple Validation Component:

```
metadata:
  id: "basic-validation-component"
  name: "Basic Trade Validation"
  version: "1.0.0"
  type: "component"
  description: "Basic trade validation rules"
  business-domain: "Trading"
  owner: "trading-team@bank.com"

rule-configurations:
  - file: "validation/required-fields.yaml"
    execution-order: 1
    failure-policy: "terminate"
  - file: "validation/data-quality.yaml"
    execution-order: 2
    failure-policy: "terminate"
```

8.5 Multi-Stage Component Example

Component with Enrichment and Validation:

```
metadata:
  id: "multi-stage-component"
  name: "Multi-Stage Processing Component"
```

```

version: "1.0.0"
type: "component"
description: "Component with enrichment followed by validation"
business-domain: "data-processing"
owner: "APEX Demo Team"
criticality: "high"
sla-ms: 200

config-files:
- file: "enrichment/trade-enrichment.yaml"
  execution-order: 1
  failure-policy: "continue-with-warnings"
- file: "validation/trade-validation.yaml"
  execution-order: 2
  failure-policy: "terminate"

```

8.6 Nested Component Example

Parent Component Referencing Child Component:

```

metadata:
  id: "nested-component-level1"
  name: "Nested Component Level 1"
  version: "1.0.0"
  type: "component"
  description: "Top-level component that references nested components"
  business-domain: "testing"
  owner: "APEX Demo Team"

config-files:
- file: "components/nested-component-level2.yaml"
  execution-order: 1
  failure-policy: "terminate"
- file: "validation/final-validation.yaml"
  execution-order: 2
  failure-policy: "terminate"

```

Child Component:

```

metadata:
  id: "nested-component-level2"
  name: "Nested Component Level 2"
  version: "1.0.0"
  type: "component"
  description: "Child component with enrichment rules"

enrichment-refs:
- file: "enrichment/basic-enrichment.yaml"
  failure-policy: "continue-with-warnings"

```

8.7 Using Components in Scenarios

Components can be referenced in scenario stages just like regular config files:

```
metadata:
  id: "trade-processing-scenario"
  name: "Trade Processing Scenario"
  type: "scenario"
  business-domain: "Trading"

scenario:
  scenario-id: "trade-processing"
  name: "Trade Processing Pipeline"

processing-stages:
  - stage-name: "validation"
    config-file: "components/validation-component.yaml" # Component file
    execution-order: 1
    required: true

  - stage-name: "enrichment"
    config-file: "components/enrichment-component.yaml" # Component file
    execution-order: 2
    required: true
```

8.8 Nesting Depth Limits

To prevent excessive complexity and circular references:

- **Levels 1-2:** Normal operation (no warnings)
- **Levels 3-5:** WARNING logged (review recommended)
- **Level 6+:** CRITICAL ERROR - processing terminates

Example Nesting:

```
Level 1: scenario.yaml
├─ Level 2: component-level1.yaml
│   └─ Level 3: component-level2.yaml (WARNING)
│       └─ Level 4: component-level3.yaml (WARNING)
│           └─ Level 5: component-level4.yaml (WARNING)
│               └─ Level 6: component-level5.yaml (ERROR - REJECTED)
```

8.9 Circular Reference Detection

APEX automatically detects and prevents circular component references:

```
# component-a.yaml
component-refs:
  - file: "component-b.yaml"

# component-b.yaml
component-refs:
  - file: "component-a.yaml" # CIRCULAR REFERENCE - REJECTED
```

Error Message:

Circular component reference detected: component-a.yaml → component-b.yaml → component-a.yaml

8.10 Best Practices

1. Keep Components Focused

- Group related configurations together
- Avoid mixing unrelated business logic
- Use descriptive names and IDs

2. Use Execution Order Wisely

- Enrichments before validations
- Critical validations first
- Optional enrichments last

3. Choose Appropriate Failure Policies

- `terminate` for critical validations
- `continue-with-warnings` for optional enrichments
- `flag-for-review` for manual intervention cases

4. Limit Nesting Depth

- Prefer flat structures (1-2 levels)
- Avoid deep nesting (3+ levels)
- Use component-refs sparingly

5. Document Dependencies

- Add clear descriptions
- Use tags for categorization
- Specify business domain and owner

10. Dataset Definitions

10.1 Inline Datasets

Inline datasets embed data directly in the configuration:

```
lookup-dataset:
  type: "inline"
  key-field: "instrumentId" # Field used for lookup matching
  data:
    - instrumentId: "GB00B03MLX29"
      name: "Royal Dutch Shell PLC"
      currency: "GBP"
      assetClass: "EQUITY"
      country: "GB"
    - instrumentId: "US0378331005"
      name: "Apple Inc"
      currency: "USD"
      assetClass: "EQUITY"
      country: "US"
```

Dataset Properties

Property	Required	Description
type	Yes	"inline" for embedded data
key-field	Yes	Field name used for lookup matching
data	Yes	Array of data objects

Multi-Key Datasets

For composite keys, use expressions in the lookup-key:

```
lookup-config:
  lookup-key: "#lei + '_' + #country"
  lookup-dataset:
    type: "inline"
    key-field: "compositeKey"
    data:
      - compositeKey: "7LTFZYICNSX8D621K86_GB"
        settlementMethod: "CREST"
        account: "CREST001234"
      - compositeKey: "7LTFZYICNSX8D621K86_US"
        settlementMethod: "DTC"
        account: "DTC567890"
```

10.2 External Datasets

Reference external data sources:

```
lookup-dataset:
  type: "external"
  source: "reference-data-service"
  endpoint: "/api/securities"
  key-field: "isin"
  cache-ttl: 3600 # Cache for 1 hour
  timeout: 5000 # 5 second timeout
```

External Dataset Properties

Property	Required	Description
type	Yes	"external" for external sources
source	Yes	Data source identifier
endpoint	No	API endpoint or query
key-field	Yes	Field used for lookup matching

11. Rule Chains

The `rule-chains` section allows for complex, multi-step rule execution flows. This feature is critical for implementing advanced business logic that requires conditional branching, sequential dependencies, or accumulative scoring.

The behavior of a rule chain is determined by its `pattern` field.

11.1 Conditional Chaining (`conditional-chaining`)

Executes a "trigger" rule. If it matches, it executes a set of "on-trigger" rules. If it doesn't match, it executes "on-no-trigger" rules.

Syntax:

```
rule-chains:
  - id: "high-value-processing"
    pattern: "conditional-chaining"
    configuration:
      trigger-rule: "high-value-check" # Rule ID
      conditional-rules:
        on-trigger:
          - "enhanced-due-diligence"
          - "senior-approval-required"
        on-no-trigger:
          - "standard-check"
```

11.2 Sequential Dependency (`sequential-dependency`)

Executes rules in a strict sequence where subsequent rules depend on the output of previous ones. Output variables from one stage are available in the context for subsequent stages.

Syntax:

```
rule-chains:
  - id: "discount-pipeline"
    pattern: "sequential-dependency"
    configuration:
      stages:
        - rule: "base-discount-calc"
          output-variable: "baseDiscount" # Stored in context as #baseDiscount
        - rule: "loyalty-bonus-calc"
          output-variable: "loyaltyBonus"
        - rule: "final-price-calc"
          output-variable: "finalPrice"
```

11.3 Result-Based Routing (**result-based-routing**)

Routes execution to different rule sets based on the *result* of a routing rule. The routing rule must return a string value that matches one of the defined routes.

Syntax:

```
rule-chains:
  - id: "risk-routing"
    pattern: "result-based-routing"
    configuration:
      router-rule:
        condition: "#riskScore > 80 ? 'HIGH' : (#riskScore > 50 ? 'MEDIUM' : 'LOW')"
        output-variable: "riskLevel"
        message: "Determining risk level"
      routes:
        "HIGH":
          rules:
            - id: "high-risk-check-1"
              condition: "true"
              message: "High risk check"
        "MEDIUM":
          rules:
            - id: "medium-risk-check"
              condition: "true"
              message: "Medium risk check"
        "LOW":
          rules:
            - id: "auto-approve"
              condition: "true"
              message: "Auto approved"
```

11.4 Accumulative Chaining (**accumulative-chaining**)

Accumulates a score or value from multiple rules and makes a final decision. Useful for credit scoring or risk assessment.

Syntax:

```
rule-chains:
- id: "credit-scoring"
  pattern: "accumulative-chaining"
  configuration:
    accumulator: "0" # Initial value (can be a number or SpEL expression)
    accumulation-rules:
      - rule: "payment-history-good"
        weight: 10 # Added to accumulator if rule matches
      - rule: "high-debt-ratio"
        weight: -5 # Subtracted if rule matches
    decision-rule: "approve-if-score-above-700" # Evaluated against final
    accumulator value
```

11.5 Complex Workflow (**complex-workflow**)

Defines a directed acyclic graph (DAG) of rules with explicit dependencies. Steps execute only when their dependencies have successfully completed.

Syntax:

```
rule-chains:
- id: "onboarding-workflow"
  pattern: "complex-workflow"
  configuration:
    steps:
      - id: "kyc-check"
        rule: "perform-kyc"
      - id: "credit-check"
        rule: "perform-credit-check"
        depends-on: ["kyc-check"] # Runs only after KYC completes
      - id: "account-setup"
        rule: "setup-account"
        depends-on: ["kyc-check", "credit-check"] # Runs after both complete
```

11.6 Fluent Builder (**fluent-builder**)

Constructs a complex object or result by chaining rules that each contribute to building the final state. Supports **on-success** and **on-failure** branching at each step.

Syntax:

```
rule-chains:
  - id: "policy-builder"
    pattern: "fluent-builder"
    configuration:
      builder-target: "policyObject" # The object being built
      steps:
        - rule: "validate-eligibility"
          on-success:
            rule: "apply-standard-coverage"
          on-failure:
            rule: "apply-standard-coverage"
  - **Infrastructure Configuration**: External, reusable data-source configurations
  - **Business Logic Configuration**: Lean, focused enrichment and validation rules
```

12.2 Benefits of External References

Clean Architecture

- **Separation of Concerns**: Infrastructure and business logic cleanly separated
- **Reusable Components**: External data-source configurations shared across multiple rule configurations
- **Maintainable Code**: Lean business logic configurations easy to understand and modify

Enterprise Scalability

- **Configuration Caching**: External configurations cached for performance
- **Connection Pooling**: Shared database connections across multiple enrichments
- **Environment Management**: Different infrastructure configurations for dev/test/prod

```
name: "Business Logic Configuration"
version: "2.0.0"
description: "Lean configuration using external data-source references"
```

```
# External data-source references (infrastructure configuration - reusable)
data-source-refs:
```

- name: "database-name"
 source: "data-sources/database-config.yaml"
 enabled: true
 description: "Reference to external database configuration"

```
# Business logic enrichments (lean and focused)
enrichments:
```

- id: "enrichment-id"
 type: "lookup-enrichment"
 condition: "#field != null"
 lookup-config:
 lookup-key: "#field"
 lookup-dataset:

```
| `enabled` | No | Whether this reference is active (default: true) | true |
| `description` | No | Human-readable description | "Customer database for profile enrichment" |
```

12.4 External Data-Source Configuration Files

External data-source configuration files contain infrastructure-specific settings:

```
#### Database Data-Source Configuration
```

```
```yaml
```

```
File: data-sources/postgresql-customer-database.yaml
```

```
metadata:
```

```
 name: "PostgreSQL Customer Database"
```

```
 version: "1.0.0"
```

```
 type: "external-data-config"
```

```
 description: "PostgreSQL customer database configuration"
```

```
Database connection configuration
```

```
connection:
```

```
 type: "database"
```

```
 driver: "postgresql"
```

```
 # For PostgreSQL
```

```
 host: "localhost"
```

```
 port: 5432
```

```
 database: "customer_data"
```

```
 username: "postgres"
```

```
 password: "password"
```

```
 pool:
```

```
 initial-size: 5
```

```
 max-size: 20
```

```
 timeout: 30000
```

```
H2 Database Configuration Examples
```

```
=====
```

```
File-based H2 (RECOMMENDED for demos)
```

```
h2-file-connection:
```

```
 type: "database"
```

```
 driver: "h2"
```

```
 # File-based H2 enables true database sharing between processes
```

```
 database: "./target/h2-demo/apex_demo_shared"
```

```
 username: "sa"
```

```
 password: ""
```

```
In-memory H2 (NOT RECOMMENDED - creates isolated instances)
```

```
h2-memory-connection:
```

```
 type: "database"
```

```
 driver: "h2"
```

```
 # WARNING: Each connection creates a separate in-memory instance
```

```
 database: "shared_demo" # Becomes jdbc:h2:mem:shared_demo
```

```
 username: "sa"
```

```
 password: ""
```

```
H2 TCP Server (for multi-process access)
```

```
h2-tcp-connection:
```

```
 type: "database"
```

```
 driver: "h2"
```

```
 host: "localhost"
```

```
 port: 9092
```

```

 database: "shared_demo"
 username: "sa"
 password: ""

Enhanced H2 with Custom Parameters (NEW!)
h2-custom-connection:
 type: "database"
 driver: "h2"
 # Custom parameters can be specified after the database path
 # Format: "path/to/database;PARAM1=value1;PARAM2=value2"
 database: "./target/h2-
demo/custom;MODE=MySQL;CACHE_SIZE=32768;TRACE_LEVEL_FILE=2"
 username: "sa"
 password: ""
 # This generates: jdbc:h2:./target/h2-
demo/custom;MODE=MySQL;CACHE_SIZE=32768;TRACE_LEVEL_FILE=2;DB_CLOSE_DELAY=-1

H2 In-memory with Custom Parameters
h2-memory-custom-connection:
 type: "database"
 driver: "h2"
 # In-memory database with custom parameters
 database: "mem:testdb;CACHE_SIZE=16384;MODE=Oracle;TRACE_LEVEL_SYSTEM_OUT=1"
 username: "sa"
 password: ""
 # This generates:
jdbc:h2:mem:testdb;CACHE_SIZE=16384;MODE=Oracle;TRACE_LEVEL_SYSTEM_OUT=1;DB_CLOSE_
DELAY=-1

H2 Parameter Reference
=====

Common H2 Parameters for Performance Tuning:
- MODE: Database compatibility mode (PostgreSQL, MySQL, Oracle, DB2, HSQLDB)
- CACHE_SIZE: Database cache size in KB (default: 16384 = 16MB)
- MAX_MEMORY_ROWS: Maximum rows kept in memory (default: 40000)
- MAX_MEMORY_UNDO: Maximum undo log entries in memory (default: 50000)

Common H2 Parameters for Debugging:
- TRACE_LEVEL_FILE: SQL logging level to file (0=off, 1=error, 2=info, 4=debug)
- TRACE_LEVEL_SYSTEM_OUT: SQL logging to console (0=off, 1=error, 2=info)
- TRACE_MAX_FILE_SIZE: Maximum trace file size in MB (default: 16)

Common H2 Parameters for Connection Management:
- DB_CLOSE_DELAY: Keep database open after last connection (-1=forever,
0=immediate, >0=seconds)
- DB_CLOSE_ON_EXIT: Close database when JVM exits (TRUE/FALSE)
- AUTO_SERVER: Enable automatic mixed mode (TRUE/FALSE)

Common H2 Parameters for Initialization:
- INIT: SQL script to run on database startup
- IFEXISTS: Only connect if database exists (TRUE/FALSE)
- ACCESS_MODE_DATA: Database access mode (r=read-only, rw=read-write)

```

```
Example Configurations:
performance-tuned-h2:
 database: "./target/h2-
demo/performance;MODE=PostgreSQL;CACHE_SIZE=65536;MAX_MEMORY_ROWS=100000"

debug-enabled-h2:
 database: "./target/h2-
demo/debug;TRACE_LEVEL_FILE=2;TRACE_LEVEL_SYSTEM_OUT=1;TRACE_MAX_FILE_SIZE=32"

mysql-compatible-h2:
 database: "./target/h2-demo/mysql;MODE=MySQL;CACHE_SIZE=32768"

read-only-h2:
 database: "./target/h2-demo/readonly;ACCESS_MODE_DATA=r;IFEXISTS=TRUE"

auto-init-h2:
 database: "./target/h2-demo/autoinit;INIT=RUNSCRIPT FROM 'classpath:schema.sql'"

Named queries for reuse
queries:
 getActiveCustomerById:
 sql: |
 SELECT
 customer_id,
 customer_name,
 customer_type,
 tier,
 region,
 status,
 created_date
 FROM customers
 WHERE customer_id = :customerId
 AND status = 'ACTIVE'
 parameters:
 - name: "customerId"
 type: "string"
 required: true
 description: "Customer identifier"

Connection health check
health-check:
 query: "SELECT 1"
 timeout: 5000
 interval: 30000
```

## 12.5 Using External References in Enrichments

### Simple Database Lookup with External Reference

```
metadata:
 name: "Customer Profile Enrichment - External Reference"
```

```

version: "2.1.0"
description: "Customer profile enrichment using external data-source reference"

External data-source references
data-source-refs:
 - name: "postgresql-customer-database"
 source: "data-sources/postgresql-customer-database.yaml"
 enabled: true

Business logic enrichments
enrichments:
 - id: "customer-profile-lookup"
 type: "lookup-enrichment"
 description: "Customer profile enrichment using external data-source
reference"
 condition: "#customerId != null && #customerId != ''"

lookup-config:
 lookup-key: "#customerId"
 lookup-dataset:
 type: "database"
 data-source-ref: "postgresql-customer-database" # External reference
 query-ref: "getActiveCustomerId" # Named query
 parameters:
 - field: "customerId"
 type: "string"

Field mappings from database columns to enriched object fields
field-mappings:
 - source-field: "CUSTOMER_NAME"
 target-field: "customerName"
 required: true
 - source-field: "CUSTOMER_TYPE"
 target-field: "customerType"
 required: true
 - source-field: "TIER"
 target-field: "customerTier"
 required: true

```

## 12.6 Advanced External Reference Patterns

### Multiple External Data-Sources

```

metadata:
 name: "Multi-Source Transaction Processing"
 version: "2.0.0"
 description: "Transaction processing with multiple external data-sources"

Multiple external data-source references
data-source-refs:
 - name: "customer-database"

```

```

 source: "data-sources/customer-database.yaml"
 enabled: true
 - name: "settlement-database"
 source: "data-sources/settlement-database.yaml"
 enabled: true
 - name: "market-data-api"
 source: "data-sources/market-data-api.yaml"
 enabled: true

Business logic using multiple external sources
enrichments:
 - id: "customer-enrichment"
 type: "lookup-enrichment"
 lookup-config:
 lookup-dataset:
 data-source-ref: "customer-database"
 query-ref: "getCustomerProfile"

 - id: "settlement-enrichment"
 type: "lookup-enrichment"
 lookup-config:
 lookup-dataset:
 data-source-ref: "settlement-database"
 query-ref: "getSettlementInstructions"

 - id: "market-data-enrichment"
 type: "lookup-enrichment"
 lookup-config:
 lookup-dataset:
 data-source-ref: "market-data-api"
 query-ref: "getCurrentPrice"

```

## 12.7 Configuration Caching and Performance

### Automatic Configuration Caching

External data-source configurations are automatically cached for performance:

```

External configurations are loaded once and cached
data-source-refs:
 - name: "shared-database"
 source: "data-sources/shared-database.yaml" # Loaded once, cached
 enabled: true

Multiple enrichments can reference the same external configuration
enrichments:
 - id: "enrichment-1"
 lookup-config:
 lookup-dataset:
 data-source-ref: "shared-database" # Uses cached configuration

```

```
- id: "enrichment-2"
 lookup-config:
 lookup-dataset:
 data-source-ref: "shared-database" # Uses cached configuration
```

## Performance Benefits

- **Configuration Loading:** External configurations loaded once and cached
- **Connection Pooling:** Database connections shared across enrichments
- **Query Preparation:** Named queries prepared once and reused
- **Memory Efficiency:** Reduced memory footprint through shared configurations

## 12.8 Field Mapping and Case Sensitivity

### Production-Ready Field Mapping

External data-source references support case-sensitive field mapping for production environments:

```
enrichments:
- id: "database-lookup"
 type: "lookup-enrichment"
 lookup-config:
 lookup-dataset:
 data-source-ref: "postgresql-database"
 query-ref: "getRecord"

Field mappings handle case sensitivity
field-mappings:
- source-field: "CUSTOMER_NAME" # Uppercase database column
 target-field: "customerName" # camelCase target field
 required: true
- source-field: "CUSTOMER_TYPE" # Uppercase database column
 target-field: "customerType" # camelCase target field
 required: true
```

## 12.9 Error Handling and Validation

### External Reference Validation

APEX validates external data-source references at configuration load time:

```
data-source-refs:
- name: "invalid-reference"
 source: "non-existent-file.yaml" # X Will cause validation error
 enabled: true

- name: "valid-reference"
```

```
source: "data-sources/valid-config.yaml" # Will validate successfully
enabled: true
```

## Error Handling Patterns

```
enrichments:
- id: "resilient-lookup"
 type: "lookup-enrichment"
 condition: "#customerId != null"

lookup-config:
 lookup-dataset:
 data-source-ref: "customer-database"
 query-ref: "getCustomer"

Error handling configuration
error-handling:
 on-error: "continue" # Continue processing on error
 fallback-value: null # Default value on lookup failure
 log-errors: true # Log errors for monitoring
```

## 8.5 Data Sinks (Output Destinations)

**Data Sinks** provide output capabilities for APEX, enabling processed data to be written to various destinations including databases, files, message queues, and REST APIs. This complements the existing data-sources functionality by providing a complete data pipeline solution.

### Overview

Data sinks follow the same architectural patterns as data sources, supporting:

- **Multiple Output Types:** Database, file system, message queue, REST API, cache
- **Batch Processing:** Efficient bulk operations with configurable batch sizes
- **Error Handling:** Comprehensive retry mechanisms and dead letter queues
- **Schema Management:** Auto-creation and validation of database schemas
- **Format Support:** JSON, CSV, XML, SQL, and custom formats

### Basic Data Sink Configuration

```
metadata:
 name: "Data Pipeline with Output"
 version: "1.0.0"
 description: "Complete data pipeline with input and output"

data-sinks:
- name: "customer-database-sink"
 type: "database"
```

```

source-type: "h2"
enabled: true
description: "H2 database for processed customer data"

connection:
 database: "./target/output/customer_data"
 username: "sa"
 password: ""
 mode: "PostgreSQL"

operations:
 insertCustomer: "INSERT INTO customers (id, name, email, processed_at)
VALUES (:id, :name, :email, :processedAt)"
 updateCustomer: "UPDATE customers SET name = :name, email = :email WHERE id
= :id"
 upsertCustomer: "MERGE INTO customers (id, name, email, processed_at) KEY
(id) VALUES (:id, :name, :email, :processedAt)"

schema:
 auto-create: true
 table-name: "customers"
 init-script: |
 CREATE TABLE IF NOT EXISTS customers (
 id INTEGER PRIMARY KEY,
 name VARCHAR(255) NOT NULL,
 email VARCHAR(255),
 processed_at TIMESTAMP
);

error-handling:
 strategy: "log-and-continue"
 max-retries: 3
 retry-delay: 1000
 dead-letter-table: "failed_records"

batch:
 enabled: true
 batch-size: 50
 timeout-ms: 10000
 transaction-mode: "per-batch"

```

## File System Data Sink

```

data-sinks:
- name: "audit-file-sink"
 type: "file-system"
 source-type: "json"
 enabled: true
 description: "Audit trail file output"

connection:

```

```
base-path: "./target/output/audit"
file-pattern: "audit_{timestamp}.json"
encoding: "UTF-8"

operations:
 writeAuditRecord: "WRITE_JSON"
 appendAuditRecord: "APPEND_JSON"

output-format:
 format: "json"
 pretty-print: true
 encoding: "UTF-8"
 include-timestamp: true

batch:
 enabled: true
 batch-size: 100
 flush-interval-ms: 5000
```

Data Sink Properties

Property	Required	Description	Example
name	Yes	Unique identifier for the data sink	"customer-database-sink"
type	Yes	Type of data sink	"database", "file-system", "message-queue"
source-type	No	Specific implementation type	"h2", "postgresql", "csv", "json"
enabled	No	Whether this sink is active (default: true)	true
description	No	Human-readable description	"Customer data output sink"
connection	Yes	Connection configuration	See connection examples
operations	Yes	Named operations (SQL, templates, etc.)	See operations examples
schema	No	Schema management configuration	See schema examples
error-handling	No	Error handling strategy	See error handling examples
batch	No	Batch processing configuration	See batch examples
output-format	No	Output format settings	See format examples

Error Handling Strategies

Strategy	Description	Use Case
fail-fast	Stop processing on first error	Critical data integrity requirements
log-and-continue	Log error and continue processing	Best effort processing
dead-letter	Send failed records to dead letter queue	Error recovery and analysis
retry-and-fail	Retry failed operations, then fail	Transient error handling
retry-and-continue	Retry failed operations, then continue	Resilient processing

Complete Pipeline Example

```
metadata:
 name: "CSV to Database Pipeline"
 version: "1.0.0"
 description: "Complete pipeline from CSV input to database output"

Input data source
data-source-refs:
 - name: "customer-csv-input"
 source: "data-sources/customer-csv.yaml"
 enabled: true

Data transformation
enrichments:
 - id: "customer-data-enrichment"
 type: "field-enrichment"
 description: "Enrich and validate customer data"
 condition: "true"

 calculations:
 - field: "processedAt"
 expression: "new java.util.Date()"
 - field: "status"
 expression: "'PROCESSED'"

Output data sink
data-sinks:
 - name: "customer-h2-output"
 type: "database"
 source-type: "h2"
 enabled: true

 connection:
 database: "./target/output/processed_customers"
 username: "sa"
 password: ""

 operations:
 insertCustomer: "INSERT INTO customers (id, name, email, processed_at,
status) VALUES (:id, :name, :email, :processedAt, :status)"
```

```
schema:
 auto-create: true
 table-name: "customers"

batch:
 enabled: true
 batch-size: 100
```

---

## 13. Pipeline Orchestration

### 13.1 Overview

**Pipeline Orchestration** is APEX's approach to YAML-driven data processing workflows. This system embodies the core APEX principle that **all processing logic should be contained in the YAML configuration file**, eliminating hardcoded orchestration in Java code.

#### Key Benefits

- **YAML-Driven Processing:** Complete pipeline workflows defined in YAML
- **Dependency Management:** Automatic step dependency resolution and validation
- **Error Handling:** Configurable error handling strategies with optional steps
- **Data Flow:** Automatic data passing between pipeline steps
- **Monitoring:** Built-in step timing and execution tracking
- **Validation:** Pipeline configuration validation with circular dependency detection

#### Core Principle

##### Before (Hardcoded Java):

```
pipelineEngine.execute("getAllCustomers", "customer-csv-input",
 "customer-h2-database", "insertCustomer");
```

##### After (YAML-Driven):

```
pipelineEngine.executePipeline("customer-etl-pipeline");
```

### 13.2 Pipeline Configuration Structure

#### Basic Pipeline Syntax

```
pipeline:
 name: "pipeline-name"
 description: "Pipeline description"
```

```

steps:
 - name: "step-name"
 type: "step-type"
 # Step-specific configuration

execution:
 mode: "sequential" # or "parallel"
 error-handling: "stop-on-error" # or "continue-on-error"

monitoring:
 enabled: true
 log-progress: true

```

## Complete Pipeline Example

```

metadata:
 name: "CSV to H2 ETL Pipeline Demo"
 version: "1.0.0"
 description: "Complete ETL pipeline using APEX orchestration"

Pipeline orchestration - defines the complete ETL workflow
pipeline:
 name: "customer-etl-pipeline"
 description: "Extract customer data from CSV, transform, and load into H2
database"

Pipeline steps executed in sequence
steps:
 - name: "extract-customers"
 type: "extract"
 source: "customer-csv-input"
 operation: "getAllCustomers"
 description: "Read all customer records from CSV file"

 - name: "load-to-database"
 type: "load"
 sink: "customer-h2-database"
 operation: "insertCustomer"
 description: "Insert customer records into H2 database"
 depends-on: ["extract-customers"]

 - name: "audit-logging"
 type: "audit"
 sink: "audit-log-file"
 operation: "writeAuditRecord"
 description: "Write audit records to JSON file"
 depends-on: ["load-to-database"]
 optional: true

Pipeline execution configuration

```

```
execution:
 mode: "sequential"
 error-handling: "stop-on-error"
 max-retries: 3
 retry-delay-ms: 1000

Pipeline monitoring and metrics
monitoring:
 enabled: true
 log-progress: true
 collect-metrics: true
 alert-on-failure: true

Data sources and sinks referenced by pipeline steps
data-sources:
 - name: "customer-csv-input"
 type: "file-system"
 # ... data source configuration

data-sinks:
 - name: "customer-h2-database"
 type: "database"
 # ... database sink configuration

 - name: "audit-log-file"
 type: "file-system"
 # ... file sink configuration
```

9.3 Pipeline Steps

Step Types

Type	Purpose	Required Fields	Description
extract	Data extraction	source, operation	Read data from external sources
load	Data loading	sink, operation	Write data to external sinks
transform	Data transformation	transformation	Transform data between steps
audit	Audit logging	sink, operation	Write audit records

Extract Steps

Extract steps read data from external data sources:

```
steps:
 - name: "extract-customers"
 type: "extract"
 source: "customer-csv-input" # Data source name
 operation: "getAllCustomers" # Named query/operation
```

```
description: "Read customer data from CSV"
parameters:
 limit: 1000
 offset: 0
```

## Load Steps

Load steps write data to external data sinks:

```
steps:
- name: "load-to-database"
 type: "load"
 sink: "customer-h2-database" # Data sink name
 operation: "insertCustomer" # Named operation
 description: "Insert customers into database"
 depends-on: ["extract-customers"]
 parameters:
 batch-size: 100
 upsert: true
```

## Transform Steps

Transform steps modify data between extraction and loading:

```
steps:
- name: "transform-data"
 type: "transform"
 description: "Apply business transformations"
 depends-on: ["extract-customers"]
 transformations:
 - name: "add-processing-timestamp"
 type: "field-addition"
 field: "processed_at"
 value: "CURRENT_TIMESTAMP"

 - name: "validate-email"
 type: "validation"
 field: "email"
 rule: "email-format"
```

## Audit Steps

Audit steps create audit trails and logging:

```
steps:
- name: "audit-logging"
```

```
type: "audit"
sink: "audit-log-file"
operation: "writeAuditRecord"
description: "Create audit trail"
depends-on: ["load-to-database"]
optional: true # Won't fail pipeline if it fails
```

## 9.4 Step Dependencies

### Dependency Declaration

Steps can declare dependencies on other steps:

```
steps:
- name: "step-a"
 type: "extract"
 # ... configuration

- name: "step-b"
 type: "transform"
 depends-on: ["step-a"] # Wait for step-a to complete

- name: "step-c"
 type: "load"
 depends-on: ["step-a", "step-b"] # Wait for both steps
```

### Dependency Validation

APEX automatically validates dependencies:

- **Circular Dependency Detection:** Prevents infinite loops
- **Missing Dependency Validation:** Ensures all referenced steps exist
- **Topological Sorting:** Executes steps in correct dependency order

## 9.5 Error Handling

### Pipeline-Level Error Handling

```
pipeline:
 execution:
 error-handling: "stop-on-error" # Stop pipeline on any error
 # OR
 error-handling: "continue-on-error" # Continue with remaining steps
 max-retries: 3
 retry-delay-ms: 1000
```

### Step-Level Error Handling

```

steps:
 - name: "optional-step"
 type: "audit"
 optional: true # Pipeline continues if this step fails
 retry:
 max-attempts: 3
 delay-ms: 1000
 backoff-multiplier: 2.0

```

## 9.6 Data Flow

### Automatic Data Passing

Data flows automatically between pipeline steps:

1. **Extract Step** → Stores data in pipeline context
2. **Transform Step** → Reads from context, transforms, stores result
3. **Load Step** → Reads transformed data, writes to sink
4. **Audit Step** → Reads original/transformed data for auditing

### Data Context

```

Data automatically available in pipeline context:
- extractedData: Raw data from extract steps
- transformedData: Processed data from transform steps
- stepResults: Results from each completed step

```

## 9.7 Monitoring and Metrics

### Built-in Monitoring

```

pipeline:
 monitoring:
 enabled: true
 log-progress: true # Log step start/completion
 collect-metrics: true # Collect timing metrics
 alert-on-failure: true # Alert on pipeline failures

```

### Execution Results

Pipeline execution provides detailed results:

```

YamlPipelineExecutionResult result = pipelineEngine.executePipeline("pipeline-
name");

```

```
// Overall pipeline status
boolean success = result.isSuccess();
long duration = result.getDurationMs();
int totalSteps = result.getTotalSteps();

// Individual step results
for (PipelineStepResult stepResult : result.getStepResults()) {
 String stepName = stepResult.getStepName();
 boolean stepSuccess = stepResult.isSuccess();
 long stepDuration = stepResult.getDurationMs();
}
```

## 10.5 Error Recovery Configuration

### 10.5.1 Overview

The **error-recovery** section provides configurable error handling and resilience strategies for APEX rule processing. This optional top-level section allows you to define how the system should respond to errors based on severity levels, enabling environment-specific behavior without code changes.

#### Key Features:

- **Severity-based policies:** Different recovery strategies for ERROR, WARNING, and INFO severities
- **Backward compatibility:** When not present, system uses default fail-fast behavior
- **Environment flexibility:** Configure strict error handling in development, resilient processing in production
- **Retry mechanisms:** Configurable retry attempts with delays
- **Observability:** Optional logging and metrics for recovery attempts

### 10.5.2 Configuration Structure

```
error-recovery:
 # Global settings
 enabled: true # Enable/disable error recovery globally
 log-recovery-attempts: true # Log recovery attempts for debugging
 metrics-enabled: true # Collect recovery metrics
 default-strategy: "CONTINUE_WITH_DEFAULT" # Default recovery strategy

 # Severity-specific policies
 severity-policies:
 ERROR:
 recovery-enabled: false # Strict error handling (backward
 compatible)
 strategy: "FAIL_FAST" # Fail immediately without recovery

 WARNING:
 recovery-enabled: true # Enable recovery for warnings
 strategy: "CONTINUE_WITH_DEFAULT" # Use default values when recovery
```

```
needed
 max-retries: 1 # Retry once before giving up
 retry-delay: 100 # Wait 100ms between retries

INFO:
 recovery-enabled: true # Enable recovery for info messages
 strategy: "CONTINUE_WITH_DEFAULT" # Use default values when recovery
needed
 max-retries: 0 # No retries, just use defaults
 retry-delay: 50 # Minimal delay
```

10.5.3 Configuration Properties

Global Properties

Property	Type	Required	Default	Description
enabled	Boolean	No	true	Enable/disable error recovery globally
log-recovery-attempts	Boolean	No	false	Log recovery attempts for debugging
metrics-enabled	Boolean	No	true	Collect recovery metrics for monitoring
default-strategy	String	No	"FAIL_FAST"	Default recovery strategy when not specified

Severity Policy Properties

Property	Type	Required	Default	Description
recovery-enabled	Boolean	No	false	Enable recovery for this severity level
strategy	String	No	"FAIL_FAST"	Recovery strategy (see strategies below)
max-retries	Integer	No	0	Maximum number of retry attempts
retry-delay	Long	No	0	Delay in milliseconds between retries

10.5.4 Recovery Strategies

Strategy	Description	Use Case
FAIL_FAST	Throw exception immediately (no recovery)	Critical validations, development environments
CONTINUE_WITH_DEFAULT	Use default/null values and continue processing	Resilient production processing, non-critical enrichments
RETRY_WITH_SAFE_EXPRESSION	Retry with simplified expressions (future)	Complex expression failures

Strategy	Description	Use Case
SKIP_RULE	Skip the failing rule and continue (future)	Optional validation rules

10.5.5 Environment-Specific Examples

Development Environment

Strict error handling for faster feedback:

```
error-recovery:
 enabled: true
 log-recovery-attempts: true # Verbose logging for debugging
 metrics-enabled: true
 severity-policies:
 ERROR:
 recovery-enabled: false # Strict error handling in development
 WARNING:
 recovery-enabled: true
 max-retries: 0 # No retries for faster feedback
 INFO:
 recovery-enabled: true
```

Production Environment

Resilient processing with retries:

```
error-recovery:
 enabled: true
 log-recovery-attempts: false # Reduce log noise in production
 metrics-enabled: true # Keep metrics for monitoring
 severity-policies:
 ERROR:
 recovery-enabled: false # Maintain strict error handling
 WARNING:
 recovery-enabled: true
 strategy: "CONTINUE_WITH_DEFAULT"
 max-retries: 2 # More retries in production
 retry-delay: 200
 INFO:
 recovery-enabled: true
 strategy: "CONTINUE_WITH_DEFAULT"
```

Test Environment

Disable all recovery for test validation:

```

error-recovery:
 enabled: false # Disable all recovery for testing
 # When disabled, all errors will be thrown for test validation

```

## 10.5.6 Complete Example

```

metadata:
 id: "customer-validation-with-recovery"
 name: "Customer Validation with Error Recovery"
 version: "1.0"
 type: "rule-config"

Error recovery configuration
error-recovery:
 enabled: true
 log-recovery-attempts: true
 metrics-enabled: true
 default-strategy: "CONTINUE_WITH_DEFAULT"

severity-policies:
 ERROR:
 recovery-enabled: false
 strategy: "FAIL_FAST"
 WARNING:
 recovery-enabled: true
 strategy: "CONTINUE_WITH_DEFAULT"
 max-retries: 1
 retry-delay: 100
 INFO:
 recovery-enabled: true
 strategy: "CONTINUE_WITH_DEFAULT"

Rules with different severities
rules:
 - id: "mandatory-field-check"
 name: "Mandatory Field Validation"
 condition: "#customerId != null && #customerId != ''"
 severity: "ERROR" # Will NOT recover (backward compatible)
 message: "Customer ID is mandatory"

 - id: "data-quality-check"
 name: "Data Quality Validation"
 condition: "#email != null && #email.contains('@')"
 severity: "WARNING" # Will recover with default behavior
 message: "Email format appears invalid"

 - id: "enrichment-info"
 name: "Enrichment Information"
 condition: "#region != null"

```

```
severity: "INFO" # Will recover with default behavior
message: "Region information available"
```

## 10.5.7 Benefits

1. **Environment-specific behavior:** Strict in development, resilient in production
2. **Backward compatibility:** ERROR severity maintains existing fail-fast behavior
3. **Granular control:** Different policies per severity level
4. **Operational flexibility:** Can be tuned without code changes
5. **Better observability:** Logging and metrics for recovery attempts
6. **Reduced downtime:** Graceful degradation for non-critical failures

## 10.5.8 Best Practices

1. **Keep ERROR strict:** Maintain `recovery-enabled: false` for ERROR severity to catch critical issues
2. **Use WARNING for recoverable failures:** Data quality issues, optional enrichments
3. **Enable logging in development:** Set `log-recovery-attempts: true` for debugging
4. **Reduce logging in production:** Set `log-recovery-attempts: false` to reduce noise
5. **Monitor metrics:** Always enable `metrics-enabled: true` for production monitoring
6. **Test without recovery:** Disable error recovery in test environments to validate error handling

# 14. Advanced Features

## 14.1 Conditional Logic

### Ternary Operators

Use ternary operators for conditional expressions:

```
Basic ternary
expression: "#condition ? 'value1' : 'value2'"

Nested ternary for multiple conditions
expression: "#score >= 90 ? 'A' : (#score >= 80 ? 'B' : (#score >= 70 ? 'C' : 'F'))"

Complex conditions
expression: "#type == 'EQUITY' && #quantity > 1000 ? 'LARGE_EQUITY' : 'OTHER'"

Null-safe ternary
expression: "#field != null ? #field : 'DEFAULT'"
```

### Complex Branching

Handle multiple conditions efficiently:

```

calculations:
 - field: "riskCategory"
 expression: |
 #assetClass == 'EQUITY' ?
 (#marketCap > 10000000000 ? 'LARGE_CAP_EQUITY' : 'SMALL_CAP_EQUITY') :
 #assetClass == 'BOND' ?
 (#creditRating.startsWith('AA') ? 'HIGH_GRADE_BOND' :
'INVESTMENT_GRADE_BOND') :
 #assetClass == 'DERIVATIVE' ?
 'DERIVATIVE' :
 'OTHER'

```

## Performance Optimization

Optimize conditions for better performance:

```

Good: Check simple conditions first
condition: "#isActive && #complexCalculation() > threshold"

Better: Use short-circuit evaluation
condition: "#isActive && (#value != null && #value > 0) && #complexCalculation() > threshold"

Best: Cache expensive calculations
calculations:
 - field: "expensiveResult"
 expression: "#complexCalculation()"
 - field: "finalResult"
 expression: "#isActive && #expensiveResult > threshold"

```

## Rule Result-Based Conditional Logic

Use rule evaluation results to drive conditional enrichment logic:

```

rules:
 - id: "high-value-rule"
 name: "High Value Transaction Rule"
 condition: "#amount > 10000"
 message: "Transaction amount exceeds $10,000"
 severity: "INFO"

 - id: "premium-customer-rule"
 name: "Premium Customer Rule"
 condition: "#customerType == 'PREMIUM'"
 message: "Customer has premium status"
 severity: "INFO"

 - id: "urgent-processing-rule"

```

```

 name: "Urgent Processing Rule"
 condition: "#priority == 'URGENT' || #amount > 50000"
 message: "Transaction requires urgent processing"
 severity: "INFO"

rule-groups:
- id: "validation-group"
 name: "Transaction Validation Group"
 operator: "OR"
 stop-on-first-failure: false
 rule-ids:
 - "high-value-rule"
 - "premium-customer-rule"

enrichments:
Simple rule result reference
- id: "high-value-enrichment"
 type: "field-enrichment"
 condition: "#ruleResults['high-value-rule'] == true"
 field-mappings:
 - target-field: "processingFee"
 expression: "#amount * 0.05"

Multiple rule results in complex condition
- id: "priority-enrichment"
 type: "field-enrichment"
 condition: "#ruleResults != null"
 field-mappings:
 - target-field: "processingPriority"
 expression: |
 #ruleResults['urgent-processing-rule'] == true ? 'IMMEDIATE' :
 #ruleResults['high-value-rule'] == true ? 'HIGH' :
 #ruleResults['premium-customer-rule'] == true ? 'ELEVATED' :
 'STANDARD'

Rule group result reference
- id: "validation-status-enrichment"
 type: "field-enrichment"
 condition: "#ruleGroupResults['validation-group']['passed'] == true"
 field-mappings:
 - target-field: "validationStatus"
 expression: "'VALIDATED'"
 - target-field: "validatedBy"
 expression: "'APEX_VALIDATION_GROUP'"

Fallback logic when validation fails
- id: "validation-failure-enrichment"
 type: "field-enrichment"
 condition: "#ruleGroupResults['validation-group']['passed'] == false"
 field-mappings:
 - target-field: "validationStatus"
 expression: "'FAILED'"
 - target-field: "failedRules"
 expression: "#ruleGroupResults['validation-group']['failedRules']"

```

### Key Patterns:

1. **Single Rule Reference:** Check if a specific rule passed

```
condition: "#ruleResults['rule-id'] == true"
```

2. **Multiple Rule Logic:** Combine multiple rule results

```
condition: "#ruleResults['rule-1'] == true && #ruleResults['rule-2'] == true"
```

3. **Rule Group Status:** Check if entire group passed

```
condition: "#ruleGroupResults['group-id']['passed'] == true"
```

4. **Failed Rules Access:** Get list of failed rules from group

```
expression: "#ruleGroupResults['group-id']['failedRules']"
```

5. **Null-Safe Access:** Check if rule was evaluated

```
condition: "#ruleResults.containsKey('rule-id') && #ruleResults['rule-id'] == true"
```

### Use Cases:

- **Conditional Enrichments:** Apply different enrichments based on validation results
- **Multi-Stage Processing:** Route data through different processing paths
- **Complex Decision Trees:** Build sophisticated business logic using rule outcomes
- **Fallback Handling:** Provide default values when validation fails
- **Audit Trails:** Track which rules triggered specific processing

## 14.2 Function Usage

### Built-in Functions

APEX provides access to standard Java functions:

```
String functions
expression: "#text.toUpperCase()"
expression: "#text.substring(0, 10)"
expression: "#text.matches('[A-Z]{2}[0-9]{10}')"

Math functions
expression: "T(java.lang.Math).max(#value1, #value2)"
expression: "T(java.lang.Math).round(#value * 100) / 100.0"

Date functions
expression: "T(java.time.LocalDate).now().plusDays(2)"
expression: "#date.format(T(java.time.format.DateTimeFormatter).ofPattern('yyyy-MM-dd'))"

Collection functions
expression: "#list.size()"
expression: "#list.contains('value')"
expression: "#list.?[field > 100].size()" # Filter and count
```

## Custom Function Integration

Access custom utility classes:

```
Custom financial calculations
expression: "T(com.company.utils.FinancialUtils).calculateYield(#price, #coupon, #maturity)"

Custom validation functions
condition: "T(com.company.validators.ISINValidator).isValid(#isin)"

Custom formatting functions
expression: "T(com.company.formatters.CurrencyFormatter).format(#amount, #currency)"
```

## Error Handling in Functions

Handle potential errors gracefully:

```
Safe division
expression: "#denominator != 0 ? #numerator / #denominator : 0"

Safe string operations
expression: "#text != null && #text.length() > 10 ? #text.substring(0, 10) : #text"

Try-catch equivalent using ternary
expression: "#value != null && #value.matches('[0-9]+') ? T(java.lang.Integer).parseInt(#value) : 0"
```

---

## 15. Best Practices

### 15.1 Performance Guidelines

#### Condition Optimization

Write efficient conditions:

```
Good: Simple conditions first
condition: "#isActive && #expensiveCheck()"

Better: Use null checks to avoid expensive operations
condition: "#field != null && #field.expensiveOperation() > 0"

Best: Cache results of expensive operations
calculations:
 - field: "cachedResult"
 expression: "#expensiveOperation()"
 - field: "finalCheck"
 expression: "#isActive && #cachedResult > threshold"
```

#### Dataset Sizing

Optimize dataset performance:

```
Good: Small inline datasets (< 100 records)
lookup-dataset:
 type: "inline"
 key-field: "code"
 data:
 - code: "USD"
 name: "US Dollar"
 # ... < 100 records

Better: Use external datasets for large data
lookup-dataset:
 type: "external"
 source: "reference-data-service"
 cache-ttl: 3600 # Cache for performance
```

#### Expression Efficiency

Write efficient expressions:

```
Avoid: Repeated expensive calculations
expression: "#complexCalc() + #complexCalc() * 0.1"

Better: Calculate once and reuse
calculations:
 - field: "baseValue"
 expression: "#complexCalc()"
 - field: "finalValue"
 expression: "#baseValue + #baseValue * 0.1"
```

## 15.2 Maintainability

### Naming Conventions

Use consistent, descriptive names:

```
Good naming conventions
rules:
 - id: "trade-id-required" # kebab-case for IDs
 name: "Trade ID Required" # Title Case for names

enrichments:
 - id: "lei-enrichment" # descriptive, specific
 field: "counterparty.lei" # clear field paths

calculations:
 - field: "tradeValueUSD" # camelCase for calculated fields
 expression: "#quantity * #price"
```

### Documentation Standards

Document complex logic:

```
enrichments:
 - id: "complex-risk-calculation"
 type: "calculation-enrichment"
 # Purpose: Calculate portfolio risk metrics according to Basel III
requirements
 # Input: position data with market values and volatilities
 # Output: VaR, expected shortfall, and risk-weighted assets
 condition: "#positions != null && #positions.size() > 0"
 calculations:
 # Calculate 1-day VaR at 99% confidence level
 - field: "var1Day99"
 expression: "#portfolioValue * 0.025" # 2.5% VaR multiplier

 # Scale to 10-day VaR using square root of time rule
```

```
- field: "var10Day99"
 expression: "#var1Day99 * T(java.lang.Math).sqrt(10)"
```

## 15.3 Error Handling

### Graceful Degradation

Handle missing or invalid data gracefully:

```
Provide defaults for missing data
calculations:
- field: "effectiveRate"
 expression: "#customRate != null ? #customRate : #standardRate"

- field: "safeCalculation"
 expression: "#denominator != null && #denominator != 0 ? #numerator /
#denominator : 0"
```

### Null Safety

Always check for null values:

```
Safe navigation
condition: "#trade?.security?.instrumentId != null"

Explicit null checks
condition: "#trade != null && #trade.security != null &&
#trade.security.instrumentId != null"

Safe string operations
expression: "#text != null && #text.trim().length() > 0 ? #text.toUpperCase() :
'UNKNOWN'"
```

---

## 16. Common Patterns

### 16.1 Financial Services Patterns

#### Reference Data Enrichment Pattern

Standard pattern for enriching with reference data:

```
enrichments:
- id: "security-master-enrichment"
 type: "lookup-enrichment"
 condition: "#instrumentId != null"
```

```
lookup-config:
 lookup-key: "instrumentId"
 lookup-dataset:
 type: "external"
 source: "security-master"
 key-field: "isin"
 field-mappings:
 - source-field: "name"
 target-field: "security.name"
 - source-field: "assetClass"
 target-field: "security.assetClass"
 - source-field: "currency"
 target-field: "security.currency"
```

## Risk Calculation Pattern

Standard risk metrics calculation:

```
enrichments:
 - id: "risk-metrics"
 type: "calculation-enrichment"
 condition: "#marketValue != null"
 calculations:
 # Value at Risk calculations
 - field: "var1Day95"
 expression: "#marketValue * 0.0164" # 1.64 * volatility
 - field: "var1Day99"
 expression: "#marketValue * 0.0233" # 2.33 * volatility
 - field: "var10Day99"
 expression: "#var1Day99 * T(java.lang.Math).sqrt(10)"

 # Risk classification
 - field: "riskLevel"
 expression: "#var1Day99 > 1000000 ? 'HIGH' : (#var1Day99 > 100000 ?
'MEDIUM' : 'LOW')"
```

## Regulatory Compliance Pattern

Standard regulatory field generation:

```
enrichments:
 - id: "regulatory-fields"
 type: "calculation-enrichment"
 calculations:
 # UTI generation
 - field: "regulatory.uti"
 expression: "#reportingEntity.lei + '-' + #tradeId + '-' +
T(java.time.LocalDate).now().format(T(java.time.format.DateTimeFormatter).ofPatter
n('yyyyMMdd'))"
```

```
Jurisdiction flags
- field: "regulatory.emirApplicable"
 expression: "#counterparty.jurisdiction == 'EU'"
- field: "regulatory.mifidApplicable"
 expression: "#venue.country == 'GB' || #venue.country == 'DE' ||
#venue.country == 'FR'"
```

## 9.2 Data Validation Patterns

### Format Validation Pattern

Standard format validation approach:

```
rules:
- id: "isin-format"
 name: "ISIN Format Validation"
 condition: "#isin == null || #isin.matches('^[A-Z]{2}[A-Z0-9]{9}[0-9]$')"
 message: "ISIN must be 12 characters: 2 letters + 9 alphanumeric + 1 digit"
 severity: "ERROR"

- id: "lei-format"
 name: "LEI Format Validation"
 condition: "#lei == null || #lei.matches('^[A-Z0-9]{18}[0-9]{2}$')"
 message: "LEI must be 20 characters: 18 alphanumeric + 2 check digits"
 severity: "ERROR"
```

### Business Rule Validation Pattern

Standard business rule validation:

```
rules:
- id: "settlement-date-business-rule"
 name: "Settlement Date Must Be Business Day"
 condition: "#settlementDate == null ||
T(com.company.utils.BusinessDayUtils).isBusinessDay(#settlementDate,
#market.country)"
 message: "Settlement date must be a business day in the market country"
 severity: "ERROR"

- id: "trade-limit-check"
 name: "Trade Limit Validation"
 condition: "#tradeValue <= #counterparty.creditLimit"
 message: "Trade value exceeds counterparty credit limit"
 severity: "ERROR"
```

### Cross-Field Validation Pattern

Validate relationships between fields:

```
rules:
 - id: "settlement-after-trade-date"
 name: "Settlement Date After Trade Date"
 condition: "#tradeDate == null || #settlementDate == null ||
#settlementDate.isAfter(#tradeDate)"
 message: "Settlement date must be after trade date"
 severity: "ERROR"

 - id: "currency-consistency"
 name: "Currency Consistency Check"
 condition: "#security.currency == null || #trade.currency == null ||
#security.currency == #trade.currency"
 message: "Security currency must match trade currency"
 severity: "WARNING"
```

## 17. Examples & Use Cases

### 17.1 Simple REST API Lookup Example

This is a complete, working example based on the [SimpleRestApiYamlTest](#):

```
Simple REST API YAML Test Configuration
The simplest possible YAML for REST API lookup validation

data-sources:
 - name: "test-api"
 type: "rest-api"
 connection:
 base-url: "http://localhost:8080"
 endpoints:
 currency-lookup: "/api/currency/{key}"
 cache:
 enabled: true # Enable caching
 ttlSeconds: 600 # 10 minutes TTL
 maxIdleSeconds: 300 # 5 minutes max idle time
 maxSize: 1000 # Max 1000 entries (LRU eviction)
 keyPrefix: "test" # Cache key prefix

enrichments:
 - id: "simple-lookup"
 type: "lookup-enrichment"
 condition: "#currencyCode != null"
 lookup-config:
 lookup-key: "#currencyCode"
 lookup-dataset:
 type: "rest-api"
 data-source-ref: "test-api"
```

```
 operation-ref: "currency-lookup"
 field-mappings:
 - source-field: "name"
 target-field: "currencyName"
 required: false
```

Processing Flow:

- 1. Input data: {currencyCode: "USD"}
- 2. **Cache Check:** Look for cached result with key test:rest-api:USD:currency-lookup
- 3. Condition #currencyCode != null evaluates to true
- 4. Lookup key #currencyCode extracts "USD"
- 5. URL /api/currency/{key} becomes /api/currency/USD
- 6. HTTP GET request to http://localhost:8080/api/currency/USD (if not cached)
- 7. Response: {"code": "USD", "name": "US Dollar", "rate": 1.0, "symbol": "\$"}
- 8. **Cache Store:** Store result with TTL and idle time tracking
- 9. Field mapping: name → currencyName
- 10. Result: {currencyCode: "USD", currencyName: "US Dollar"}

12.2 Data Source Cache Configuration Reference

APEX provides **comprehensive caching** for all data source types. Here are the **actually implemented** cache features:

Supported Cache Properties

```
data-sources:
 - name: "cached-source"
 type: "rest-api" # Works with: rest-api, database, file-system
 cache:
 enabled: true # Enable/disable caching (default: true)
 ttlSeconds: 600 # Time-to-live in seconds (default: 3600)
 maxIdleSeconds: 300 # Max idle time in seconds (default: 0 = disabled)
 maxSize: 1000 # Maximum cache entries (default: 10000)
 keyPrefix: "api" # Cache key prefix (default: empty)
```

Cache Feature Details

Property	Description	Default	Implementation
enabled	Enable/disable caching	true	Fully Implemented
ttlSeconds	Time-to-live expiration	3600 (1 hour)	Fully Implemented
maxIdleSeconds	Max idle time expiration	0 (disabled)	Fully Implemented
maxSize	Maximum cache entries	10000	Fully Implemented
keyPrefix	Cache key namespace prefix	"" (empty)	Fully Implemented

## Advanced Cache Features

- **LRU Eviction:** When `maxSize` is reached, least recently used entries are automatically evicted
- **Dual Expiration:** Entries expire based on both TTL (`ttlSeconds`) and idle time (`maxIdleSeconds`)
- **Thread Safety:** All cache operations are thread-safe with concurrent access support
- **Cache Statistics:** Built-in metrics for hit/miss ratios, eviction counts, and performance monitoring
- **Key Generation:** Automatic cache key generation with optional prefixing to prevent collisions

## 12.3 Additional Examples

### Basic Lookup Example

Simple counterparty LEI lookup:

```
metadata:
 name: "Simple LEI Lookup"
 version: "1.0.0"
 type: "rule-config"

enrichments:
 - id: "lei-lookup"
 type: "lookup-enrichment"
 condition: "#counterpartyName != null"
 lookup-config:
 lookup-key: "counterpartyName"
 lookup-dataset:
 type: "inline"
 key-field: "name"
 data:
 - name: "Deutsche Bank AG"
 lei: "7LTWFZYICNSX8D621K86"
 - name: "JPMorgan Chase"
 lei: "8EE8DF3643E15DBFDA05"
 field-mappings:
 - source-field: "lei"
 target-field: "counterpartyLEI"
```

### Basic Calculation Example

Simple trade value calculation:

```
metadata:
 name: "Trade Value Calculation"
 version: "1.0.0"
 type: "rule-config"

enrichments:
 - id: "trade-value"
 type: "calculation-enrichment"
```

```

condition: "#quantity != null && #price != null"
calculations:
 - field: "tradeValue"
 expression: "#quantity * #price"
 - field: "commission"
 expression: "#tradeValue * 0.001" # 0.1% commission
 - field: "netAmount"
 expression: "#tradeValue + #commission"

```

## Basic Validation Example

Simple field validation:

```

metadata:
 name: "Basic Validation"
 version: "1.0.0"
 type: "rule-config"

rules:
 - id: "required-fields"
 name: "Required Fields Validation"
 condition: "#tradeId != null && #counterpartyName != null && #instrumentId != null"
 message: "Trade ID, counterparty name, and instrument ID are required"
 severity: "ERROR"
 priority: 1

```

## 10.2 Complex Examples

### Multi-Step Enrichment Example

Complex enrichment with multiple dependencies:

```

metadata:
 name: "Complex Settlement Enrichment"
 version: "1.0.0"
 type: "rule-config"

enrichments:
 # Step 1: Enrich counterparty data
 - id: "counterparty-enrichment"
 type: "lookup-enrichment"
 condition: "#counterpartyName != null"
 lookup-config:
 lookup-key: "counterpartyName"
 lookup-dataset:
 type: "inline"
 key-field: "name"
 data:

```

```

 - name: "Deutsche Bank AG"
 lei: "7LTFZYICNSX8D621K86"
 jurisdiction: "DE"
 creditRating: "A1"
 field-mappings:
 - source-field: "lei"
 target-field: "counterparty.lei"
 - source-field: "jurisdiction"
 target-field: "counterparty.jurisdiction"
 - source-field: "creditRating"
 target-field: "counterparty.creditRating"

Step 2: Calculate trade metrics
- id: "trade-calculations"
 type: "calculation-enrichment"
 condition: "#quantity != null && #price != null"
 calculations:
 - field: "tradeValue"
 expression: "#quantity * #price"
 - field: "tradeValueUSD"
 expression: "#currency == 'USD' ? #tradeValue : #tradeValue * #fxRate"

Step 3: Determine settlement instructions based on enriched data
- id: "settlement-instructions"
 type: "lookup-enrichment"
 condition: "#counterparty.lei != null && #venue.country != null"
 lookup-config:
 lookup-key: "#counterparty.lei + '_' + #venue.country"
 lookup-dataset:
 type: "inline"
 key-field: "key"
 data:
 - key: "7LTFZYICNSX8D621K86_GB"
 method: "CREST"
 account: "CREST001234"
 - key: "7LTFZYICNSX8D621K86_US"
 method: "DTC"
 account: "DTC567890"
 field-mappings:
 - source-field: "method"
 target-field: "settlement.method"
 - source-field: "account"
 target-field: "settlement.account"

Step 4: Calculate fees based on trade value and counterparty rating
- id: "fee-calculations"
 type: "calculation-enrichment"
 condition: "#tradeValueUSD != null && #counterparty.creditRating != null"
 calculations:
 - field: "commissionRate"
 expression: "#counterparty.creditRating.startsWith('A') ? 0.0005 : 0.001"

Premium rate for A-rated
- field: "commission"
 expression: "#tradeValueUSD * #commissionRate"

```

```
- field: "clearingFee"
 expression: "#tradeValueUSD * 0.0001" # 1 bp clearing fee
- field: "totalFees"
 expression: "#commission + #clearingFee"
- field: "netSettlementAmount"
 expression: "#tradeValueUSD + #totalFees"
```

---

## 18. Troubleshooting

### 18.1 Common Errors

#### Syntax Errors

##### Missing field reference prefix:

```
Wrong - no # prefix
condition: "quantity > 0"

Correct - direct field reference
condition: "#quantity > 0"
```

##### Incorrect field access:

```
Wrong - using # prefix in lookup-key
lookup-key: "#counterparty.name"

Correct - no # prefix in lookup-key
lookup-key: "counterparty.name"
```

##### Invalid SpEL syntax:

```
Wrong - invalid operator
condition: "#value = 100"

Correct - use == for comparison
condition: "#value == 100"
```

#### Runtime Errors

##### NullPointerException:

```
Problematic - can throw NPE
expression: "#trade.security.instrumentId.substring(0, 2)"
```

```
Safe - use null checks
expression: "#trade?.security?.instrumentId != null ?
#trade.security.instrumentId.substring(0, 2) : null"
```

### Type conversion errors:

```
Problematic - string to number conversion
expression: "#stringValue + 100"

Safe - explicit conversion with validation
expression: "#stringValue != null && #stringValue.matches('[0-9]+') ?
T(java.lang.Integer).parseInt(#stringValue) + 100 : 100"
```

### Performance Issues

#### Expensive operations in conditions:

```
Problematic - expensive operation repeated
condition: "#expensiveCalculation() > 0 && #expensiveCalculation() < 1000"

Better - calculate once
calculations:
 - field: "calculationResult"
 expression: "#expensiveCalculation()"
 - field: "isValid"
 expression: "#calculationResult > 0 && #calculationResult < 1000"
```

## 11.2 Debugging Techniques

### Expression Testing

Test expressions in isolation:

```
Add debug calculations to test expressions
calculations:
 - field: "debug.inputQuantity"
 expression: "#quantity"
 - field: "debug.inputPrice"
 expression: "#price"
 - field: "debug.multiplication"
 expression: "#quantity * #price"
 - field: "debug.finalResult"
 expression: "#debug.multiplication"
```

Logging Strategies

Add logging fields for troubleshooting:

```
calculations:
 - field: "log.processingTimestamp"
 expression: "T(java.time.Instant).now().toString()"
 - field: "log.inputSummary"
 expression: "'Processing trade: ' + #tradeId + ' for ' + #counterpartyName"
 - field: "log.calculationDetails"
 expression: "'Quantity: ' + #quantity + ', Price: ' + #price + ', Result: ' +
(#quantity * #price)"
```

19. Reference

19.1 Syntax Quick Reference

Operators Table

Operator	Description	Example
==	Equality	#status == 'ACTIVE'
!=	Inequality	#quantity != 0
>, >=	Greater than	#price > 100
<, <=	Less than	#discount < 0.1
&&	Logical AND	#isActive && #quantity > 0
\\ \\	Logical OR	#status == 'PENDING' \\ \\  #status == 'PROCESSING'
!	Logical NOT	!#isDeleted
?:	Ternary	#value > 0 ? 'POSITIVE' : 'NEGATIVE'
?.	Safe navigation	#trade?.security?.instrumentId
+	Addition/Concatenation	#quantity + #bonus
-	Subtraction	#total - #discount
*	Multiplication	#quantity * #price
/	Division	#amount / #rate
%	Modulo	#value % 10

Function Reference

String Functions:

```

#text.toUpperCase() # Convert to uppercase
#text.toLowerCase() # Convert to lowercase
#text.trim() # Remove whitespace
#text.substring(0, 10) # Extract substring
#text.length() # Get string length
#text.contains('substring') # Check if contains
#text.startsWith('prefix') # Check if starts with
#text.endsWith('suffix') # Check if ends with
#text.matches('regex') # Regex match
#text.replace('old', 'new') # Replace text

```

### Math Functions:

```

T(java.lang.Math).max(a, b) # Maximum of two values
T(java.lang.Math).min(a, b) # Minimum of two values
T(java.lang.Math).abs(value) # Absolute value
T(java.lang.Math).sqrt(value) # Square root
T(java.lang.Math).pow(base, exp) # Power
T(java.lang.Math).round(value) # Round to nearest integer
T(java.lang.Math).ceil(value) # Round up
T(java.lang.Math).floor(value) # Round down

```

### Date Functions:

```

T(java.time.LocalDate).now() # Current date
T(java.time.Instant).now().toString() # Current timestamp
#date.plusDays(2) # Add days
#date.minusMonths(1) # Subtract months
#date.isAfter(otherDate) # Date comparison
#date.format(T(java.time.format.DateTimeFormatter).ofPattern('yyyy-MM-dd')) #
Format date

```

## 12.2 SpEL Integration

### Supported SpEL Features

APEX YAML supports these SpEL features:

- **Literal expressions:** 'Hello World', 123, true
- **Property access:** #property, #nested.property
- **Method invocation:** #text.toUpperCase()
- **Operators:** Arithmetic, comparison, logical, ternary
- **Variables:** #root, #this, custom variables
- **Collection operations:** #list[0], #list.size()
- **Type references:** T(java.lang.Math).max(a, b)
- **Safe navigation:** #optional?.property

## APEX-Specific Extensions

APEX adds these extensions to standard SpEL:

- **Field references:** Direct field access in lookup keys
- **Enrichment chaining:** Reference fields created by previous enrichments
- **Null-safe operations:** Enhanced null safety beyond standard SpEL

## Limitations and Constraints

### Not supported:

- **Variable assignment:** Cannot create new variables (except in calculations)
- **Loops:** No for/while loop constructs
- **Complex object creation:** Limited to simple expressions
- **File I/O:** No direct file system access
- **Network operations:** No direct HTTP/network calls

### Performance constraints:

- **Expression complexity:** Keep expressions reasonably simple
- **Recursion:** Avoid recursive expressions
- **Memory usage:** Large datasets should use external sources
- **Patch versions** (1.1.1 → 1.1.2): Bug fixes, fully compatible

## Migration Strategies

### From Version 1.0 to 1.1

No breaking changes, but new features available:

```
New in 1.1: Enhanced error handling
rules:
 - id: "example-rule"
 name: "Example Rule"
 condition: "#field != null"
 message: "Field is required"
 severity: "ERROR"
 # New in 1.1: Custom error codes
 error-code: "FIELD_REQUIRED"
 # New in 1.1: Retry configuration
 retry-on-failure: true
```

## Deprecated Features

### Version 1.0 deprecated syntax:

```
Deprecated: Old action syntax
actions:
 - type: "lookup"
 source: "dataset"

Current: New enrichment syntax
enrichments:
 - type: "lookup-enrichment"
 lookup-config:
 lookup-dataset:
 type: "inline"
```

## From Version 1.x to 2.0 - External Data-Source References

**APEX 2.0** introduces external data-source references for clean architecture:

### Legacy Approach (1.x):

```
Old: Inline data-source configuration
metadata:
 name: "Legacy Configuration"
 version: "1.0.0"

data-sources:
 - name: "customer-database"
 type: "database"
 connection:
 url: "jdbc:postgresql://localhost:5432/customers"
 username: "user"
 password: "pass"
 queries:
 getCustomer:
 sql: "SELECT * FROM customers WHERE id = :id"

enrichments:
 - id: "customer-lookup"
 type: "lookup-enrichment"
 lookup-config:
 lookup-dataset:
 type: "database"
 data-source: "customer-database"
 query: "getCustomer"
```

### Modern Approach (2.0):

```
New: External data-source references
metadata:
 name: "Modern Configuration"
```

```
version: "2.0.0"

Clean separation: Infrastructure references
data-source-refs:
 - name: "customer-database"
 source: "data-sources/customer-database.yaml" # External file
 enabled: true

Clean separation: Business logic only
enrichments:
 - id: "customer-lookup"
 type: "lookup-enrichment"
 lookup-config:
 lookup-dataset:
 type: "database"
 data-source-ref: "customer-database" # Reference to external config
 query-ref: "getCustomer" # Named query from external config
```

### Migration Benefits:

- **Clean Architecture:** Infrastructure and business logic separated
- **Reusable Components:** External configurations shared across multiple rules
- **Configuration Caching:** External configurations cached for performance
- **Enterprise Scalability:** Environment-specific infrastructure configurations

### Future Roadmap

#### Planned features:

- **Enhanced debugging:** Better error messages and debugging tools
- **Performance optimizations:** Improved expression evaluation
- **Extended functions:** More built-in functions and utilities
- **IDE integration:** Better tooling support
- **Schema validation:** Runtime schema validation
- **Advanced external references:** Support for more external data-source types

---

## Conclusion

This APEX YAML Syntax Reference provides comprehensive guidance for creating maintainable, efficient, and robust business rules and enrichment logic. APEX 2.0's **external data-source reference system** enables enterprise-grade clean architecture with separation of concerns.

The key to success with APEX YAML is:

1. **Start simple:** Begin with basic patterns and gradually add complexity
2. **Use external references:** Leverage external data-source references for clean architecture
3. **Follow best practices:** Use proper naming, error handling, and performance optimization
4. **Test thoroughly:** Validate your configurations with comprehensive test data
5. **Document well:** Add comments and maintain clear, readable configurations

6. **Monitor performance:** Keep track of execution times and optimize as needed
7. **Separate concerns:** Keep infrastructure and business logic configurations separate

For additional support and examples, refer to the APEX documentation and community resources.

---

## Appendix A: Implemented APEX Keyword Dictionary

This appendix provides an alphabetical reference of **functionally implemented** APEX YAML keywords that affect system behavior during rule/enrichment execution.

**Note:** Keywords marked as "Metadata Only" are stored but do not affect execution logic. See Appendix C for planned/future keywords.

### A

- **accumulator** - Initial value for accumulative chaining
- **accumulation-rules** - List of rules for accumulative chaining
- **actions** - List of actions to execute when a rule matches or condition is met
- **actions-false** - Actions to execute when transformation rule condition evaluates to false (alias for else-actions)
- **actions-true** - Actions to execute when transformation rule condition evaluates to true (takes precedence over actions)
- **alert-on-failure** - Boolean flag to trigger alerts when processing fails (used in health checks and pipeline monitoring)
- **authentication** - Authentication configuration for external data sources (username, password, tokens)

### B

- **backoff-multiplier** - Multiplier for exponential backoff in retry logic
- **batch** - Batch processing configuration for handling multiple records
- **builder-target** - Target object for fluent builder pattern
- **business-domain** - Business domain classification (e.g., "Trading", "Compliance", "Risk")
- **business-owner** - Business owner responsible for the rule or configuration

### C

- **cache** - Caching configuration for data sources (TTL, max size, eviction policy)
- **cache-enabled** - Boolean flag to enable/disable caching
- **cache-ttl-seconds** - Cache time-to-live in seconds
- **calculation-config** - Configuration for calculation-enrichment type
- **categories** - List of category definitions or category assignments
- **category** - Single category for rule/enrichment classification
- **circuit-breaker** - Circuit breaker configuration for resilience (failure threshold, timeout) - fully implemented in RestApiDataSource
- **collect-metrics** - Boolean flag to enable metrics collection in pipelines and data sources
- **component-refs** - References to other component files
- **condition** - SpEL expression defining when rule/enrichment/stage applies
- **conditional-mappings** - List of conditional field mapping configurations

- **conditional-rules** - Rules to execute based on trigger result in conditional chaining
- **conditions** - Multiple conditions for complex logic
- **config-files** - List of configuration files with execution order and failure policy
- **configuration** - General configuration object
- **connection** - Database or external system connection configuration (URL, credentials)
- **connection-name** - Name identifier for a connection
- **custom-properties** - Custom extensible properties for rules or enrichments (stored but not processed)
- **custom-validators** - Custom validation logic references (defined but not fully implemented)

## D

- **data** - Data payload or data object
- **data-sinks** - Output destinations for processed data
- **data-source-ref** - Reference to an external data source configuration
- **data-source-refs** - List of references to external data source configurations
- **data-sources** - Inline data source definitions
- **debug-mode** - Enable debug mode for detailed logging and execution traces
- **decision-rule** - Final decision rule for accumulative chaining
- **default-strategy** - Default strategy for error recovery or processing
- **default-value** - Fallback value when source field is missing or null
- **default-values** - Multiple default values for different scenarios
- **delay-ms** - Delay in milliseconds before retry or execution
- **dependencies** - List of dependencies for execution order
- **depends-on** - List of stage or component dependencies
- **description** - Human-readable description of the configuration
- **display-name** - Display name for UI presentation
- **document-position** - Position of item in YAML document for ordering
- **documentation-url** - URL to external documentation
- **drop-if-exists** - Drop existing database tables or resources before creating

## E

- **effective-date** - Date when rule becomes effective (ISO 8601)
- **else-actions** - Actions to execute when condition is false
- **enabled** - Boolean flag indicating if rule/enrichment/component is active
- **endpoint** - Single endpoint URL for REST API
- **endpoints** - Map of REST API endpoint definitions
- **enrichment-group** - Singular reference to another enrichment group for hierarchical composition
- **enrichment-group-references** - References to other enrichment groups (plural form)
- **enrichment-groups** - Enrichment group definitions
- **enrichment-id** - Unique identifier for an enrichment
- **enrichment-ids** - List of enrichment IDs in a group
- **enrichment-references** - Structured references to enrichments with sequence and overrides
- **enrichment-refs** - References to external enrichment configuration files
- **enrichments** - Data enrichment configurations
- **environment** - Environment identifier (e.g., "dev", "test", "prod")
- **error-code** - Error code to set when enrichment or rule fails

- **error-handling** - Exception handling strategy (fail-fast, continue-on-error, skip-on-error)
- **error-recovery** - Error recovery configuration for resilience and fault tolerance
- **execution** - Execution configuration object
- **execution-config** - Execution behavior configuration
- **execution-order** - Numeric execution order for stages or file references
- **execution-settings** - Execution behavior configuration for enrichments
- **expiration-date** - Date when rule expires (ISO 8601)
- **expression** - SpEL expression for field transformation or calculation

## F

- **failure-policy** - Failure handling policy (terminate, continue-with-warnings, flag-for-review)
- **fallback-value** - Fallback value when primary value is unavailable
- **field** - Field name for operations
- **field-mappings** - List of field mapping configurations
- **field-types** - Expected data types for field validation
- **file** - Path to configuration file in component file reference
- **file-format** - File format configuration (CSV, JSON, XML)
- **file-path** - Path to file resource
- **format-config** - Format configuration for file parsing

## H

- **health-check** - Health check configuration for data sources

## I

- **id** - Unique identifier for the configuration
- **implementation** - Implementation class for custom data sources or validators
- **init-script** - Initialization script for database setup
- **init-scripts** - List of initialization scripts

## K

- **key-field** - Field name used as key for lookups
- **key-patterns** - Key pattern definitions for key-value stores
- **kind** - Kubernetes-style resource kind

## L

- **labels** - Key-value labels for classification
- **last-modified** - Last modification timestamp (ISO 8601)
- **log-matched-rule** - Boolean flag to log which rule matched
- **log-progress** - Boolean flag to log processing progress
- **log-recovery-attempts** - Boolean flag to log error recovery attempts
- **lookup-config** - Configuration for lookup-enrichment type
- **lookup-dataset** - Dataset configuration for lookup operations
- **lookup-key** - SpEL expression for lookup key
- **lookup-service** - Service name for lookup operations

## M

- **map-to-field** - Target field(s) for mapping results (String or List)
- **mapping** - Single mapping configuration
- **mapping-rules** - Complex mapping rule definitions
- **max-attempts** - Maximum number of retry attempts
- **max-delay-ms** - Maximum delay in milliseconds for backoff
- **max-retries** - Maximum number of retries
- **message** - Message displayed when rule is triggered or for logging
- **metadata** - Document metadata section containing identification and configuration
- **metrics-enabled** - Boolean flag to enable metrics collection
- **mode** - Processing mode (e.g., "sequential", "parallel")
- **monitoring** - Monitoring configuration

## N

- **name** - Human-readable name for the configuration

## O

- **on-failure** - Rules to execute on failure in fluent builder pattern
- **on-no-trigger** - Rules to execute if trigger rule fails in conditional chaining
- **on-success** - Rules to execute on success in fluent builder pattern
- **on-trigger** - Rules to execute if trigger rule matches in conditional chaining
- **operation** - Single operation definition
- **operation-ref** - Reference to a named operation
- **operations** - Map of operation definitions for REST APIs
- **operator** - Logical operator for rule group (AND/OR)
- **optional** - Boolean flag indicating if field or configuration is optional
- **output-format** - Output format specification
- **output-variable** - Variable to store result in sequential dependency pattern
- **override-priority** - Override priority for rule within group
- **owner** - Owner of the component or configuration

## P

- **parallel-execution** - Enable parallel execution of rules in group
- **parameter-names** - Array of parameter names for parameterized queries
- **parameters** - Map of parameters for operations or queries
- **parent-category** - Parent category for hierarchical categorization
- **pattern** - Regular expression pattern for validation
- **pipeline** - Pipeline configuration for processing stages
- **priority** - Execution priority (lower numbers = higher priority)
- **processing-mode** - Processing mode (e.g., "document-order", "priority-order")

## Q

- **queries** - Named query definitions for database sources
- **query** - Single query definition

- **query-ref** - Reference to a named query

## R

- **recovery-enabled** - Boolean flag to enable error recovery
- **required** - Boolean flag indicating if field mapping is mandatory
- **required-fields** - List of required fields for validation
- **response-mapping** - Response transformation configuration for REST APIs
- **result-field** - Field name to store rule or enrichment evaluation result
- **retry** - Retry configuration object
- **retry-count** - Number of retry attempts
- **retry-delay** - Delay between retry attempts
- **retry-delay-ms** - Retry delay in milliseconds
- **routes** - Map of routes for result-based routing
- **routing-rule** - Rule to determine execution route
- **rule** - Single rule definition
- **rule-chains** - Rule chain definitions for sequential rule execution
- **rule-configurations** - References to rule configuration files in components
- **rule-group-references** - References to other rule groups
- **rule-groups** - Rule group definitions
- **rule-id** - ID of rule being referenced
- **rule-ids** - List of rule IDs in the group
- **rule-references** - Detailed rule references with metadata
- **rule-refs** - References to external rule configurations
- **rules** - Rule definitions

## S

- **scenario** - Scenario configuration for multi-stage processing
- **schema** - Schema definition for data validation
- **sequence** - Execution sequence for rule within group
- **severity** - Severity level (CRITICAL, ERROR, WARNING, INFO)
- **severity-policies** - Severity-based policy configurations
- **sink** - Single data sink configuration
- **sla-ms** - Service level agreement in milliseconds
- **source** - Source identifier or configuration
- **source-field** - Source field name in field mappings
- **source-system** - Source system identifier for audit trails
- **source-type** - Type of data source (database, rest-api, file, cache, kafka)
- **spec** - Kubernetes-style specification object
- **stages** - List of stages for sequential dependency pattern
- **steps** - List of processing steps
- **stop-on-first-failure** - Stop group execution on first rule failure
- **stop-on-first-match** - Stop processing on first matching rule
- **strategy** - Strategy configuration (e.g., recovery strategy, execution strategy)
- **success-code** - Success code to set when enrichment or rule succeeds

## T

- **tags** - Classification tags for the configuration
- **target-field** - Target field name in field mappings
- **target-type** - Target object type for enrichment or transformation
- **timeout-ms** - Timeout in milliseconds
- **topics** - Kafka topic definitions
- **transformation-rules** - Transformation rule definitions
- **transformations** - Data transformation configurations
- **trigger-rule** - Rule to trigger conditional chaining
- **type** - Document type (rule-config, enrichment, dataset, scenario, external-data-config, component)

## V

- **validate-result** - Boolean flag to validate processing result
- **validation** - Validation configuration for rules
- **value** - Static value to set or use
- **version** - Version identifier for the configuration

## W

- **weight** - Weight for accumulative chaining rules

---

**Note:** This dictionary includes all @JsonProperty annotations found in apex-core. Some keywords are nested properties within parent objects (e.g., **cache-enabled** is a property within **cache**). Refer to the main reference sections for complete usage examples and context.

---

## Appendix B: Keywords Organized by APEX Category

This appendix organizes APEX keywords by their functional category for easier navigation.

### Document Structure Keywords

Top-level sections that define the structure of APEX YAML documents:

- **metadata** - Document metadata section
- **data-sources** - Inline data source definitions
- **data-source-refs** - References to external data source configurations
- **enrichments** - Data enrichment configurations
- **enrichment-groups** - Enrichment group definitions
- **enrichment-refs** - References to external enrichment files
- **rules** - Rule definitions
- **rule-groups** - Rule group definitions
- **rule-refs** - References to external rule configurations
- **rule-chains** - Rule chain definitions for sequential execution
- **transformations** - Data transformation configurations
- **categories** - Category definitions for classification
- **data-sinks** - Output destinations for processed data
- **pipeline** - Pipeline configuration for multi-stage processing

- **error-recovery** - Error recovery configuration
- **scenario** - Scenario configuration
- **component-refs** - References to component files
- **config-files** - List of configuration files

## Metadata Keywords

Keywords used in the **metadata** section for document identification:

- **id** - Unique identifier
- **name** - Human-readable name
- **description** - Detailed description
- **version** - Version identifier
- **type** - Document type (rule-config, enrichment, dataset, scenario, external-data-config, component)
- **author** - Author or creator
- **created** - Creation timestamp
- **last-modified** - Last modification timestamp
- **tags** - Classification tags
- **processing-mode** - Processing mode (document-order, priority-order)
- **environment** - Environment identifier (dev, test, prod)
- **business-domain** - Business domain classification
- **business-owner** - Business owner responsible
- **created-by** - Creator identifier
- **source-system** - Source system identifier
- **effective-date** - Date when configuration becomes effective
- **expiration-date** - Date when configuration expires
- **documentation-url** - URL to external documentation
- **display-name** - Display name for UI
- **criticality** - Criticality level (HIGH, MEDIUM, LOW)
- **owner** - Owner of the configuration
- **sla-ms** - Service level agreement in milliseconds

## Rule Keywords

Keywords specific to rule definitions:

- **rule** - Single rule definition
- **rule-id** - Rule identifier
- **rule-ids** - List of rule IDs
- **rule-references** - Detailed rule references with metadata
- **rule-configurations** - References to rule configuration files
- **condition** - SpEL expression for rule condition
- **conditions** - Multiple conditions
- **message** - Message when rule triggers
- **severity** - Severity level (CRITICAL, ERROR, WARNING, INFO)
- **priority** - Execution priority
- **enabled** - Enable/disable flag
- **category** - Category assignment

- **categories** - Multiple category assignments
- **validation** - Validation configuration
- **custom-properties** - Custom extensible properties
- **default-value** - Default value when condition fails
- **log-matched-rule** - Log which rule matched
- **stop-on-first-match** - Stop on first matching rule
- **result-field** - Field to store rule result

## Rule Group Keywords

Keywords for organizing rules into groups:

- **rule-group-references** - References to other rule groups
- **operator** - Logical operator (AND/OR)
- **stop-on-first-failure** - Stop on first failure
- **parallel-execution** - Enable parallel execution
- **execution-config** - Execution configuration
- **debug-mode** - Enable debug mode
- **sequence** - Execution sequence
- **override-priority** - Override priority for rule in group

## Enrichment Keywords

Keywords specific to enrichment definitions:

- **enrichment-id** - Enrichment identifier
- **enrichment-ids** - List of enrichment IDs
- **enrichment-references** - Structured enrichment references
- **enrichment-group-references** - References to enrichment groups
- **lookup-config** - Lookup enrichment configuration
- **calculation-config** - Calculation enrichment configuration
- **field-mappings** - Field mapping configurations
- **conditional-mappings** - Conditional field mappings
- **mapping-rules** - Complex mapping rules
- **target-field** - Target field name
- **source-field** - Source field name
- **target-type** - Target object type
- **lookup-key** - Lookup key expression
- **lookup-dataset** - Dataset for lookup
- **lookup-service** - Service for lookup
- **map-to-field** - Target field(s) for results
- **success-code** - Success code to set
- **error-code** - Error code to set
- **execution-settings** - Execution behavior configuration

## Transformation Keywords

Keywords for data transformations:

- **transformation-rules** - Transformation rule definitions
- **expression** - SpEL expression for transformation
- **actions** - Actions to execute (legacy, use actions-true for new configurations)
- **actions-true** - Actions to execute when condition is true (takes precedence over actions)
- **actions-false** - Actions to execute when condition is false (alias for else-actions)
- **else-actions** - Actions when condition is false (legacy, use actions-false for new configurations)
- **mapping** - Single mapping configuration
- **value** - Static value to set
- **field** - Field name for operations

### Conditional Transformation Actions (actions-true / actions-false)

The **actions-true** and **actions-false** keywords provide explicit conditional branching in transformation rules. These are the modern, preferred keywords for conditional transformations.

#### Keyword Precedence:

- When both **actions** and **actions-true** are specified, **actions-true** takes precedence
- **actions-false** is an alias for **else-actions** (both work identically)

#### Example: Basic Conditional Transformation

```
transformations:
- id: "priority-check"
 type: "conditional-transformation"
 transformation-rules:
 - condition: "#root['amount'] > 10000"
 actions-true:
 - type: "set-field"
 field: "priority"
 value: "high"
 actions-false:
 - type: "set-field"
 field: "priority"
 value: "normal"
```

#### Example: Nested Conditional Transformations

```
transformations:
- id: "complex-nesting"
 type: "conditional-transformation"
 transformation-rules:
 - condition: "#root['level1'] == true"
 actions-true:
 - type: "set-field"
 field: "l1_executed"
 value: true
 - type: "conditional-transformation"
 transformation-rules:
```

```

- condition: "#root['level2'] == true"
 actions-true:
 - type: "set-field"
 field: "l2_result"
 value: "true-path"
 actions-false:
 - type: "set-field"
 field: "l2_result"
 value: "false-path"

```

### Example: Multiple Sibling Rules

```

transformations:
- id: "sibling-rules"
 type: "conditional-transformation"
 transformation-rules:
 - condition: "#root['type'] == 'A'"
 actions-true:
 - type: "set-field"
 field: "mark"
 value: "A"
 - condition: "#root['value'] > 10"
 actions-true:
 - type: "set-field"
 field: "size"
 value: "big"

```

## Data Source Keywords

Keywords for configuring data sources:

- **source-type** - Type of data source (database, rest-api, file, cache, kafka)
- **connection** - Connection configuration
- **connection-name** - Connection identifier
- **data-source-ref** - Reference to external data source
- **queries** - Named query definitions
- **query** - Single query definition
- **query-ref** - Reference to named query
- **parameter-names** - Query parameter names
- **parameters** - Operation or query parameters
- **endpoints** - REST API endpoint definitions
- **endpoint** - Single endpoint URL
- **operations** - REST API operation definitions
- **operation** - Single operation definition
- **operation-ref** - Reference to named operation
- **response-mapping** - Response transformation
- **authentication** - Authentication configuration
- **health-check** - Health check configuration

- **circuit-breaker** - Circuit breaker configuration
- **topics** - Kafka topic definitions
- **file-path** - Path to file resource
- **file-format** - File format configuration
- **format-config** - Format parsing configuration
- **init-script** - Database initialization script
- **init-scripts** - List of initialization scripts
- **auto-create** - Auto-create tables/resources
- **auto-update** - Auto-update resources
- **drop-if-exists** - Drop existing resources

## Caching Keywords

Keywords for caching configuration:

- **cache** - Caching configuration
- **cache-enabled** - Enable/disable caching
- **cache-ttl-seconds** - Cache time-to-live

## Error Handling & Recovery Keywords

Keywords for error handling and resilience:

- **error-handling** - Exception handling strategy
- **error-recovery** - Error recovery configuration
- **failure-policy** - Failure handling policy
- **severity-policies** - Severity-based policies
- **retry** - Retry configuration
- **retry-count** - Number of retries
- **retry-delay** - Delay between retries
- **retry-delay-ms** - Retry delay in milliseconds
- **max-retries** - Maximum retry attempts
- **max-attempts** - Maximum attempts
- **backoff-multiplier** - Exponential backoff multiplier
- **delay-ms** - Delay in milliseconds
- **max-delay-ms** - Maximum delay for backoff
- **timeout-ms** - Timeout in milliseconds
- **alert-on-failure** - Trigger alerts on failure
- **recovery-enabled** - Enable error recovery
- **log-recovery-attempts** - Log recovery attempts
- **default-strategy** - Default recovery strategy
- **strategy** - Strategy configuration
- **fallback-value** - Fallback value when unavailable

## Execution Control Keywords

Keywords controlling execution behavior:

- **execution** - Execution configuration

- **execution-order** - Numeric execution order
- **depends-on** - Dependencies for execution order
- **dependencies** - List of dependencies
- **steps** - Processing steps
- **batch** - Batch processing configuration
- **log-progress** - Log processing progress
- **validate-result** - Validate processing result

## Component & Configuration Keywords

Keywords for component architecture:

- **configuration** - General configuration object
- **custom-validators** - Custom validation logic
- **implementation** - Implementation class reference
- **schema** - Schema definition
- **required** - Required field flag
- **optional** - Optional field flag
- **required-fields** - List of required fields
- **field-types** - Expected data types
- **pattern** - Regular expression pattern
- **default-values** - Multiple default values

## Monitoring & Metrics Keywords

Keywords for monitoring and observability:

- **metrics-enabled** - Enable metrics collection
- **collect-metrics** - Collect metrics flag
- **monitoring** - Monitoring configuration

## Data Sink Keywords

Keywords for output destinations:

- **sink** - Single data sink configuration
- **output-format** - Output format specification

## Miscellaneous Keywords

Other utility keywords:

- **data** - Data payload or object
- **document-position** - Position in YAML document
- **mode** - Processing mode
- **source** - Source identifier
- **key-field** - Key field for lookups
- **key-patterns** - Key pattern definitions

**Note:** Some keywords may appear in multiple categories as they serve different purposes in different contexts. For detailed usage examples, refer to the main reference sections.

---

## Appendix C: Planned/Future Keywords

This appendix lists keywords that are **defined in apex-core** (via @JsonProperty annotations) but are **NOT yet functionally implemented** in the execution logic. These keywords are stored in configuration objects but do not currently affect rule/enrichment execution behavior.

### Metadata-Only Keywords

These keywords are stored for audit, governance, and documentation purposes but do not affect execution:

#### Audit & Governance Metadata

- **author** - Author or creator of the configuration (stored but not checked during execution)
- **created** - Creation timestamp in ISO 8601 format (stored but not checked during execution)
- **created-by** - Creator identifier for audit trails (stored but not checked during execution)
- **last-modified** - Last modification timestamp (stored but not checked during execution)
- **owner** - Owner identifier (stored but not checked during execution)

#### Business Metadata

- **business-domain** - Business domain classification (e.g., "Trading", "Compliance") - stored and used for categorization/search in apex-yaml-manager, but does not affect rule execution
- **business-owner** - Business owner responsible for the rule (stored and used for search/filtering in apex-yaml-manager, but does not affect rule execution)
- **criticality** - Criticality level (e.g., "HIGH", "MEDIUM", "LOW") - stored but not checked during execution

#### Display & Documentation Metadata

- **display-name** - Display name for UI presentation (stored but not used in execution)
- **documentation-url** - URL to external documentation (stored but not used in execution)

### Date-Based Execution Control (Not Implemented)

These date fields are stored in metadata but are **NOT checked before rule execution**:

- **effective-date** - Date when rule becomes effective (ISO 8601 format)
  - **Status:** Stored in RuleMetadata, used in apex-yaml-manager for category filtering
  - **NOT implemented:** Rules execute regardless of effective date in apex-core RulesEngine
- **expiration-date** - Date when rule expires (ISO 8601 format)
  - **Status:** Stored in RuleMetadata, used in apex-yaml-manager for category filtering
  - **NOT implemented:** Rules execute regardless of expiration date in apex-core RulesEngine

**Future Implementation:** To make these functional, the RulesEngine.executeRule() and UnifiedRuleEvaluator.evaluateRule() methods would need to check these dates before executing rules.

## Database Management Keywords (Not Implemented)

These keywords are defined but database management features are not implemented:

- **auto-create** - Automatically create database tables or resources if they don't exist
  - **Status:** Defined in configuration classes but no implementation found
- **auto-update** - Automatically update resources when configuration changes
  - **Status:** Defined in configuration classes but no implementation found
- **drop-if-exists** - Drop existing database tables or resources before creating
  - **Status:** Defined in configuration classes but no implementation found

## Validation Keywords (Partially Implemented)

- **custom-validators** - Custom validation logic references
  - **Status:** Defined in configuration but custom validator invocation not fully implemented
  - **Current:** Basic validation exists, but extensible custom validators not supported
- **validate-result** - Validate enrichment/rule results
  - **Status:** Defined but comprehensive result validation not implemented

## Usage Notes

1. **Metadata Keywords:** These are useful for documentation, governance, and management tools (like apex-yaml-manager) but do not affect runtime behavior.
2. **Date-Based Control:** If you need date-based rule activation/deactivation, you must implement this in your rule conditions:

```
condition:
 "T(java.time.LocalDate).now().isAfter(T(java.time.LocalDate).parse('2025-01-01')) && #amount > 1000"
```

3. **Database Management:** For database table creation/management, use external database migration tools (Flyway, Liquibase) rather than expecting APEX to manage schema.
4. **Future Enhancements:** These keywords may be implemented in future versions of APEX. Check the release notes for updates.

---

## Summary of Keyword Counts

- **Appendix A (Implemented):** ~140 functionally implemented keywords
- **Appendix C (Planned/Future):** ~15 metadata-only keywords
- **Total Defined:** ~155 keywords in apex-core

**Note:** The original claim of "73 keywords" in line 41 was incorrect. The actual count is approximately 155 keywords defined in apex-core, with ~140 being functionally implemented in execution logic.